

Experiencia Educativa: Aprendizaje Máquina

Dr. David Martínez Galicia

Ingeniería en Ciberseguridad e Infraestructura de Cómputo
Facultad de Economía e Informática
Universidad Veracruzana



Presentación

- Ingeniero en Mecatrónica.
- Maestro en Inteligencia Artificial.
- Doctor en Inteligencia Artificial.



Información del curso

- **Objetivo de la experiencia:** Brindar los fundamentos para analizar grandes **conjuntos de datos**, identificar **anomalías** y **valores atípicos**, así como identificar rápidamente **tendencias** y **patrones**, con el **entramiento** de algoritmos de **aprendizaje automático**, en diferentes ámbitos asociados a la ciberseguridad.
- Miércoles (07:00 – 08:59) y jueves (07:00 – 08:59).
- Oportunidades de evaluación: ordinario, extraordinario y título (si aplica).
- Contacto: davidmartinez02@uv.mx



Criterios de evaluación

Criterio	Porcentaje
Exámenes	40%
Tareas	30%
Proyecto	30%
Total	100%

- Para la acreditación tanto en ordinario, extraordinario y título de suficiencia es necesario obtener un mínimo de 6 en todos los criterios de evaluación.
- Cumplir con los porcentajes de asistencia que indica el estatuto.



Saberes teóricos

Introducción	ST-01. Manejo de datos	ST-02. Aprendizaje supervisado
- Inteligencia Artificial (IA) - Definición de la IA - Prueba de Turing - Campos de la IA	- Estructurados y no estructurados - Etiquetados y no etiquetados - Inconsistencia de datos	- Redes neuronales - Redes bayesianas - Árboles de decisión
ST-03. Aprendizaje no supervisado	ST-04. Aprendizaje por refuerzo	ST-05. Evaluación de modelos
- K-means - Variantes de K-means	- Q-learning	- Precisión - Sensibilidad y especificidad - ROC - Validación cruzada - Distancia intra e inter clúster



Calendarización

Mes	Febrero			Marzo				Abril					Mayo				Jun.	
Semana	2°	3°	4°	1°	2°	3°	4°	1°	2°	3°	4°	5°	1°	2°	3°	4°	1°	
Introducción								SS										
ST-01																		
ST-02						18												
ST-03																		
ST-04																		
ST-05																		
Notación: SS = Semana Santa. Días de suspensión en blanco.																		



Nacimiento de la Inteligencia Artificial

- El concepto de Inteligencia Artificial (IA) se acuña en 1956.
- Surge como un campo de estudio en un taller en Dartmouth College.
 - *Cada aspecto de la inteligencia puede ser descrito con suficiente precisión que puede fabricarse una máquina para simularlo.*
- Muchas preguntas, pocas respuestas y aún más esperanzas.



Objeto de estudio de la Inteligencia Artificial

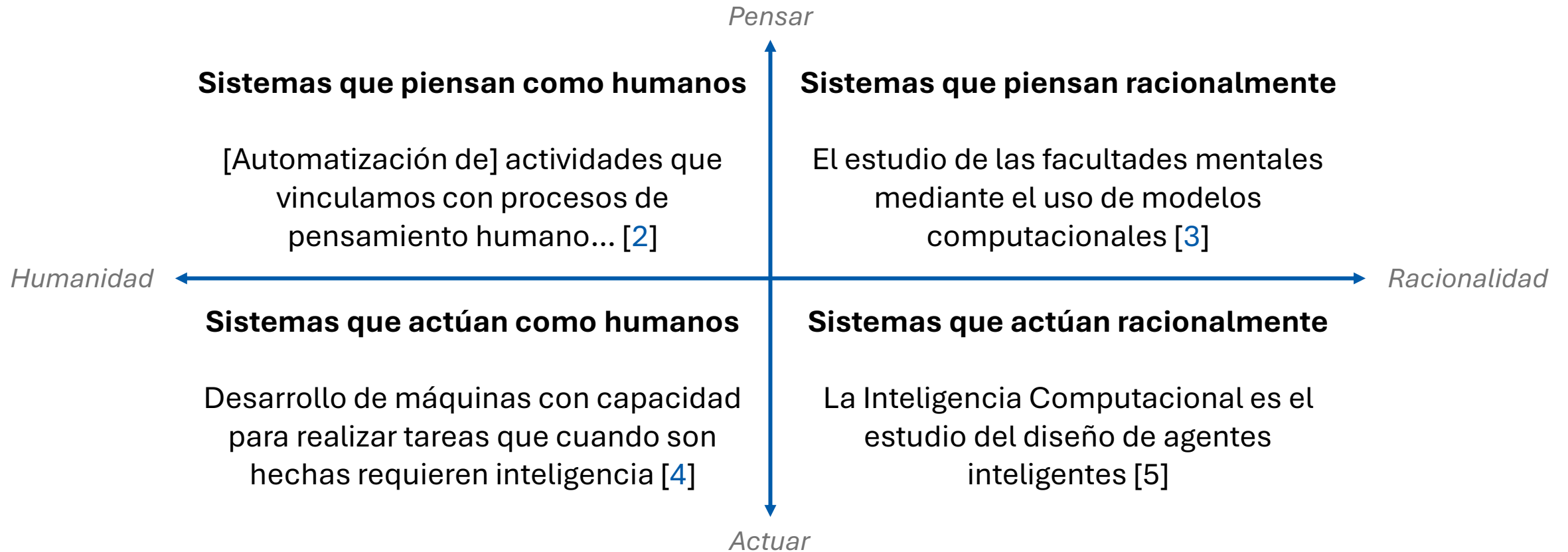
- Rusell y Norvig [1] analizaron ocho libros de texto.
- Encontraron cuatro enfoques sobre cuál debería ser el objeto de estudio.
- Concuerdan en que la IA debe de estudiar sistemas inteligentes.
- ¿Qué características deben tener para ser inteligentes?

Cognición	Comportamiento
Pensar (crear)	Humano
Actuar (simular)	Racional

[1] Russell, S. J., y Norvig, P. (2021). *Inteligencia artificial: un enfoque moderno* (4.^a ed.). Pearson.



Definición de la Inteligencia Artificial



[2] Bellman, R. E. (1978). *An introduction to artificial intelligence: Can computers think?* Boyd & Fraser.

[3] Charniak, E., & McDermott, D. (1985). *Introduction to artificial intelligence*. Addison-Wesley.

[4] Kurzweil, R. (1990). *The age of intelligent machines*. MIT Press.

[5] Poole, D. L., et al. (1998). *Computational intelligence: A logical approach*. Oxford University Press.



Simulación del comportamiento humano

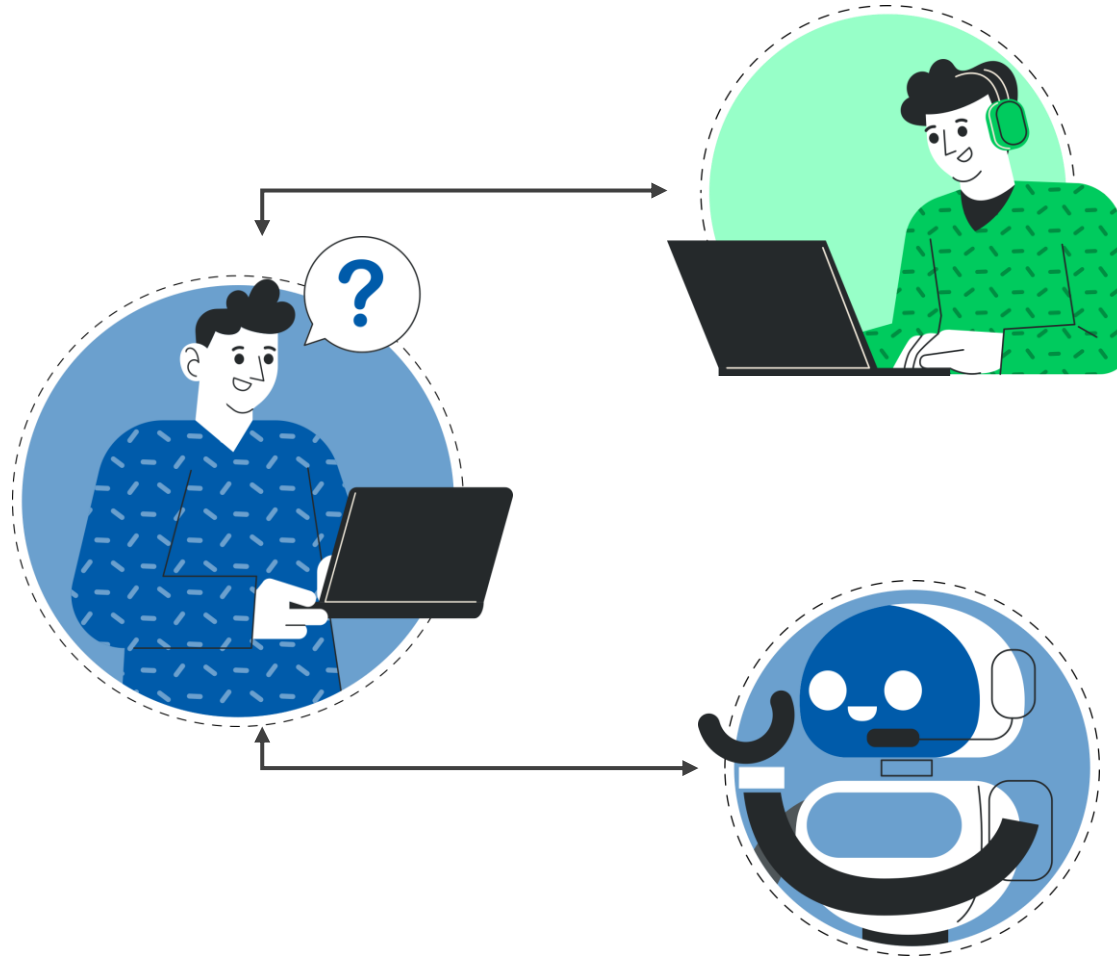
- La Prueba de Turing [6] fue propuesta por Alan Turing (1950).
- Se diseñó para proporcionar una definición de inteligencia.
- No proporciona una lista de cualidades para obtener inteligencia.
- Se basa en la incapacidad de diferenciar entre entidades inteligentes.
- La prueba se supera si no se distingue entre una persona y un sistema.

[6] Turing, A. M. (1950). *Computing machinery and intelligence*. Mind, 59(236), 433–460.



La prueba de Turing

Habitación 1:
Evaluador humano
Cuenta con dos terminales de texto para comunicarse.



Habitación 2:
Ser humano
Recibe preguntas y responde por texto.

Habitación 3:
Sistema de IA
Recibe las mismas preguntas y responde por texto.



Capacidades de la prueba de Turing

El sistema debería poseer las siguientes capacidades:

- **Procesamiento de lenguaje natural** que le permita comunicarse en inglés.
- **Representación del conocimiento** para almacenar lo que se conoce.
- **Razonamiento automático** para responder preguntas usando información.
- **Aprendizaje máquina** para adaptarse a circunstancias y problemas.



La prueba global de Turing

- La prueba original evitó la interacción física entre evaluador y sistema.
- La prueba global [7] también evalúa la percepción y la manipulación física.
- Para superar la prueba global de Turing la computadora debe incluir:
 - **Visión computacional** para percibir objetos.
 - **Robótica** para manipular y mover objetos.
- Estas seis capacidades constituyen los campos de la IA actual.

[7] Harnad, S. (1989). *Minds, machines and Searle*. Journal of Experimental & Theoretical Artificial Intelligence, 1(1), 5–25.



Aprendizaje máquina

- El aprendizaje se refiere a un espectro de situaciones en las cuales un agente incrementa su conocimiento para cumplir una tarea.
- Según Tom Mitchell [8], un programa de computadora aprende de la **experiencia E**, con respecto a alguna clase de **tareas T** y una **medida de desempeño P**, si su desempeño en las tareas de T, medido por P, mejora con la experiencia E.

[8] Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill.



Tipos de aprendizaje

- Según el tipo de experiencia, se distinguen tres tipos de aprendizaje:
 - El **aprendizaje supervisado** consiste en aprender una función a partir de ejemplos de sus entradas y sus salidas.
 - El **aprendizaje no supervisado** consiste en aprender a partir de patrones de entradas que no tienen valores de salidas.
 - El **aprendizaje por refuerzo** consiste en aprender una forma de actuar usando un refuerzo (recompensa o penalización).



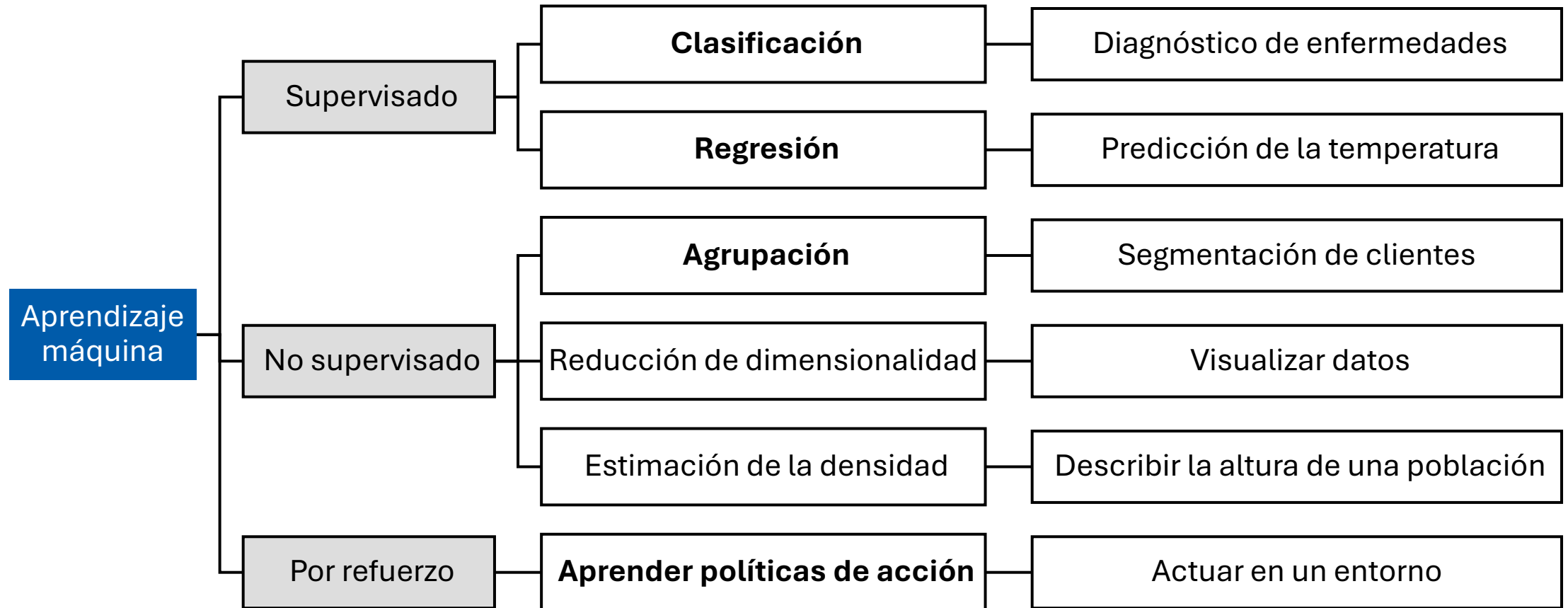
Ejemplos de aprendizaje

- Un sistema puede usar varios tipos de aprendizaje para distintas tareas.

SISTEMA TAXISTA		
Supervisado	No supervisado	Por refuerzo
Cuando el instructor de conducción grite “frena”, el agente puede aprender una regla condición-acción sobre cuándo frenar.	Desarrollar gradualmente los conceptos “día de tráfico bueno” y “día de tráfico malo”, sin que se le haya dado una solución.	La falta de propina al final del viaje da al agente algunas indicaciones de que su comportamiento no es el deseable.



Tareas de aprendizaje



Introducción al manejo de datos

- La calidad de los datos determina la calidad del modelo.
- Sin una preparación adecuada de los datos, no hay aprendizaje efectivo
- Principio fundamental: "Garbage in, garbage out".
- Transformamos datos crudos en patrones para la toma de decisiones.
- Datos limpios detectan amenazas reales en ciberseguridad.
- Se clasifican los datos por su estructura, etiquetas y consistencia.



Datos estructurados

- Datos organizados en formato tabular con esquema definido.

Características:

- Formato predecible (filas y columnas).
- Tipos de datos definidos (enteros, flotantes, categóricos).
- Fáciles de almacenar y consultar en bases de datos relacionales.

Ejemplos:

- Tablas SQL.
- Hojas de cálculo Excel/CSV.
- Logs de sistemas con formato fijo.



Datos no estructurados

- Datos sin formato predefinido ni organización tabular.

Características:

- Sin esquema fijo.
- Requieren procesamiento previo para extraer información.
- Representan ~80% de los datos empresariales.

Ejemplos:

- Texto (correos, código fuente, documentos, páginas web).
- Imágenes y videos.
- Audio y grabaciones.



Datos etiquetados

- Conjuntos de datos donde cada ejemplo tiene una etiqueta/clase asociada.

Características:

- Requieren proceso manual o semi-automático de etiquetado.
- Esenciales para entrenar modelos supervisados.
- Costosos de obtener y validar.

Ejemplos:

- Correos etiquetados como "spam" o "no spam".
- Imágenes de rostros con nombre de persona.
- Transacciones marcadas como "fraudulentas" o "legítimas".



Datos no etiquetados

- Datos sin categorías o respuestas conocidas.

Características:

- Abundantes y fáciles de recolectar.
- Requieren técnicas de descubrimiento de patrones.
- Útiles para detección de anomalías.

Ejemplos:

- Tráfico de red.
- Comportamientos de usuario.
- Código malicioso.



Repositorio de datos

- Los repositorios son bancos de datos públicos y organizados.
- Proporcionan conjuntos de datos para entrenar y validar modelos.
- Ofrecen datos etiquetados y documentados para distintos problemas.
- Son esenciales para la investigación y el desarrollo en aprendizaje máquina.
- Ejemplos: Kaggle, UCI, OpenML, gubernamentales y académicos.
- Su uso acelera el desarrollo y facilita la reproducibilidad.



Formatos de datos de texto

- Los formatos de texto permiten almacenar y procesar datos eficientemente.
- Son los más comunes para cargar en la mayoría de las herramientas.
 - CSV: formato tabular simple y universalmente compatible.
 - TSV: similar al CSV, pero con tabulaciones como delimitador.
 - JSON: ligero y flexible para datos estructurados y anidados.
 - XML: formato estructurado y autodescriptivo, aunque verboso.
 - ARFF: incluye metadatos, usado comúnmente en Weka.
 - Texto plano: sin formato, para datos sin estructura o documentos.



Limpieza y preprocesamiento de datos

- Los datos del mundo real son inexactos, inconsistentes e incompletos.
- El preprocesamiento de datos asegura la integridad de los datos.
- Permite generar análisis confiables y conclusiones válidas.
- Evita decisiones incorrectas o modelos ineficaces.



Flujo de trabajo de limpieza y preprocesamiento

- Recopilación: Obtener datos de bases de datos, APIs o scraping.
- Limpieza: Corregir errores, eliminar duplicados y manejar valores faltantes.
- Integración: Unir datos de múltiples fuentes, resolviendo inconsistencias.
- Transformación: Codificar, normalizar y crear nuevas variables.
- Reducción: Seleccionar características para simplificar el análisis.



Ejemplo de un flujo de trabajo

Recopilación	Limpieza	Integración	Transformación	Reducción
Fuentes comunes: Bases de datos, APIs, archivos CSV, scraping web.	Eliminar duplicados y observaciones irrelevantes. Manejar valores faltantes: Eliminar, imputar o marcar.	Alinear columnas y resolver conflictos entre fuentes.	Codificar variables categóricas ("Sí/No" a 1/0) Normalizar características numéricas (escalar entre 0 y 1)	Eliminar variables irrelevantes. Reducir dimensionalidad combinando variables.
Datos de ventas de múltiples tiendas.	Corregir errores tipográficos en nombres de productos.	Unir datos de clientes y ventas.	Crear características derivadas (edad a partir de fecha de nacimiento).	Usar solo 10 de 50 características para un modelo predictivo.



Python para el aprendizaje máquina

- **Python:** Lenguaje preferido por su facilidad y bibliotecas especializadas.
- Bibliotecas importantes:
 - Pandas: Manipulación de datos.
 - NumPy: Cálculos numéricos.
 - SciPy: Cómputo científico, estadística y señales.
 - Matplotlib/Seaborn: Visualización de datos.
 - Scikit-learn: Machine learning y preprocesamiento.



Pandas para la manipulación de datos

- Funcionalidades :
 - Filtrado, ordenamiento, agregación y fusión de datos.
 - Lectura y escritura de archivos (CSV, Excel, etc.).

```
# Uso de datos desde un archivo
import pandas as pd
df = pd.read_csv('datos.csv')
df.head()
```

```
# Uso de datos precargados
import pandas as pd
import seaborn as sns
df = sns.load_dataset('iris')
df.head()
```

- Uso típico: Limpieza, transformación y análisis exploratorio



NumPy para cálculos numéricos

- Funcionalidades:
 - Arreglos multidimensionales y operaciones matemáticas eficientes.
 - Base para muchas operaciones en Pandas y Scikit-learn.

```
import numpy as np                                # Carga de la biblioteca
arr = np.array([1, 2, 3, 4, 5])                   # Definición de arreglos
arr2 = arr * 2                                     # Multiplicación vectorial
```

- Uso típico: Cálculos numéricos y transformaciones de datos.



SciPy para cómputo científico y estadístico

- Contiene una amplia colección de algoritmos y funciones:
 - stats: pruebas estadísticas, distribuciones, medidas descriptivas.
 - signal: procesamiento de señales.
 - optimize: optimización y ajuste de curvas.
 - interpolate: interpolación de datos.
 - linalg: álgebra lineal avanzada.
- Se integra con NumPy (estructuras de datos) y Pandas (análisis de datos).



Matplotlib para la visualización de datos

- Funcionalidades:
 - Creación de gráficos (líneas, barras, dispersión, etc.).
 - Diagnóstico de problemas de calidad de datos.

```
import matplotlib.pyplot as plt          # Carga de la biblioteca
plt.plot([1, 2, 3, 4, 5])                # Creación de una gráfica
plt.title('Gráfico de línea simple')     # Título de la gráfica
plt.show()                               # Visualización de la gráfica
```

- Uso típico: Análisis exploratorio y visualización de datos.



Seaborn para la visualización estadística

- Funcionalidades:
 - Gráficos atractivos e informativos con pocas líneas de código.
 - Integración con Pandas para análisis rápido.

```
import seaborn as sns                                # Carga de la biblioteca
df = sns.load_dataset('tips')                        # Uso de datos precargados
sns.histplot(df['total_bill'])                       # Creación de un histograma
plt.title('Histograma de Total Bill')               # Título de la gráfica
plt.show()                                           # Visualización de la gráfica
```

- Uso típico: Visualización avanzada y análisis estadístico.



Caso de estudio

- Un conjunto de datos ampliamente conocido y utilizado para practicar la limpieza y preprocesamiento de datos es el [Titanic Dataset](#).
- Este conjunto contiene información sobre los pasajeros del Titanic, como edad, género, clase del boleto, tarifa, puerto de embarque y si sobrevivieron.

```
import pandas as pd
import seaborn as sns

df = sns.load_dataset('titanic')      # Cargar Titanic desde Seaborn
df.head()                            # Mostrar las primeras 5 filas
```



Análisis exploratorio de datos

Exploratory Data Analysis (EDA)

- El análisis exploratorio es el primer acercamiento sistemático a los datos.
- Su objetivo es comprender la estructura y detectar problemas.
- Permite identificar patrones, tendencias y relaciones entre variables.
- Orienta las decisiones sobre limpieza, transformación y modelado.
- Un EDA sólido evita conclusiones erróneas y modelos mal especificados.
- Se apoya en estadísticas descriptivas y visualizaciones.



Dimensiones y tipos de datos

- Es importante conocer la forma del conjunto: número de filas y columnas.
- Identificar el tipo de cada variable (numérica, categórica, texto, fecha).
- Verificar que los tipos sean adecuados para el análisis.

```
# Dimensiones del conjunto de datos
df.shape
print('Filas:', df.shape[0], 'Columnas:', df.shape[1])

# Tipos de datos en cada variable
print(df.dtypes)

# Resumen de información
df.info()
```



Tipos e información del conjunto

```
print(df.dtypes)
```

```
survived      int64
pclass        int64
sex           object
age           float64
sibsp         int64
parch         int64
...
adult_male    bool
deck          category
embark_town    object
alive         object
alone         bool
dtype: object
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column      Non-Null Count  Dtype
---  ---
 0   survived    891 non-null    int64
 1   pclass      891 non-null    int64
...
13  alive       891 non-null    object
14  alone       891 non-null    bool
dtypes: bool(2), category(2),
float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```



Estadísticos descriptivos básicos

- Resumen de variables categóricas: frecuencias y proporciones.
- Para variables continuas: media, mediana, mínimo, máximo, cuartiles.
- Permite detectar rangos anómalos y escala de las variables.

```
# Estadísticos para variables numéricas  
df.describe()
```

```
# Estadísticos para variables categóricas  
df['sex'].value_counts()  
df['embarked'].value_counts(dropna=False)
```



Distribuciones y frecuencias

- Analizar cómo se distribuyen los valores de una variable.
- Histogramas (*histograms*) para variables continuas.
- Diagramas de barras (*bar plots*) para variables categóricas.

```
# Histograma de edad
```

```
sns.histplot(df['age'].dropna(), bins=30, kde=True)  
plt.title('Distribución de edad')  
plt.show()
```

```
# Frecuencia de clase de pasajero
```

```
sns.countplot(x='class', data=df)  
plt.title('Pasajeros por clase')  
plt.show()
```



Correlaciones entre variables

- Mide la relación lineal entre dos variables numéricas.
- Tiene valores entre -1 y 1. La cercanía a ± 1 indica fuerte asociación.
- Útil para detectar redundancias y relaciones relevantes.
- No implica causalidad.

```
# Matriz de correlación
corr = df[['age', 'fare', 'pclass', 'sibsp', 'parch']].corr()
print(corr)

# Mapa de calor
sns.heatmap(corr, annot=True, cmap='coolwarm')
plt.title('Correlaciones numéricas')
plt.show()
```



Visualización inicial de datos

- Gráficos simples que resumen múltiples aspectos de los datos.
- Diagramas de dispersión (*scatter plots*) para pares de variables.
- Matriz de dispersión (*pair plots*) para una vista general multivariada.
- Diagramas de caja (*box plots*) para comparar distribuciones entre categorías.

```
# Dispersión: edad vs tarifa coloreada por supervivencia
```

```
sns.scatterplot(x='age', y='fare', hue='survived', data=df)  
plt.title('Relación edad-tarifa según supervivencia')  
plt.show()
```

```
# Pairplot (solo algunas columnas)
```

```
sns.pairplot(df[['age', 'fare', 'survived']].dropna(), hue='survived')  
plt.show()
```



Identificación de valores faltantes

- Identificar valores faltantes puede ser complejo en datos reales.
- Los datos faltantes pueden tomar varias formas.
- Celdas vacías, marcadores como “N/A” o “-999” o datos mal ingresados.

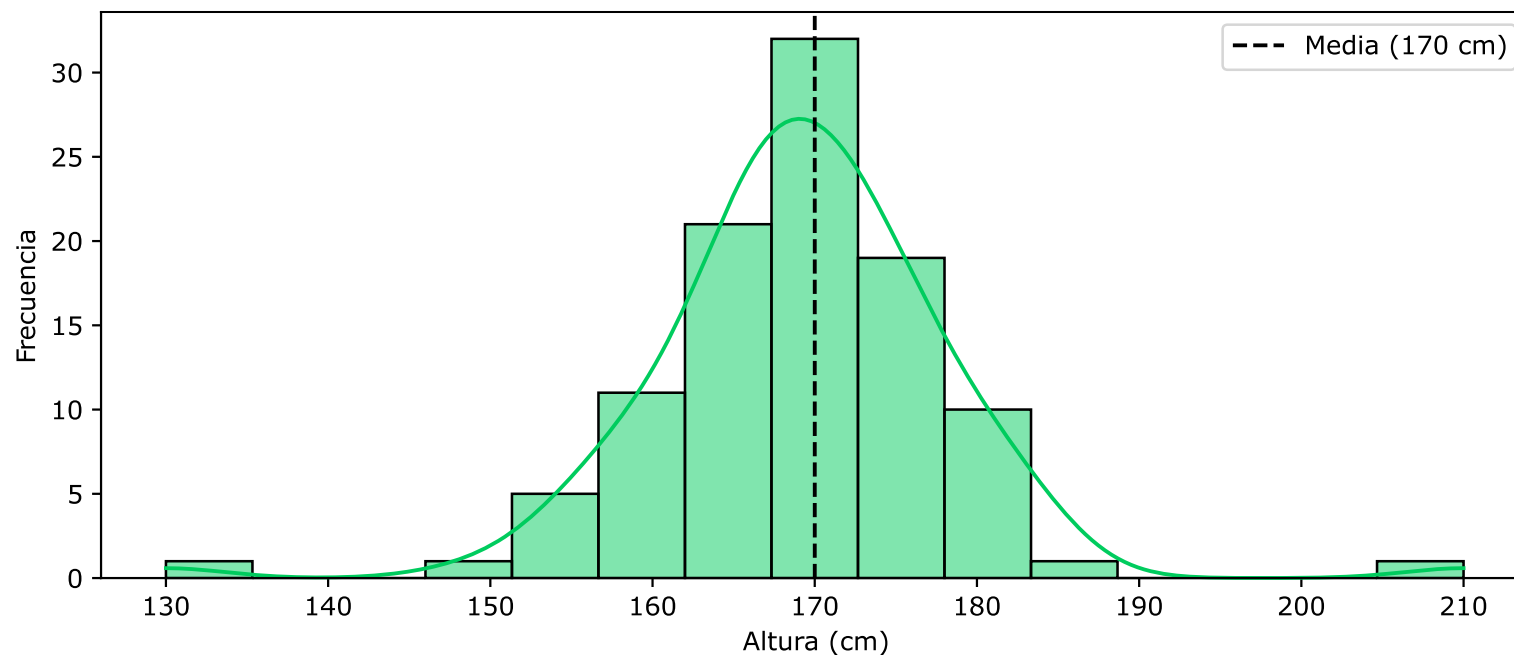
```
df.isnull()                                # Presencia de valores faltantes
faltantes = df.isnull().sum()              # Conteo de valores faltantes
faltantes

df['age'].isnull()                          # Valores faltantes por columna
df[df['age'].isnull()]                     # Registros con valores faltantes
df[df['deck'].isnull()]
```



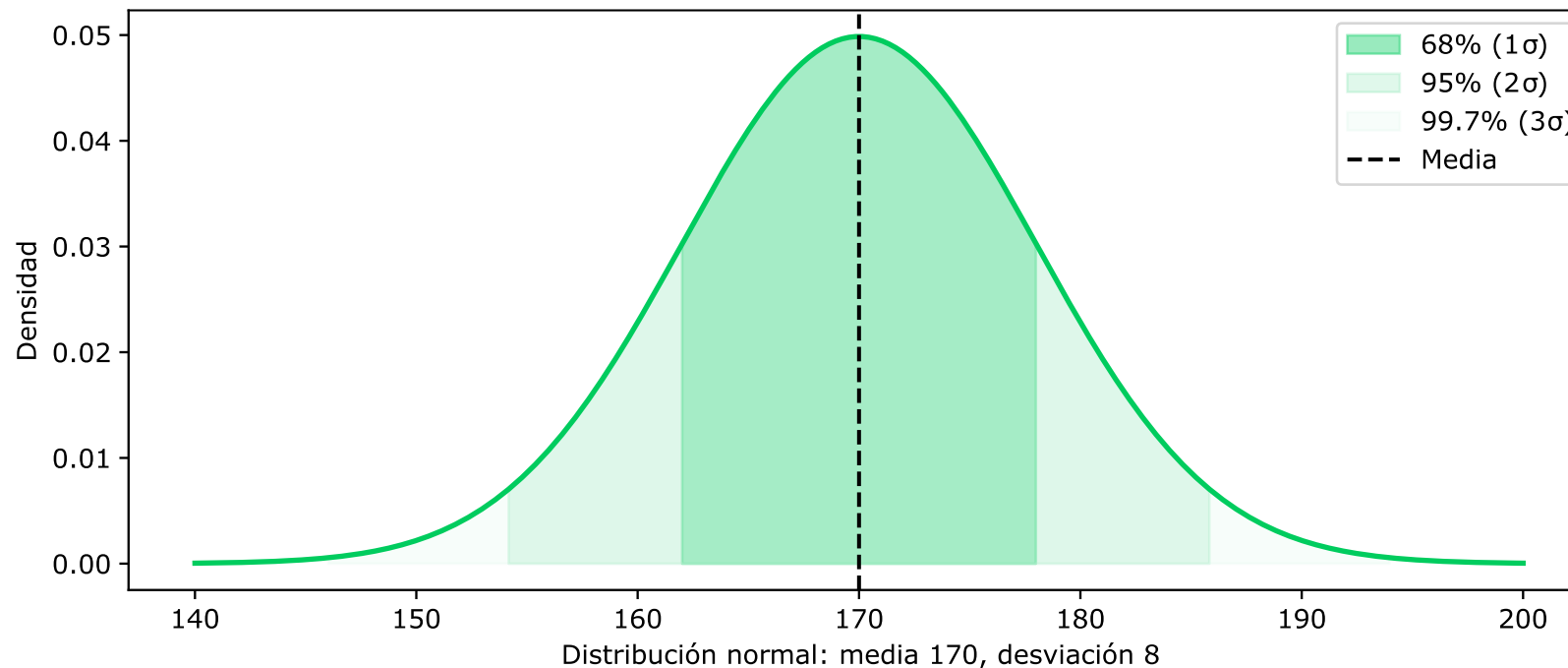
Valores atípicos

- Se realizó la medición de la altura en centímetros de 100 personas adultas.
- La mayoría de las personas tienen alturas cercanas a 170 cm.
- Algunas son más bajas o más altas, existiendo valores como 130 o 210 cm.
- Estos valores difieren significativamente del resto de los datos. ¿Son errores?



Distribución normal

- Muchas variables en la naturaleza siguen una distribución normal
- Se caracteriza por dos parámetros:
 - Media (μ): centro de la distribución.
 - Desviación estándar (σ): dispersión de los datos.



Puntaje Z (Z-score)

- El puntaje Z indica a cuántas desviaciones se encuentra un valor de la media.

$$Z = \frac{(x - \mu)}{\sigma}$$

- Si es positivo, el valor está por encima de la media, si es negativo por debajo.
- Valores con $|z| > 3$ son poco probables en una distribución normal (0.3%).

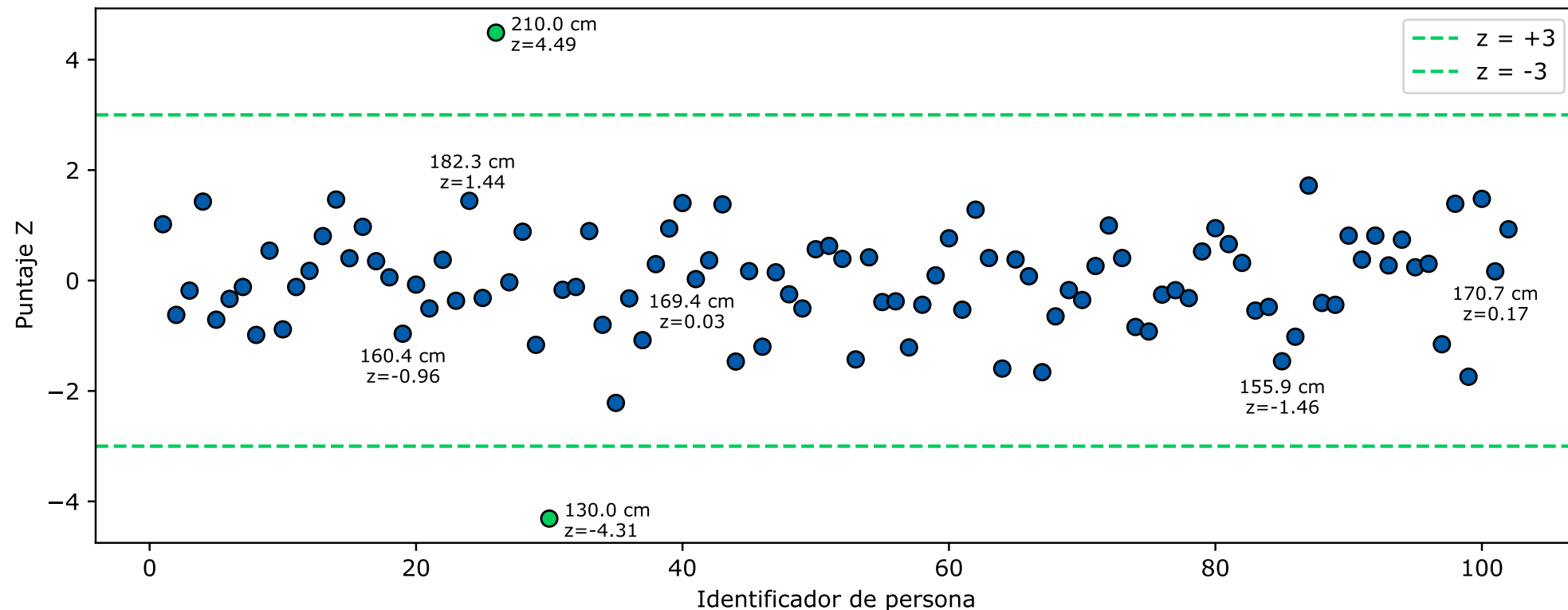
```
# Carga de librerías
from scipy import stats
import numpy as np

# Cálculo de puntajes Z, cuidado con los valores faltantes
z_scores = np.abs(stats.zscore(df['fare']))
df2[z_scores > 3]
```



Visualizando los puntajes Z

- Los puntos por encima de +3 o por debajo de -3 se consideran atípicos.
- En el ejemplo, dos puntos claramente fuera de ese rango.
- Este método asume que los datos siguen una distribución normal.



Cuartiles

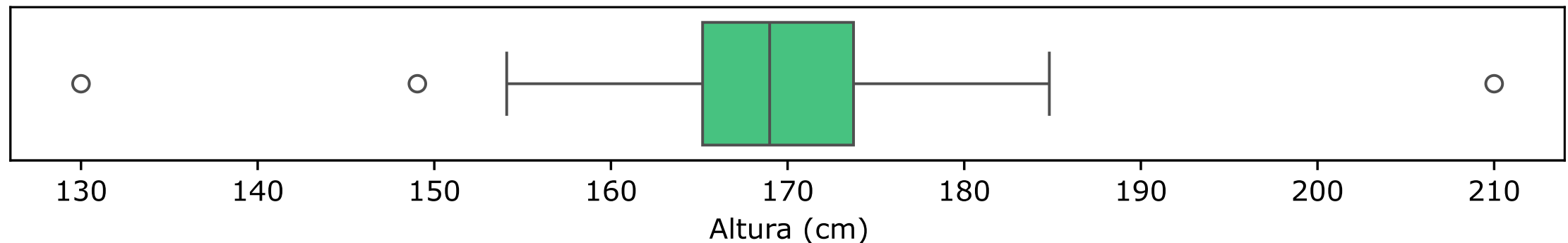
- Los cuartiles dividen un conjunto de datos ordenados en cuatro partes.
- Cada parte contiene el 25% de las observaciones.
- Tres cuartiles principales:
 - Q1 (primer cuartil): valor por debajo del cual queda el 25% de los datos.
 - Q2 (segundo cuartil): la mediana, 50% de los datos por debajo.
 - Q3 (tercer cuartil): valor por debajo del cual queda el 75% de los datos.

Observación	1	...	24	25	...	49	50	...	74	75	...	100
Valor	130.0	...	164.8	165.0	...	168.5	168.8	...	172.8	173.0	...	210.0
Cuartil			Q1 = 164.9			Q2 = 168.8			Q3 = 172.9			



Rango intercuartil (IQR)

- El IQR mide la dispersión de la mitad central de los datos.
- Se calcula como la diferencia entre el tercer y el primer cuartil ($Q3 - Q1$).
- El diagrama de caja y bigotes es una representación gráfica del IQR:
 - La caja abarca desde $Q1$ hasta $Q3$.
 - La línea dentro de la caja es la mediana ($Q2$).
 - Los bigotes se extienden hasta el último valor dentro del IQR.
 - Los puntos más allá de los bigotes son valores atípicos.



Visualización con un diagrama de caja y bigotes

```
# Carga de librerías
import matplotlib.pyplot as plt

# Gráfico de caja
plt.boxplot(df['fare'])
plt.show()

# Gráfico de dispersión
plt.scatter(range(len(df['fare'])), df['fare'])
plt.show()
```



Identificación de valores atípicos usando el IQR

- Se considera que un valor es atípico si está más allá de 1.5 veces el IQR.
- El límite inferior es $(Q1 - 1.5 * IQR)$ y el superior es $(Q3 + 1.5 * IQR)$.
- Es una convención empírica que funciona bien en la práctica.

```
# Carga de librerías
```

```
import numpy as np
```

```
# Cálculo de cuartiles y el rango
```

```
Q1 = np.percentile(df['fare'], 25)
```

```
Q3 = np.percentile(df['fare'], 75)
```

```
IQR = Q3 - Q1
```

```
# Detección
```

```
df[(df['fare'] < (Q1 - 1.5 * IQR)) | (df['fare'] > (Q3 + 1.5 * IQR))]
```



Tarea 1: Ejercicio de análisis exploratorio de datos

- Selecciona datos de [seaborn](#) con variables numéricas y categóricas.
- Realiza las siguientes actividades de análisis exploratorio:
 - Carga el conjunto de datos y muestra las primeras 5 filas.
 - Identifica las dimensiones y los tipos de datos de cada variable.
 - Calcula estadísticos descriptivos básicos.
 - Visualiza la distribución de al menos dos variables numéricas.
 - Visualiza la frecuencia de al menos dos variables categóricas.
 - Identifica valores faltantes (si los hay) y calcula su porcentaje.
 - Detecta valores atípicos en una variable numérica usando [boxplot](#) o IQR.
 - Calcula la matriz de correlación y muéstrala con un [heatmap](#).
 - Genera un [pairplot](#) con cuatro variables y colorea por una categórica.
- Entrega una libreta con el código, las visualizaciones y tus observaciones.



Limpieza de datos

Data Cleaning (DC)

- Los datos rara vez vienen limpios y listos para usar.
 - **Valores faltantes:** Datos incompletos que pueden sesgar análisis.
 - **Outliers:** Valores atípicos que distorsionan resultados.
 - **Formato inconsistente:** Datos mal estructurados o inconsistentes.
 - **Duplicados:** Registros repetidos que hacen crecer el conjunto de datos.
 - **Datos redundantes:** Información que no aporta valor.
- Estos problemas pueden sesgar el análisis y generar modelos erróneos.



Preservación de datos originales

- Las técnicas para manejar datos faltantes modifican el conjunto de datos.
- Si el resultado no es satisfactorio, no se puede volver al punto de partida.
- La solución es crear una copia de trabajo antes de cualquier manipulación.

```
# Copia del conjunto original  
df2 = df.copy(deep = True)
```



Eliminación de valores faltantes

- **Eliminación de registros:** Se usa solo si los valores faltantes constituyen una pequeña fracción del conjunto de datos.

```
# Eliminación de datos faltantes  
df2.dropna(inplace=True)  
# Comprobación de datos faltantes  
df2.isnull().sum( )
```

- **Desventaja:** Puede llevar a la pérdida de información valiosa.



Sustitución de valores faltantes con estadísticos

- **Sustituir datos faltantes con valores estadísticos:**
 - Media – Para datos continuos.
 - Mediana – Para datos ordinales.
 - Moda – Para datos categóricos.

Imputación de datos

```
df2.fillna({'age': df2['age'].mean()}, inplace = True)
df2.fillna({'age': df2['age'].median()}, inplace = True)
df2.fillna({'age': df2['age'].mode()}, inplace = True)
```

- **Desventaja:** Puede reducir la varianza y afectar la correlación.



Sustitución de valores faltantes con constantes

- **Sustituir datos faltantes por un valor constante:** Útil cuando se puede hacer una suposición razonable sobre el valor.

```
# Imputación de datos  
df2.fillna({'age': 0}, inplace = True)
```

- **Desventaja:** Puede introducir sesgo si el valor elegido no es representativo.



Eliminación de valores atípicos

- **Eliminación:** Eliminar valores atípicos solo si están claramente mal registrados o medidos.

```
# Carga de librerías
from scipy import stats
import numpy as np

# Cálculo de puntajes Z, cuidado con los valores faltantes
z_scores = np.abs(stats.zscore(df2['fare']))
df2[z_scores <= 3]
```

- **Desventaja:** Puede causar pérdida de información si el valor no es un error.



Transformación de atípicos

- **Transformación:** Las transformaciones pueden reducir el efecto de los valores extremos. Por ejemplo: Transformación logarítmica.

```
# Carga de librerías
import matplotlib.pyplot as plt
import numpy as np

# Transformación de datos
df2['fare'] = np.log(df2['fare'])

# Gráfico de dispersión
plt.scatter(range(len(df2['fare'])), df2['fare'])
plt.show()
```

- **Desventaja:** Puede alterar la interpretación de los datos.



Formato inconsistente

- Los datos categóricos suelen presentar variaciones en su escritura.
- Mayúsculas/minúsculas, espacios, abreviaciones, tipeo erróneo.
- Deben unificarse para que una misma categoría no sea tratada como varias.

```
# Estandarizar texto (minúsculas, sin espacios)
df2['sex'] = df2['sex'].str.lower().str.strip()

# Unificar categorías
df2['embark_town'] = df2['embark_town'].replace({'S': 'Southampton',
                                                'C': 'Cherbourg',
                                                'Q': 'Queenstown'})
```



Datos duplicados

- Registros idénticos (o casi idénticos) que aparecen más de una vez.

```
# Identificar filas duplicadas
duplicados = df2.duplicated()
print("Filas duplicadas:", duplicados.sum())

# Mostrar duplicados (si existen)
df2[duplicados]

# Eliminar duplicados (conservar el primero)
df2.drop_duplicates(inplace=True)

# Eliminar duplicados considerando solo algunas columnas
df2.drop_duplicates(subset=['pclass', 'sex', 'age'], keep='last',
                    inplace=True)
```



Datos redundantes

- Variables que expresan la misma información de forma diferente.

```
# Identificar redundancias mediante correlación o inspección
df2[['survived', 'alive']].head()

# Eliminar variable redundante
df2.drop('alive', axis=1, inplace=True)

# Otras redundancias comunes:
# - 'class' (First, Second, Third) y 'pclass' (1,2,3)
# - 'adult_male' derivado de 'age' y 'sex'
df2.drop('class', axis=1, inplace=True)
```



Tarea 2: Ejercicio de limpieza de datos

- Utiliza el mismo conjunto de datos de la tarea anterior.
- Realiza las siguientes actividades de limpieza:
 - Crea una copia de trabajo del conjunto para no modificar el original.
 - Identifica valores faltantes y calcula su porcentaje por variable.
 - Maneja los valores faltantes y justifica tu decisión.
 - Detecta valores atípicos en tres variables numéricas con IQR o z-score.
 - Decide cómo tratar los valores atípicos y justifica.
 - Identifica y elimina registros duplicados si existen.
 - Identifica variables redundantes (correlación > 0.9 o valores repetidos).
 - Elimina las variables redundantes y justifica por qué no aportan valor.
 - Compara las dimensiones del conjunto antes y después de la limpieza.
- Entrega una libreta con el código, las transformaciones y tus justificaciones.



Transformación de datos

- Los datos transformados permiten que los algoritmos aprendan.
- Se aplican cambios sobre los datos limpios para:
 - Adaptar variables al formato requerido por el modelo.
 - Crear nuevas representaciones que capturen mejor la información.
 - Mejorar el rendimiento.
- Principales tipos de transformación:
 - Codificación de variables categóricas.
 - Escalado de variables numéricas.
 - Remuestreo para equilibrar clases.



Técnicas de codificación

- Las variables categóricas representan grupos o categorías.
- Algunos modelos de aprendizaje máquina requieren entradas numéricas.
- Transformar categorías en números para análisis y modelado.
- Técnicas comunes:
 - **One-Hot Encoding:** Para categorías sin orden (ej: puerto de embarque).
 - **Label Encoding:** Para categorías con orden (ej: nivel educativo).



One-hot encoding

- Crea una columna binaria (0/1) para cada categoría.

```
df = pd.get_dummies(df, columns=['embarked'], prefix='embarked')
```

- Columnas: embarked_S, embarked_C, embarked_Q.
- **Ventaja:** Elimina supuestos de orden entre categorías.



Label encoding

- Asigna un número único a cada categoría.

```
# Crear un diccionario de mapeo para 'sex'  
gender_map = {'male': 0, 'female': 1}  
df['sex_encoded'] = df['sex'].map(gender_map)
```

- **Ventaja:** Mantiene la dimensionalidad del conjunto de datos.



Desbalance de clases

- En problemas de clasificación, los datos desbalanceados ocurren cuando el número de observaciones en una clase es significativamente menor.
- Ejemplo:
 - Clase A: 90 muestras.
 - Clase B: 10 muestras.
- La mayoría de los algoritmos de clasificación están diseñados para maximizar la precisión y minimizar el error, provocando que el modelo se enfoque en la clase mayoritaria.
- La precisión puede parecer alta, pero la capacidad de predecir correctamente la clase minoritaria es baja.



Técnicas para manejar datos desbalanceados

- **Re-muestreo** (*Resampling*): Crear nuevas muestras a partir de existentes:
 - **Submuestreo** (*undersampling*): Reducir la clase mayoritaria.
 - **Sobremuestreo** (*oversampling*): Aumentar la clase minoritaria.



Submuestreo

- Elimina ejemplos de la clase mayoritaria para equilibrar las clases.

```
# Verificación de la distribución de clases
```

```
df['class'].value_counts()
```

```
# División del conjunto de datos
```

```
mayoritaria = df[df['class'] == 0]
```

```
minoritaria = df[df['class'] == 1]
```

```
# Selección aleatoria de la clase mayoritaria
```

```
submuestreo = mayoritaria.sample(len(minoritaria))
```

```
# Combinación de datos submuestreados
```

```
df_submuestreado = pd.concat([submuestreo, minoritaria])
```

```
# Comprobación
```

```
print(df_submuestreado ['class'].value_counts())
```



Sobremuestreo

- Duplica o genera nuevos ejemplos para la clase minoritaria.

```
# Verificación de la distribución de clases
df['class'].value_counts()

# Sobremuestreo manual
sobremuestreo = minoritaria.sample(len(mayoritaria), replace=True)

# Combinación de datos sobremuestreados
df_sobremuestreado = pd.concat([mayoritaria, sobremuestreo])

# Comprobación
print(df_sobremuestreado ['class'].value_counts())
```



Escalado de variables numéricas

- Muchos algoritmos son sensibles a la escala de las variables.
- Variables con rangos diferentes pueden dominar el modelo.
- Técnicas comunes:
 - Normalización (Min-Max): Escala los valores a un rango $[0, 1]$.
 - Estandarización (Z-score): Centra en media 0 y desviación estándar 1.



Implementación del escalado

```
# Importación de funciones de escalado
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Normalización Min-Max
scaler_minmax = MinMaxScaler()
df['fare_minmax'] = scaler_minmax.fit_transform(df[['fare']])

# Estandarización Z-score
scaler_std = StandardScaler()
df['fare_std'] = scaler_std.fit_transform(df[['fare']])

# Comparar resultados
print(df[['fare', 'fare_minmax', 'fare_std']].head())
```



Uso del escalado de variables numéricas

Criterio	Normalización	Estandarización
Distribución de los datos	No se asume distribución normal.	Se prefiere en datos distribuidos aproximadamente normal.
Presencia de valores atípicos	No recomendada, los valores atípicos distorsionan el rango.	Más robusta, aunque valores extremos pueden afectar.
Rango de salida	Valores en un rango fijo, generalmente [0, 1] o [-1, 1].	Valores con media 0 y desviación 1; el rango puede exceder [-1, 1].
Interpretación	Proporción relativa dentro del rango observado.	Número de desviaciones estándar respecto a la media.
Fórmula	$X_{\text{norm}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$	$X_{\text{std}} = \frac{X - \mu}{\sigma}$
Cuándo usarla	Se conocen los límites de los datos.	Se busca comparabilidad entre variables de diferentes escalas.



Tarea 3: Ejercicio de transformación de datos

- Trabaja con el conjunto de datos limpio de la tarea anterior.
- Realiza las siguientes actividades de transformación:
 - Identifica las variables categóricas nominales y ordinales en tu conjunto.
 - Aplica **One-Hot Encoding** a una variable nominal.
 - Aplica **Label Encoding** a una variable ordinal.
 - Muestra el conjunto antes y después de la codificación.
 - Selecciona dos variables numéricas para escalar.
 - Aplica normalización a una de ellas y estandarización a la otra.
 - Si la variable objetivo está desbalanceada, aplica remuestreo.
 - Verifica la nueva distribución de clases después del remuestreo.
 - Explica por qué elegiste cada técnica de transformación.
- Entrega una libreta con el código, las transformaciones y tus comentarios.



Aprendizaje supervisado

- Los humanos aprendemos de experiencias pasadas.
- Un niño aprende a identificar animales viendo ejemplos.
- La experiencia se acumula y permite generalizar a nuevos casos.
- Una computadora no tiene experiencias. En su lugar, aprende de datos.
- Estos datos son ejemplos históricos con entradas y salidas conocidas.
- El objetivo es obtener una función que prediga salidas para nuevas entradas.



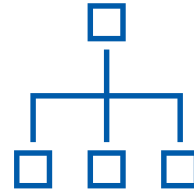
Flujo del aprendizaje supervisado

- Para generar un modelo se necesita un conjunto de datos, un algoritmo de aprendizaje y una forma de evaluarlo o interpretarlo.



Conjunto de datos

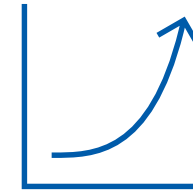
Variable 1	Variable 2	Objetivo
A	B	V_1
C	D	V_2
E	F	V_3



Modelo supervisado

Existen más de ciento cincuenta modelos clasificadores [9]:

1. Regresión lineal
2. Redes neuronales
3. Redes bayesianas
4. Árboles de decisión



Interpretación/Evaluación

Una vez obtenido el modelo, se puede:

1. Analizar el conocimiento o los patrones capturados.
2. Medir su desempeño en datos no observados antes.

[9] Fernández-Delgado, M., Cernadas, E., Barro, S., & Amorim, D. (2014). Do we Need Hundreds of Classifiers to Solve Real World Classification Problems?



Datos en el aprendizaje supervisado

- Un conjunto de datos está compuesto por:
 - Una lista de **registros** (observaciones o ejemplos).
 - Un conjunto de variables **predictoras** (características o atributos).
 - Las variables **cuantitativas** se conocen como continuas o numéricas.
 - Las variables **cualitativas** se conocen como discretas o categóricas.
 - Dentro de las categóricas existen las nominales y las ordinales.
 - Una variable **por predecir** (dependiente, objetivo o clase).
- Si la variable objetivo es categórica se vuelve un problema de **clasificación**.
- Si la variable objetivo es numérica se vuelve un problema de **regresión**.



Conjunto de datos: Diamonds

id	carat	cut	color	clarity	depth	table	price	x	y	z
0	0.23	Ideal	E	SI2	61.5	55.0	326	3.95	3.98	2.43
1	0.21	Premium	E	SI1	59.8	61.0	326	3.89	3.84	2.31
2	0.23	Good	E	VS1	56.9	65.0	327	4.05	4.07	2.31
3	0.29	Premium	I	VS2	62.4	58.0	334	4.20	4.23	2.63
4	0.31	Good	J	SI2	63.3	58.0	335	4.34	4.35	2.75
5	0.24	Very Good	J	VVS2	62.8	57.0	336	3.94	3.96	2.48
6	0.24	Very Good	I	VVS1	62.3	57.0	336	3.95	3.98	2.47
7	0.26	Very Good	H	SI1	61.9	55.0	337	4.07	4.11	2.53
8	0.22	Fair	E	VS2	65.1	61.0	337	3.87	3.78	2.49
9	0.23	Very Good	H	VS1	59.4	61.0	338	4.00	4.05	2.39



Características de Diamonds

- **Uso:** Regresión (price) o clasificación (cut).

Variable	Tipo	Descripción
price	numérica continua	Precio en dólares
carat	numérica continua	Peso del diamante
cut	categorica ordinal	Calidad del corte (Fair, Good, Very Good, Premium, Ideal)
color	categorica ordinal	Color (D=mejor a J=peor)
clarity	categorica ordinal	Claridad (I1, SI2, SI1, VS2, VS1, VVS2, VVS1, IF)
depth	numérica continua	Profundidad $2*z/(x+y)$
table	numérica continua	Ancho de la mesa
x	numérica continua	Largo en mm
y	numérica continua	Ancho en mm
z	numérica continua	Profundidad en mm



Modelos supervisados de aprendizaje

- Un modelo es una **función** o **teoría** que se aprende de los datos.
- Puede verse como una **hipótesis** sobre la relación entre entradas y salidas.
- Se **ajusta** (entrena) un modelo usando un algoritmo de aprendizaje.
- El algoritmo (método o técnica) encuentra los **parámetros** del modelo.
- Los algoritmos se configuran a través de sus **hiperparámetros**.
- Una vez entrenado, el modelo puede predecir salidas para nuevos datos.

- Si la variable objetivo es categórica, el modelo se llama **clasificador**.
- Si la variable objetivo es numérica, se llama modelo de regresión.



Interpretación y evaluación de modelos

- Los modelos no solo predicen, también **representan conocimiento**.
- Ese conocimiento revela **patrones** sobre cómo se relacionan las variables.
- Es importante para **tomar decisiones** en situaciones complejas.
- Algunos modelos permiten interpretar ese conocimiento.
- Los modelos que no se pueden interpretar se denominan de **caja negra**.
- Las redes neuronales son un ejemplo clásico de caja negra.
- **Evaluar** un modelo es tan importante como entrenarlo.
- La evaluación determina si el modelo es confiable o debe sustituirse.



Modelo de regresión lineal simple

- Describe la **relación** entre una variable independiente y una dependiente.
- Asume que dicha relación puede representarse mediante una **línea recta**.
- Permite **predecir** valores continuos a partir de un solo atributo.
- Es el punto de partida para entender las tareas de **regresión**.



Forma del modelo de regresión lineal simple

$$\hat{y} = \theta_0 + \theta_1 x$$

Donde:

- $\hat{y}$: valor predicho por el modelo (estimación de la variable dependiente)
 - x : variable independiente
 - θ_0 : intercepto que representa el valor de $\hat{y}$ cuando $x = 0$
 - θ_1 : pendiente que representa el cambio en $\hat{y}$ por cada unidad en x
-
- El objetivo del modelo es que $\hat{y}$ sea lo más cercano posible a y .
 - El modelo define una familia de todas las líneas posibles.
 - Los parámetros $\theta = [\theta_0, \theta_1]^T$ determinan la línea específica.



Ejemplo sobre el salario mensual

- Se cuenta con datos de siete personas con diferentes años de experiencia y sus salarios mensuales en miles de pesos.

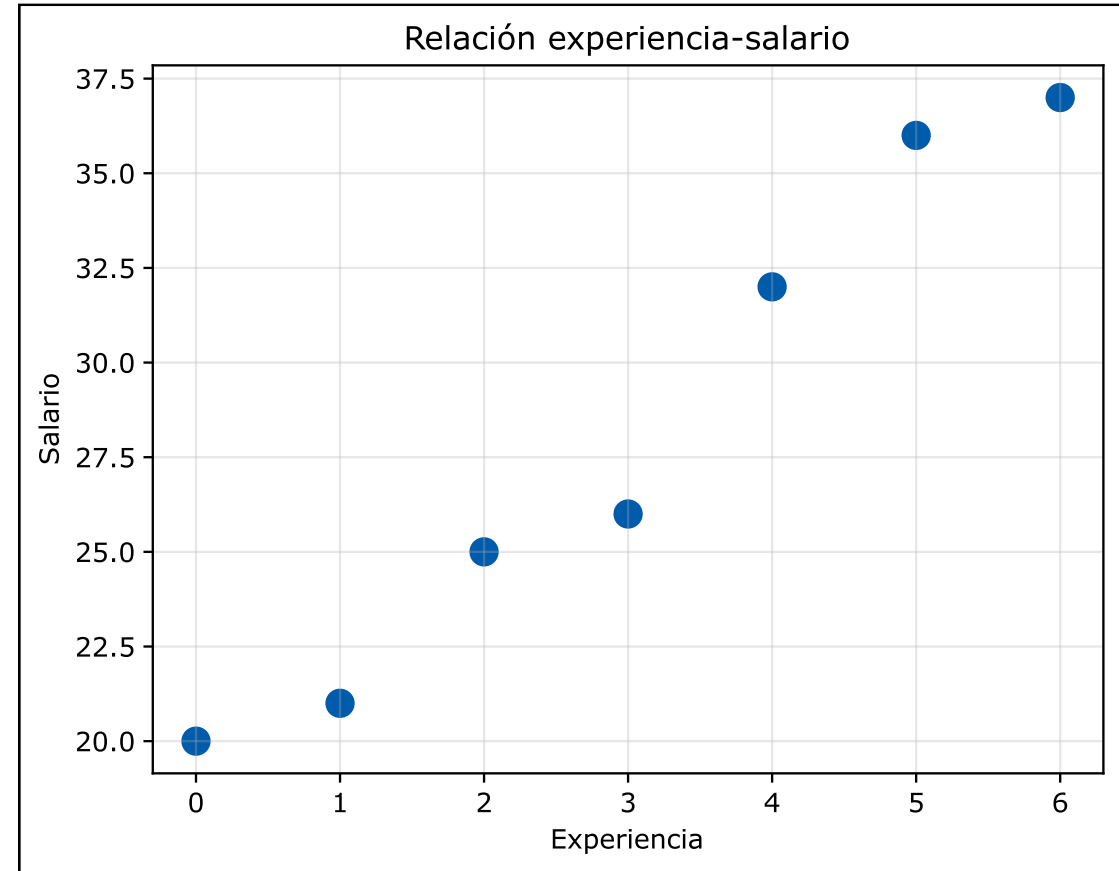
Experiencia	0	1	2	3	4	5	6
Salario	20	21	25	26	32	36	37



Visualización de la relación experiencia-salario

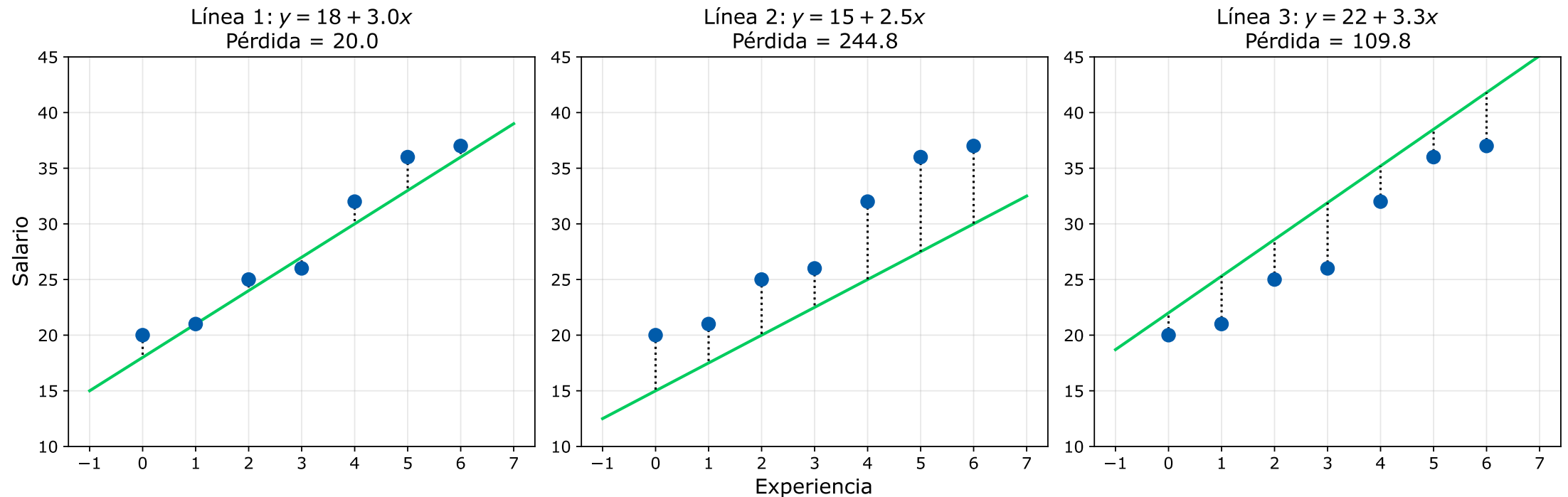
```
# Importar NumPy y Matplotlib
# Datos
x = np.array([0, 1, 2, 3, 4, 5, 6])
y = np.array([20, 21, 25, 26, 32, 36, 37])

# Gráfica de puntos
plt.scatter(x, y, color='#005BAA', s=100)
plt.xlabel('Experiencia')
plt.ylabel('Salario')
plt.title('Relación experiencia-salario')
plt.grid(True, alpha=0.3)
plt.show()
```



Ajuste de diferentes parámetros

- Cada elección de θ_0 y θ_1 define una línea diferente.
- Algunas líneas se ajustan mejor a los datos que otras.
- La calidad del ajuste se mide mediante la función de pérdida.



Visualización de múltiples parámetros

```
lineas = [(18, 3.0, 20), (15, 2.5, 244.8), (22, 3.3, 109.8)]
x_plot = np.linspace(-1, 7, 100)
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for i, (t0, t1, p) in enumerate(lineas):
    y_pred = t0 + t1 * x
    axes[i].scatter(x, y, color='#005BAA', s=80, zorder=3)
    axes[i].plot(x_plot, t0 + t1 * x_plot, '-', color="#00CC5E", linewidth=2)

    for xi, yi, ypi in zip(x, y, y_pred):
        axes[i].plot([xi, xi], [yi, ypi], color='black', linestyle=':')

    # Configurar título, etiquetas, ejes y rejilla
    axes[i].set_title(f'Línea {i+1}:  $y = \{t0\} + \{t1\}x$ \nPérdida =  $\{p:.1f\}$ ', fontsize=14)
    axes[i].tick_params(axis='both', which='major', labelsize=11)
    axes[i].grid(True, alpha=0.3)
    axes[i].set_ylim(10, 45)

axes[1].set_xlabel('Experiencia', fontsize=14)
axes[0].set_ylabel('Salario', fontsize=14)

plt.tight_layout()
plt.show()
```

Parámetros y pérdidas
Valores para las líneas
Figura con 3 columnas

Bucle sobre las líneas

Dibujar puntos
Dibujar líneas

Dibujar residuos



Función de pérdida (*Loss Function*)

- Mide qué tan mal se ajusta el modelo a los datos.
- Para cada punto, calculamos el error: diferencia entre valor real y predicho.
- Se usa el error cuadrático para que la dirección no importe.
- La pérdida total es la suma de errores cuadráticos (SSE).

$$L[\theta_0, \theta_1] = \sum_i (y_i - \hat{y}_i)^2$$

$$L[\theta_0, \theta_1] = \sum_i (y_i - \theta_0 - \theta_1 x_i)^2$$



Ejercicio: cálculo de la suma del error cuadrático

- A partir del conjunto de datos sobre salarios, predice las salidas de los siguientes modelos y luego calcula la suma de errores cuadrados:

Experiencia	0	1	2	3	4	5	6
Salario	20	21	25	26	32	36	37

Modelo	θ_0	θ_1
1	13	5
2	18	-3



Solución: error cuadrático del modelo I

x	y_{real}	y_{pred}	$(y_{\text{real}} - y_{\text{pred}})$	$(y_{\text{real}} - y_{\text{pred}})^2$
0	20	$13 + 5 * 0 = 13$	7	49
1	21	$13 + 5 * 1 = 18$	3	9
2	25	$13 + 5 * 2 = 23$	2	4
3	26	$13 + 5 * 3 = 28$	-2	4
4	32	$13 + 5 * 4 = 33$	-1	1
5	36	$13 + 5 * 5 = 38$	-2	4
6	37	$13 + 5 * 6 = 43$	-6	36
		Suma	1	107



Solución: error cuadrático del modelo 2

x	y_{real}	y_{pred}	$(y_{\text{real}} - y_{\text{pred}})$	$(y_{\text{real}} - y_{\text{pred}})^2$
0	20	$18 - 3 * 0 = 18$	2	4
1	21	$18 - 3 * 1 = 15$	6	36
2	25	$18 - 3 * 2 = 12$	13	169
3	26	$18 - 3 * 3 = 9$	17	289
4	32	$18 - 3 * 4 = 6$	26	676
5	36	$18 - 3 * 5 = 3$	33	1089
6	37	$18 - 3 * 6 = 0$	37	1369
		Suma	134	3632



Programación de la función de pérdida

```
def funcion_perdida(x, y, theta0, theta1):  
    """  
    Calcula la función de pérdida de mínimos cuadrados (SSE).  
    Parámetros:  
    x : Vector de valores de la variable independiente.  
    y : Vector de valores reales de la variable dependiente.  
    theta0 : Intercepto del modelo de regresión lineal.  
    theta1 : Pendiente del modelo de regresión lineal.  
  
    Retorna:  
    Suma de cuadrados de los errores (SSE).  
    """  
    predicciones = theta0 + theta1 * x  
    errores = y - predicciones  
    return np.sum(errores ** 2)
```



Ajuste de parámetros en forma cerrada

- El método de mínimos cuadrados busca los parámetros óptimos θ_0 y θ_1 .
- Se minimiza la suma de errores cuadráticos: $\sum_i (\theta_0 + \theta_1 x_i - y_i)^2$.
- Para la regresión lineal simple, existe una solución analítica cerrada.
- Se obtiene igualando las derivadas parciales de la función de pérdida a cero.
- La pendiente e intercepto óptimos se calculan de la siguiente manera:

$$\theta_1 = \frac{\text{Cov}(x, y)}{\text{Var}(x)} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}$$

$$\theta_0 = \bar{y} - \theta_1 \bar{x}$$

- Donde $\bar{x}$ y $\bar{y}$ son las medias muestrales de x y y , respectivamente.



Ejercicio: cálculo de parámetros en forma cerrada

- A partir del conjunto de datos sobre salarios, calcula las medias de cada variable, la varianza del atributo y la covarianza. Después, calcula θ_1 y θ_0 .

Experiencia	0	1	2	3	4	5	6
Salario	20	21	25	26	32	36	37



Solución: cálculo de las medias y la varianza

$$\bar{x} = 0 + 1 + 2 + 3 + 4 + 5 + 6 = 21/7 = 3$$

$$\bar{y} = 20 + 21 + 25 + 26 + 32 + 36 + 37 = 197/7 \approx 28.1429$$

$$\text{Var}(x) = (0 - 3)^2 + (1 - 3)^2 + (2 - 3)^2 + (3 - 3)^2 + (4 - 3)^2 + (5 - 3)^2 + (6 - 3)^2$$

$$\text{Var}(x) = 9 + 4 + 1 + 0 + 1 + 4 + 9 = 28$$



Solución: cálculo de la covarianza

$$\text{Cov}(x, y) = \sum (x_i - \bar{x})(y_i - \bar{y})$$

x_i	y_i	$x_i - \bar{x}$	$y_i - \bar{y}$	$(x_i - \bar{x})(y_i - \bar{y})$
0	20	-3	$20 - 197/7 = -57/7$	$(-3)(-57/7) = 171/7$
1	21	-2	$21 - 197/7 = -50/7$	$(-2)(-50/7) = 100/7$
2	25	-1	$25 - 197/7 = -22/7$	$(-1)(-22/7) = 22/7$
3	26	0	$26 - 197/7 = -15/7$	0
4	32	1	$32 - 197/7 = 27/7$	$1 \cdot 27/7 = 27/7$
5	36	2	$36 - 197/7 = 55/7$	$2 \cdot 55/7 = 110/7$
6	37	3	$37 - 197/7 = 62/7$	$3 \cdot 62/7 = 186/7$
			SUMA	$616/7 = 88$



Solución: cálculo de los parámetros

$$\theta_1 = \frac{\text{Cov}(x, y)}{\text{Var}(x)} = \frac{88}{28} = \frac{22}{7} \approx 3.1429$$

$$\theta_0 = \bar{y} - \theta_1 \bar{x} = \frac{197}{7} - 3 \left(\frac{22}{7} \right) = \frac{197 - 66}{7} = \frac{131}{7} \approx 18.7143$$

Modelo de regresión óptimo:

$$\hat{y} = \frac{131}{7} - \frac{22}{7}x$$

$$\hat{y} \approx 18.7143 - 3.1429x$$



Ajuste de parámetros con *Numpy*

- *NumPy* permite implementar la solución de forma directa.
- Opera sobre arreglos completos, evitando bucles explícitos.
- Las operaciones reflejan las fórmulas matemáticas.

```
# Calcular medias
x_media = np.mean(x)
y_media = np.mean(y)

# Calcular pendiente
covarianza = np.sum((x - x_media) * (y - y_media))
varianza = np.sum((x - x_media) ** 2)
theta1_opt = covarianza / varianza

# Calcular intercepto
theta0_opt = y_media - theta1_opt * x_media
```



Regresión Lineal con *scikit-learn*

- No es necesario implementar las fórmulas manualmente.
- Requiere que X sea una matriz de 2 dimensiones (incluso para una variable).

```
# Calcular medias
from sklearn.linear_model import LinearRegression

# Preparar datos
X = x.reshape(-1, 1) # Convertir a matriz

# Crear y entrenar modelo
rl = LinearRegression()
rl.fit(X, y)

# Obtener parámetros
print(f"Theta0: {rl.intercept_:.2f}, theta1: {rl.coef_[0]:.2f}")
```



Predicción con *scikit-learn*

- Los datos de prueba y entrenamiento deben tener la misma forma.
- `predict()` aplica el modelo a los nuevos datos y devuelve las predicciones.
- `zip()` empareja cada ejemplo de prueba con su predicción para mostrarlas.

```
X_prueba = np.array([7, 8, 9]).reshape(-1, 1)           # Entradas
predicciones = rl.predict(X_prueba)                     # Predicciones

for prueba, prediccion in zip(X_prueba, predicciones):
    print(f"Prueba {prueba[0]}, predicción {prediccion:.2f}")
```



Evaluación de los modelos de predicción

- Entrenar un modelo no es suficiente; se debe de evaluar su rendimiento.
- Las métricas cuantifican la calidad de las predicciones.
- Permiten comparar diferentes modelos y seleccionar el mejor.
- Guían el ajuste de hiperparámetros y la toma de decisiones.
- Una buena métrica debe ser interpretable y relevante para el problema.



Métricas de error más comunes

Métrica	Características	Fórmula
MSE (<i>Mean Squared Error</i>) Promedio de errores al cuadrado	Penaliza errores grandes. Unidades al cuadrado.	$\text{MSE} = \frac{\sum_i (y_i - \hat{y}_i)^2}{n}$
RMSE (<i>Root Mean Squared Error</i>) Raíz cuadrada del MSE	Interpretación directa. Unidades originales.	$\text{RMSE} = \sqrt{\frac{\sum_i (y_i - \hat{y}_i)^2}{n}}$
MAE (<i>Mean Absolute Error</i>) Promedio de errores absolutos	Robusto a valores atípicos. Unidades originales.	$\text{MAE} = \frac{\sum_i y_i - \hat{y}_i }{n}$
R² (<i>Determination Coefficient</i>) Coeficiente de determinación	Proporción de varianza explicada. Rango [0,1]; 1 es ajuste perfecto.	$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$



Evaluación usando *scikit-learn*

- Una vez entrenado el modelo, se generan predicciones sobre la prueba.
- *scikit-learn* proporciona funciones para calcular métricas de evaluación.
- Todas las métricas comparan los valores reales y valores predichos.
- El orden de los argumentos es siempre (y_real, y_pred).

```
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.metrics import r2_score
```

```
y_pred = rl.predict(X)
```

```
print(f"MSE = {mean_squared_error(y, y_pred):.2f}")
print(f"RMSE = {np.sqrt(mean_squared_error(y, y_pred)):.2f}")
print(f"MAE = {mean_absolute_error(y, y_pred):.2f}")
print(f"R2 = {r2_score(y, y_pred):.4f}")
```



Ejercicio: ajuste de una regresión lineal simple

- Carga el conjunto de datos `penguins` directamente de `seaborn`.
- Obtén la matriz de correlación usando el método `corr()` del dataframe.
- Selecciona un par de variables con un coeficiente superior a 0.7.
- Calcula los parámetros θ_0 y θ_1 usando `Numpy` y `Scikit-learn`.



División del conjunto de datos

- Entrenar y evaluar con los mismos datos da una visión optimista.
- El modelo podría simplemente memorizar (sobreajuste) en vez de aprender.
- Se necesita simular cómo se comportará con datos nuevos no vistos.
- La división permite estimar la capacidad de generalización del modelo.



Conjuntos de entrenamiento y prueba

- Entrenamiento (**training set**): datos con los que el modelo aprende.
- Prueba (**test set**): datos que el modelo no ve durante el entrenamiento.
- Se usan solo al final para evaluar el rendimiento real.
- Proporción típica: 70-80% entrenamiento, 20-30% prueba.

```
from sklearn.model_selection import train_test_split

# Dividir datos: 80% entrenamiento, 20% prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print(f"Entrenamiento: {len(X_train)} ejemplos")
print(f"Prueba: {len(X_test)} ejemplos")
```



Tarea 4: Ajuste de una regresión lineal simple

- Utiliza un conjunto de datos precargado en `seaborn`.
- Obtén la matriz de correlación usando el método `corr()` del dataframe.
- Selecciona un par de variables con un coeficiente superior a 0.7.
- Divide el conjunto de datos usando una proporción de 30% para prueba.
- Ajusta un modelo de regresión lineal con el 70% de los datos.
- Calcula las cuatro métricas de error sugeridas.



Regresión lineal múltiple

- Modela la relación entre varias variables independientes y una dependiente.
- Permite capturar relaciones más complejas usando más información.
- Cada variable predictora tiene su propio coeficiente que indica su impacto.
- Con D variables predictoras:

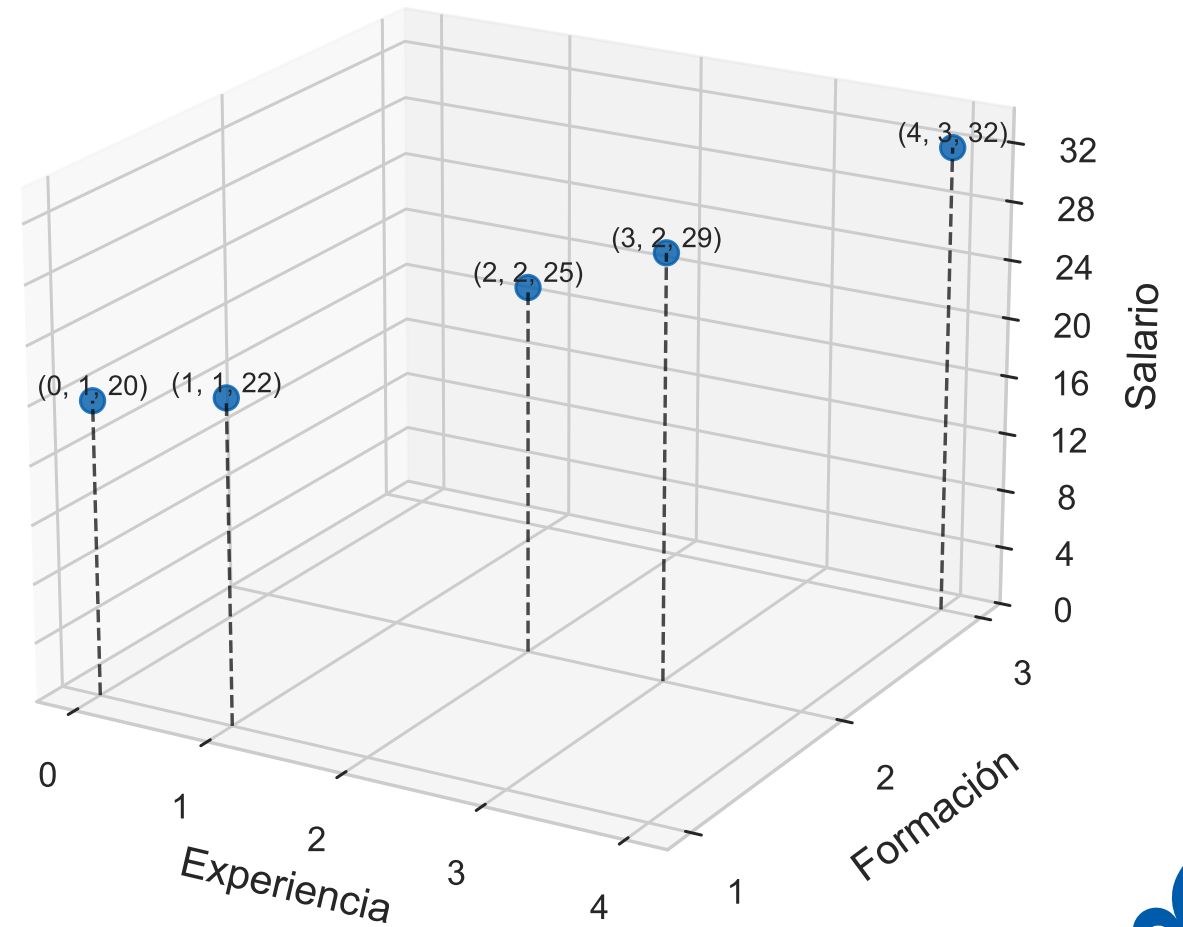
$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_D x_D$$

- En notación vectorial: $\hat{y} = \boldsymbol{\theta}^T \boldsymbol{x}$
- Donde:
 - $\boldsymbol{\theta} = [\theta_0, \theta_1, \dots, \theta_D]^T$ es un vector de parámetros.
 - $\boldsymbol{x} = [1, x_0, x_1, \dots, x_D]^T$ es un vector de características.



Salario en función de la experiencia y la formación

Experiencia (x_1)	Formación (x_2)	Salario (y)
0	1	20
1	1	22
2	2	25
3	2	29
4	3	32



Método de mínimos cuadrados

- La función de pérdida de mínimos cuadrados se escribe como:

$$L[\boldsymbol{\theta}] = (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

- Expandido:

$$L[\boldsymbol{\theta}] = \sum_i (\theta_0 + \theta_1 x_{i,1} + \theta_2 x_{i,2} + \dots - y_i)^2$$

- El objetivo es encontrar $\boldsymbol{\theta}$ que minimice esta pérdida.
- Derivando e igualando a cero, se obtiene la solución cerrada:

$$\boldsymbol{\theta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- La función normal proporciona los parámetros óptimos directamente.



Ejercicio: regresión lineal múltiple

- A partir del conjunto de datos sobre salarios, calcula los parámetros θ_0 , θ_1 y θ_2 , usando el método de mínimos cuadrados con matrices.

Experiencia (x_1)	0	1	2	3	4	5	6
Formación (x_2)	1	1	2	2	3	3	4
Salario (y)	20	22	25	29	32	32	40

```
# Valores de los atributos dentro de una matriz
X = np.array([[0, 1], [1, 1], [2, 2], [3, 2],
              [4, 3], [5, 3], [6, 4]])

# Valores de la variable objetivo
y = np.array([20, 22, 25, 29, 32, 36, 40])
```



Solución: cálculo de $X^T X$

$$X = \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 3 & 2 \\ 1 & 4 & 3 \end{bmatrix}$$

$$X^T X = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 4 \\ 1 & 1 & 2 & 2 & 3 \end{bmatrix} \begin{bmatrix} 1 & 0 & 1 \\ 1 & 1 & 1 \\ 1 & 2 & 2 \\ 1 & 3 & 2 \\ 1 & 4 & 3 \end{bmatrix} =$$

$$\begin{bmatrix} 1 + 1 + 1 + 1 + 1 & 0 + 1 + 2 + 3 + 4 & 1 + 1 + 2 + 2 + 3 \\ 0 + 1 + 2 + 3 + 4 & 0 + 1 + 4 + 9 + 16 & 0 + 1 + 4 + 6 + 12 \\ 1 + 1 + 2 + 2 + 3 & 0 + 1 + 4 + 6 + 12 & 1 + 1 + 4 + 4 + 9 \end{bmatrix} = \begin{bmatrix} 5 & 10 & 9 \\ 10 & 30 & 23 \\ 9 & 23 & 19 \end{bmatrix}$$



Solución: cálculo del determinante $X^T X$

$$\det(A) = a_{11}(a_{22}a_{33} - a_{23}a_{32}) - a_{12}(a_{21}a_{33} - a_{23}a_{31}) + a_{13}(a_{21}a_{32} - a_{22}a_{31})$$

$$\det(A) = 5 \cdot (30 \cdot 19 - 23 \cdot 23) - 10 \cdot (10 \cdot 19 - 23 \cdot 9) + 9 \cdot (10 \cdot 23 - 30 \cdot 9)$$

$$\det(A) = 5 \cdot (570 - 529) - 10 \cdot (190 - 207) + 9 \cdot (230 - 270)$$

$$\det(A) = 5 \cdot 41 - 10 \cdot (-17) + 9 \cdot (-40)$$

$$\det(A) = 205 + 170 - 360 = 15$$



Solución: cálculo de la adjunta $X^T X$

$$\text{adj}(A) = \begin{bmatrix} \det \begin{pmatrix} 30 & 23 \\ 23 & 19 \end{pmatrix} & -\det \begin{pmatrix} 10 & 23 \\ 9 & 19 \end{pmatrix} & \det \begin{pmatrix} 10 & 30 \\ 9 & 23 \end{pmatrix} \\ -\det \begin{pmatrix} 10 & 9 \\ 23 & 19 \end{pmatrix} & \det \begin{pmatrix} 5 & 9 \\ 9 & 19 \end{pmatrix} & -\det \begin{pmatrix} 5 & 10 \\ 9 & 23 \end{pmatrix} \\ \det \begin{pmatrix} 10 & 9 \\ 30 & 23 \end{pmatrix} & -\det \begin{pmatrix} 5 & 9 \\ 10 & 23 \end{pmatrix} & \det \begin{pmatrix} 5 & 10 \\ 10 & 30 \end{pmatrix} \end{bmatrix}$$

$$\text{adj}(A) = \begin{bmatrix} (30 \cdot 19 - 23 \cdot 23) & -(19 \cdot 19 - 23 \cdot 9) & (10 \cdot 23 - 30 \cdot 9) \\ -(10 \cdot 19 - 9 \cdot 23) & (5 \cdot 19 - 9 \cdot 9) & -(5 \cdot 23 - 10 \cdot 9) \\ (10 \cdot 23 - 9 \cdot 30) & -(5 \cdot 23 - 9 \cdot 10) & (5 \cdot 30 - 10 \cdot 10) \end{bmatrix}$$

$$\text{adj}(A) = \begin{bmatrix} 41 & 17 & -40 \\ 17 & 14 & -25 \\ -40 & -25 & 50 \end{bmatrix}$$



Solución: cálculo de la inversa $X^T X$

$$A^{-1} = \frac{1}{\det(A)} \text{adj}(A)$$

$$(X^T X)^{-1} = \frac{1}{15} \begin{bmatrix} 41 & 17 & -40 \\ 17 & 14 & -25 \\ -40 & -25 & 50 \end{bmatrix}$$



Solución: cálculo de $X^T \mathbf{y}$

$$X^T \mathbf{y} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 1 & 2 & 3 & 4 \\ 1 & 1 & 2 & 2 & 3 \end{bmatrix} \begin{bmatrix} 20 \\ 22 \\ 25 \\ 29 \\ 32 \end{bmatrix}$$

$$X^T \mathbf{y} = \begin{bmatrix} 1 \cdot 20 + 1 \cdot 22 + 1 \cdot 25 + 1 \cdot 29 + 1 \cdot 32 \\ 0 \cdot 20 + 1 \cdot 22 + 2 \cdot 25 + 3 \cdot 29 + 4 \cdot 32 \\ 1 \cdot 20 + 1 \cdot 22 + 2 \cdot 25 + 2 \cdot 29 + 3 \cdot 32 \end{bmatrix}$$

$$X^T \mathbf{y} = \begin{bmatrix} 128 \\ 287 \\ 246 \end{bmatrix}$$



Solución: cálculo de $(X^T X)^{-1} X^T \mathbf{y}$

$$\boldsymbol{\theta} = (X^T X)^{-1} X^T \mathbf{y}$$

$$\boldsymbol{\theta} = \frac{1}{15} \begin{bmatrix} 41 & 17 & -40 \\ 17 & 14 & -25 \\ -40 & -25 & 50 \end{bmatrix} \begin{bmatrix} 128 \\ 287 \\ 246 \end{bmatrix}$$

$$\boldsymbol{\theta} = \frac{1}{15} \begin{bmatrix} 41 \cdot 128 + 17 \cdot 287 - 40 \cdot 246 \\ 17 \cdot 128 + 14 \cdot 287 - 25 \cdot 246 \\ -40 \cdot 128 - 25 \cdot 287 + 50 \cdot 246 \end{bmatrix}$$

$$\boldsymbol{\theta} = \frac{1}{15} \begin{bmatrix} 287 \\ 44 \\ 5 \end{bmatrix} \approx \begin{bmatrix} 19.1333 \\ 2.9333 \\ 0.3333 \end{bmatrix}$$



Cálculo de parámetros usando *NumPy*

- La ecuación normal proporciona una solución directa.
- Requiere construir la matriz de diseño X que incluye una columna de unos.
- Los coeficientes resultantes minimizan la suma de errores cuadrados.

```
# Construir matriz de diseño con columna de 1s
X_con_intercepto = np.c_[np.ones(X.shape[0]), X]

# Calcular  $XTX$  y  $XTy$ 
X_T = X_con_intercepto.T
XTX = X_T @ X_con_intercepto
XTy = X_T @ y

# Resolver ecuación normal
theta_opt = np.linalg.inv(XTX) @ XTy
```



Problema de colinealidad

- Ocurre cuando dos o más variables están fuertemente relacionadas.
- Ejemplo típico: años de experiencia y años en la misma empresa .
- Consecuencias:
 - La matriz $X^T X$ se vuelve casi singular (determinante cercano a cero).
 - Pequeños cambios en datos generan grandes variaciones en parámetros.
 - La ecuación normal puede producir resultados incorrectos.
- Soluciones:
 - Eliminar una de las variables redundantes.
 - Recurrir a métodos más estables como el usado por [scikit-learn](#).



Regresión lineal múltiple con *scikit-learn*

- *scikit-learn* añade internamente la columna de unos.
- Maneja casos de colinealidad, aunque se siendo recomendable revisarla.
- Es la herramienta estándar en la práctica profesional.

```
from sklearn.linear_model import LinearRegression

# Crear y entrenar modelo
modelo_rl = LinearRegression()
modelo_rl.fit(X, y)

# Obtener parámetros
print(f"Intercepto: {modelo_rl.intercept_:.4f}")
print(f"Coeficientes: {modelo_rl.coef_}")
```



Predicción con el modelo entrenado

- Los datos nuevos deben tener la misma estructura del entrenamiento.
- No es necesario añadir la columna de unos; se maneja internamente.

```
# Nuevos datos: [experiencia, formación]
X_nuevos = np.array([[2.5, 2],
                     [5.0, 4],
                     [7.0, 3]])

predicciones = modelo_rl.predict(X_nuevos)

for i, (exp, form) in enumerate(X_nuevos):
    print(f"Exp: {exp}, Form: {form}, Pred: {predicciones[i]:.2f}")
```



Cálculo de métricas de error con *scikit-learn*

- El cálculo de métricas es idéntico al ejemplo de la regresión lineal simple.
- El único cambio es que el objeto representa una regresión lineal múltiple.

```
from sklearn.metrics import mean_squared_error, mean_absolute_error
from sklearn.metrics import r2_score

y_pred = modelo_rl.predict(X_prueba)

mse = mean_squared_error(y, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y, y_pred)
r2 = r2_score(y, y_pred)
```



Ejercicio: Ajuste de una regresión lineal múltiple

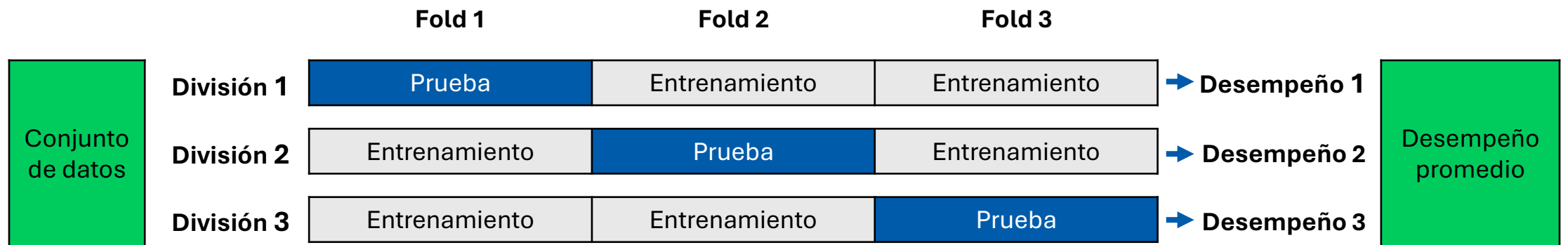
1. Carga el conjunto de datos `'mpg'` de seaborn.
2. Elimina los valores faltantes (`dropna()`).
3. Selecciona la variable `'mpg'` (millas por galón) como dependiente.
4. Calcula la matriz de correlación entre todas las variables numéricas.
5. Identifica las tres variables predictoras con mayor correlación con `'mpg'`.
6. Extrae esas tres variables como matriz X.
7. Ajusta un modelo de regresión lineal múltiple:
 - Usando `NumPy` (forma cerrada con ecuación normal) y `scikit-learn`.
8. Compara los coeficientes obtenidos.
9. Evalúa e interpreta el desempeño del modelo usando todas las métricas.



Validación cruzada

Cross-Validation (CV)

- Técnica para evaluar el rendimiento de un modelo.
- No depende de una sola partición entrenamiento/prueba.
- Algoritmo:
 1. Dividir los datos en k subconjuntos (*folds*) de igual tamaño.
 2. Ajustar el modelo k veces con $k-1$ subconjuntos y el restante de prueba.
 3. Promediar las métricas de error obtenidas.



Comparación de modelos con validación cruzada

- La validación cruzada ayuda a comparar algoritmos y elegir el mejor.
- Pero el modelo que usamos debe entrenarse con más datos.
- Pasos para seleccionar el modelo final:
 1. Evaluar diferentes opciones (algoritmos) con validación cruzada.
 2. Identificar la configuración con mejor rendimiento promedio.
 3. Entrenar un modelo con la mejor configuración y todos los datos.
 4. Este modelo final es el que se despliega para predecir nuevos casos.



Ventajas de la validación cruzada

- Es adecuada cuando se tienen pocos datos.
- Cada observación se usa tanto para entrenar como para probar.
- Evalúa el modelo con múltiples conjuntos de prueba diferentes.
- El promedio de varias pruebas es más confiable que una sola.
- Reduce el impacto de una partición "afortunada" o "desafortunada".
- Detecta el sobreajuste cuando el modelo funciona en el entrenamiento, pero mal en la validación.



Validación cruzada en *scikit-learn*

- Retorna un arreglo con los puntajes de cada subconjunto.
- El promedio da una estimación robusta del rendimiento.
- La desviación indica qué tan estable es el modelo en diferentes particiones.
- Otras métricas pueden usarse cambiando el parámetro `scoring`, como:
 - `'r2'`, `'neg_mean_absolute_error'`, `'neg_mean_squared_error'`.

```
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LinearRegression

modelo = LinearRegression()
puntajes = cross_val_score(modelo, X, y, cv=5, scoring='r2')
print(f"R2 promedio: {puntajes.mean():.4f}")
print(f"Desviación estándar: {puntajes.std():.4f}")
```



Tarea 5: Ajuste de una regresión lineal múltiple

- Selecciona un conjunto de datos de [seaborn](#) o de un repositorio de datos.
- Define una variable dependiente numérica con sentido práctico.
- Calcula la matriz de correlación y selecciona dos grupos de atributos:
 - Grupo A: dos variables con alta correlación con la variable dependiente.
 - Grupo B: dos variables diferentes con correlación moderada.
- Para cada grupo:
 - Define un modelo de regresión lineal múltiple con [scikit-learn](#).
 - Evalúa mediante CV de 5 *folds* usando [RMSE](#) promedio y desviación.
- Entrena el modelo final con todos los datos y muestra sus coeficientes.
- Entrega una libreta con código, comparación de resultados y conclusiones.

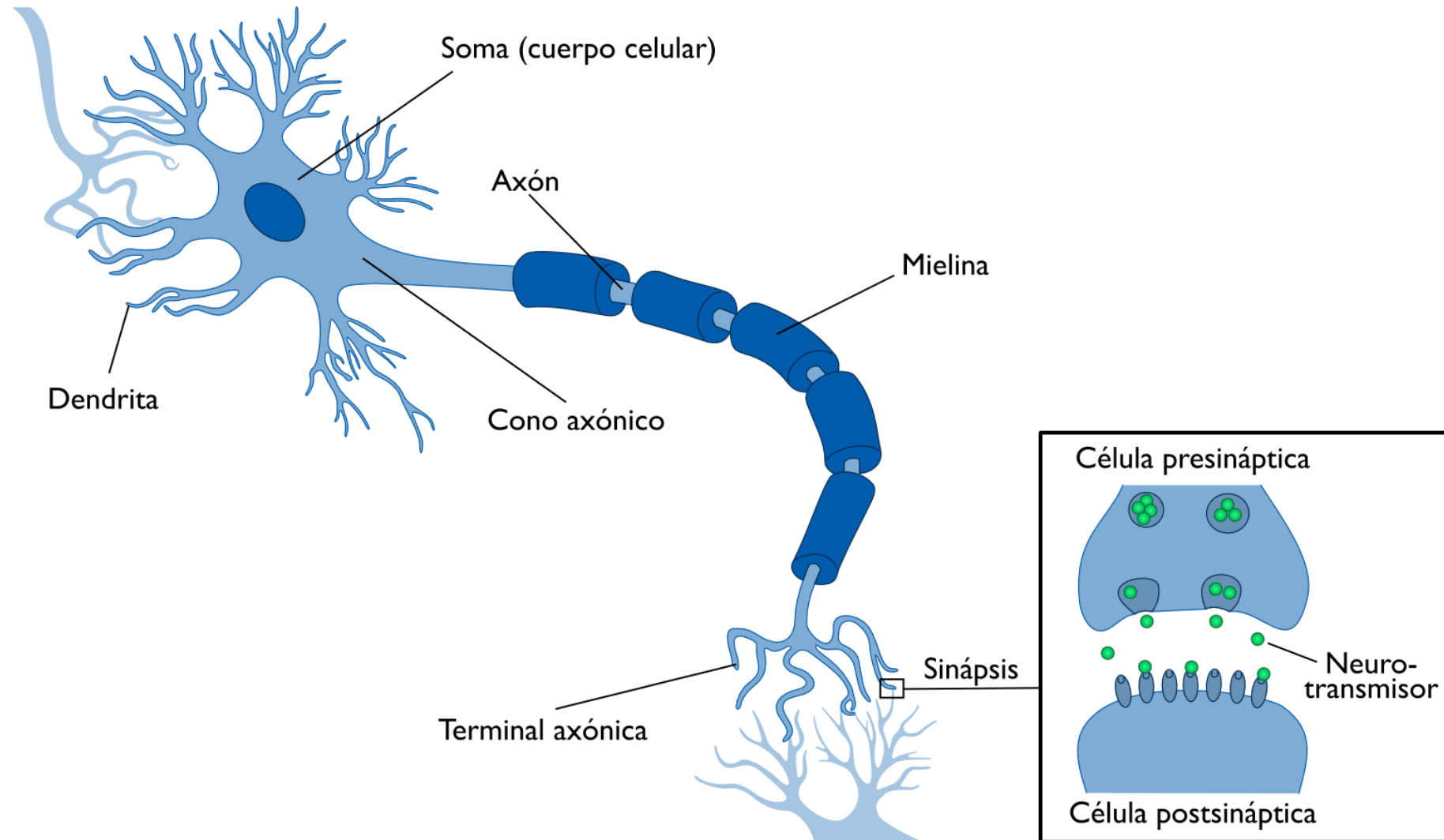


Neurona biológica

- El cerebro humano contiene aproximadamente 86 mil millones de neuronas.
- La neurona es una célula especializada en procesar y transmitir información.
- Morfología:
 - Las dendritas son prolongaciones que reciben señales de otras neuronas.
 - El soma o cuerpo celular integra las señales recibidas.
 - Si la suma supera un umbral, la neurona se activa y genera una señal.
 - El axón transmite la señal de salida a otras neuronas.
 - La sinapsis es la conexión entre un axón y las dendritas de otra neurona.



Morfología de la neurona biológica



Neurona artificial

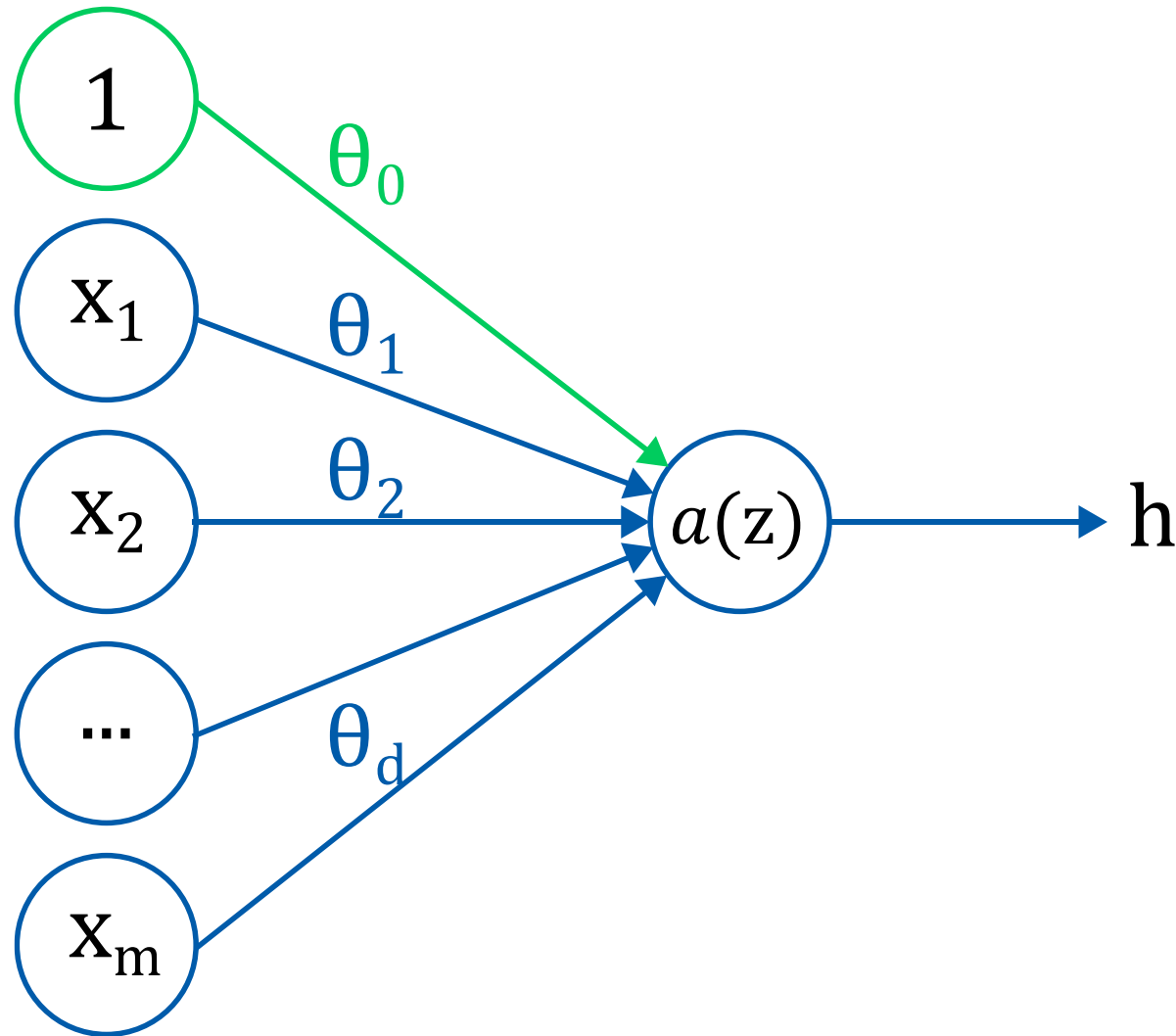
- Es una función no lineal de una combinación lineal de las entradas.
- Se inspira en la neurona biológica, pero es un modelo matemático simple.

$$h = a \left(\theta_0 + \sum_{i=1}^m \theta_i x_i \right)$$

- Donde:
 - $x_1, x_2, \dots, x_m$ son las entradas o variables predictoras.
 - θ_0 es el sesgo (*bias*), análogo al intercepto en regresión.
 - θ_i son los pesos (*weights*), análogos a los coeficientes de regresión.
 - $a(\cdot)$ es la función de activación no lineal.
 - h es la salida de la neurona (activación).



Morfología de la neurona artificial



Función de activación

- Una neurona aplica una función de activación a una combinación lineal.

$$z = \theta_0 + \theta_1 x_1 + \dots + \theta_m x_m$$

$$h = a(z)$$

- Sin función de activación, la neurona es idéntica a una regresión lineal.

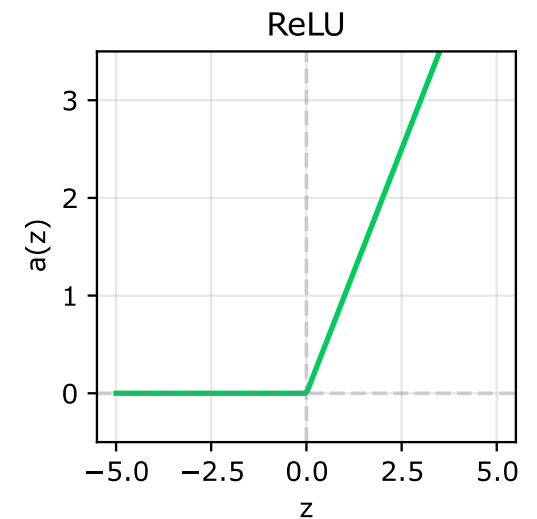
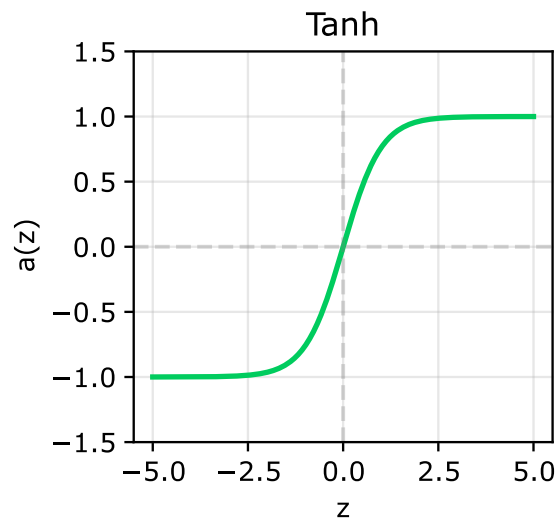
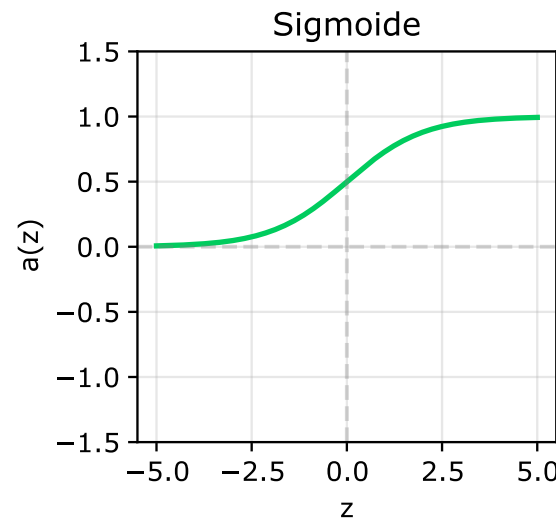
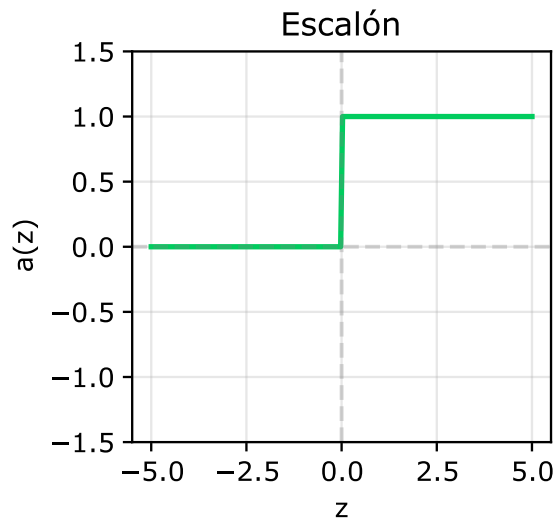
$$h = \theta_0 + \theta_1 x_1 + \dots + \theta_m x_m$$

- La función de activación transforma la combinación lineal.
- Esto permite representar relaciones más complejas que un simple plano.
- La no linealidad sirve para que la red pueda aprender patrones complejos.
- Sin ella, apilar múltiples neuronas seguiría siendo lineal.



Funciones comunes de activación

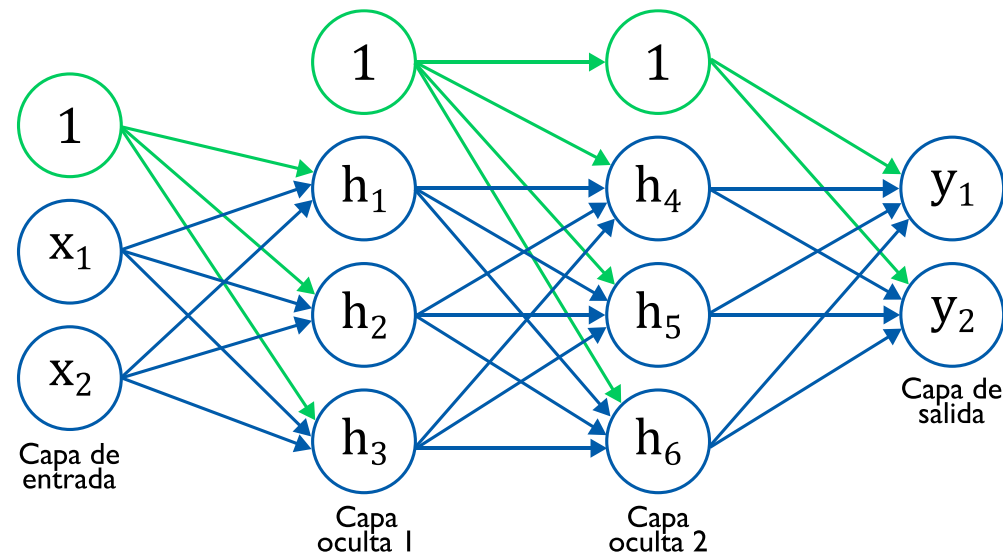
- Escalón: Devuelve 1 si la entrada supera un umbral, 0 en caso contrario.
- Sigmoide: Convierte cualquier entrada a un valor entre 0 y 1.
- Tanh: Similar a sigmoide pero centrada en cero con una salida entre -1 y 1.
- ReLU (*Rectified Linear Unit*): Lineal para valores positivos y 0 para negativos.



Redes neuronales superficiales

Shallow Neural Networks (SNN)

- Extiende la idea de una neurona individual a múltiples neuronas en paralelo.
- Es una red con pocas capas ocultas de neuronas, generalmente una o dos.
- Permite representar funciones mucho más complejas que una sola neurona.
- La estructura de una SNN puede tener múltiples entradas y salidas.



Redes neuronales para regresión

- Aunque las redes sirven para clasificación, este curso se centra en regresión.
- La capa de entrada puede representar a un valor escalar o un vector.
- Una sola neurona en la última capa produce como salida un valor continuo.
- El objetivo es minimizar la pérdida ajustando todos los parámetros de la red.
- La función de pérdida es el MSE, igual que en regresión lineal.



Arquitectura sugerida de una red neuronal superficial

- La arquitectura de SNN sugerida tiene una capa oculta con p neuronas.
- Cada neurona calcula su activación a partir de las entradas.
- La salida final es una combinación lineal de las activaciones.

$$y = \phi_0 + \sum_{j=1}^p \phi_j \cdot \text{ReLU}(\theta_{j0} + \theta_{j1}x_1 + \dots + \theta_{jm}x_m)$$

- Donde:
 - $\theta_{j0}, \theta_{j1}, \dots, \theta_{jm}$ son los parámetros (pesos y sesgo) de la neurona j .
 - ϕ_j son los pesos de la capa de salida.
 - ϕ_0 es el sesgo de salida.

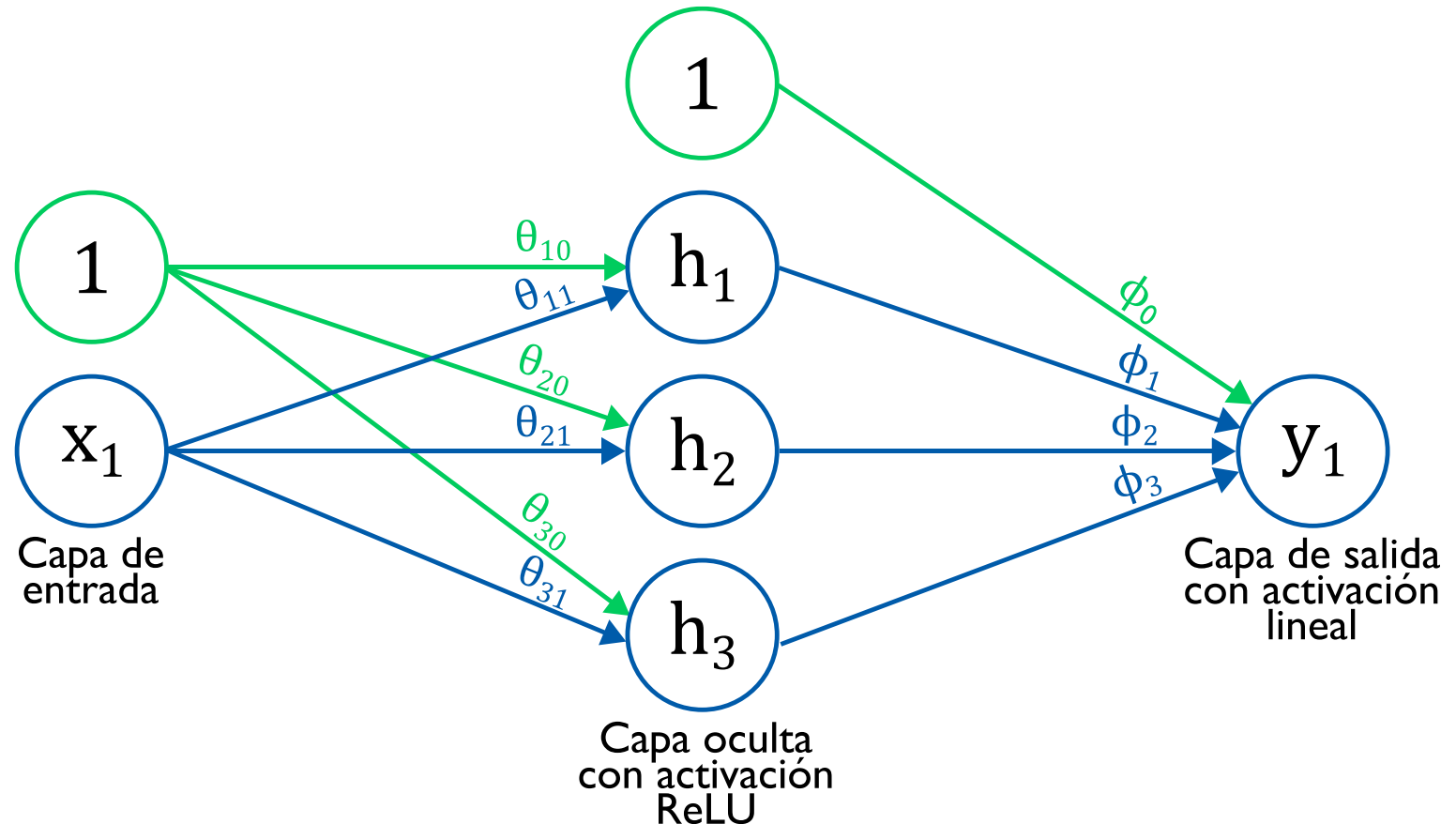


Caso de estudio de una SNN

- Arquitectura de una red neuronal superficial con una neurona de entrada, tres neuronas en la capa oculta y una neurona de salida.

Parámetros

$$\begin{aligned}\theta_{10} &= 0.30 \\ \theta_{11} &= -1.0 \\ \theta_{20} &= -1.0 \\ \theta_{21} &= 2.00 \\ \theta_{30} &= -0.5 \\ \theta_{31} &= 0.65 \\ \phi_0 &= -0.3 \\ \phi_1 &= 2.00 \\ \phi_2 &= -1.0 \\ \phi_3 &= 7.00\end{aligned}$$



Función para graficar el comportamiento de una red

```
def graficar_salidas(x, ys, ylabel='Valor', title='Neurona'):

    fig, axes = plt.subplots(1, 3, figsize=(12,4))
    for i, ax in enumerate(axes):
        ax.plot(x, ys[i], color='#00CC5E', linewidth=2)
        ax.set_title(title+f" {i+1}", fontsize=12)
        ax.set_xlabel('Entrada', fontsize=10)
        ax.set_ylabel(ylabel, fontsize=10)
        ax.set_xlim([0,1])
        ax.set_ylim([-1,1])
        ax.axhline(0, color='black', linestyle='--', alpha=0.7)
        ax.grid(True, alpha=0.3)
    plt.tight_layout()

    return fig
```



Graficación de preactivaciones (z_i)

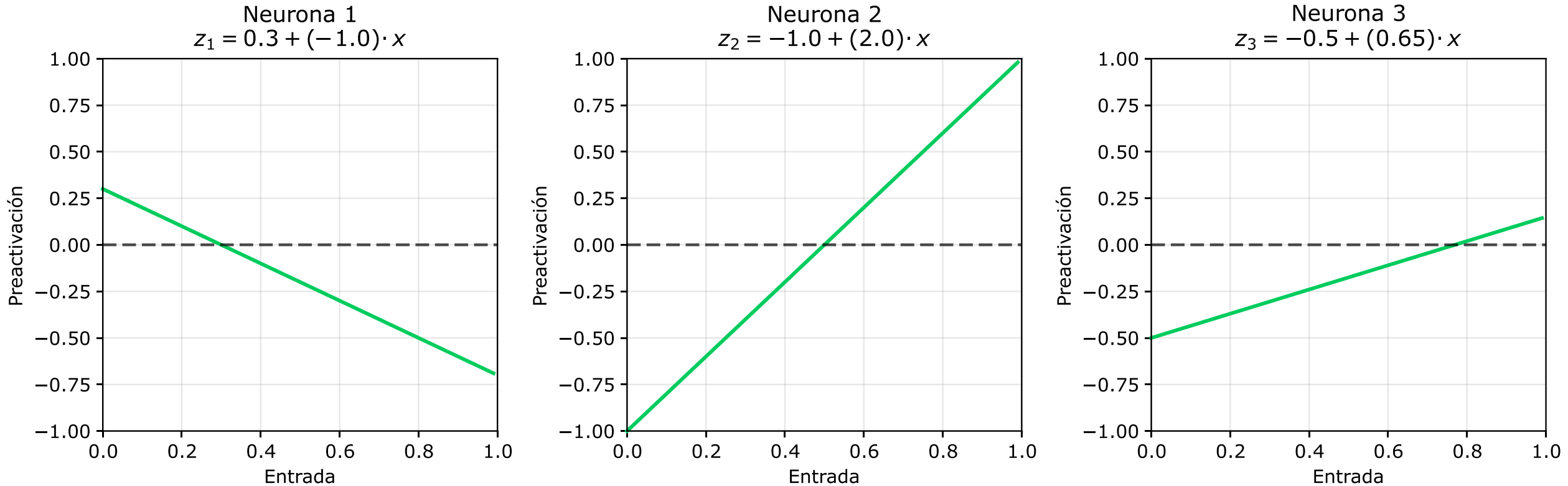
```
# Entradas
x = np.arange(0, 1, 0.01)

# Preactivaciones
thetas = [(0.3, -1.0), (-1.0, 2.0), (-0.5, 0.65)]
zs = [theta[0] + theta[1]*x for theta in thetas]

# Gráfica
fig = graficar_salidas(x, zs, 'Preactivaciones', 'Neurona')
```



Preactivaciones (z_i) de las neuronas ocultas



- Las preactivaciones son combinaciones lineales: $z_i = \theta_{i0} + \theta_{i1}x_1$.
- El parámetro θ_{ij} es el peso que se le asigna a la entrada j de la neurona i .



Graficación de activaciones (h_i)

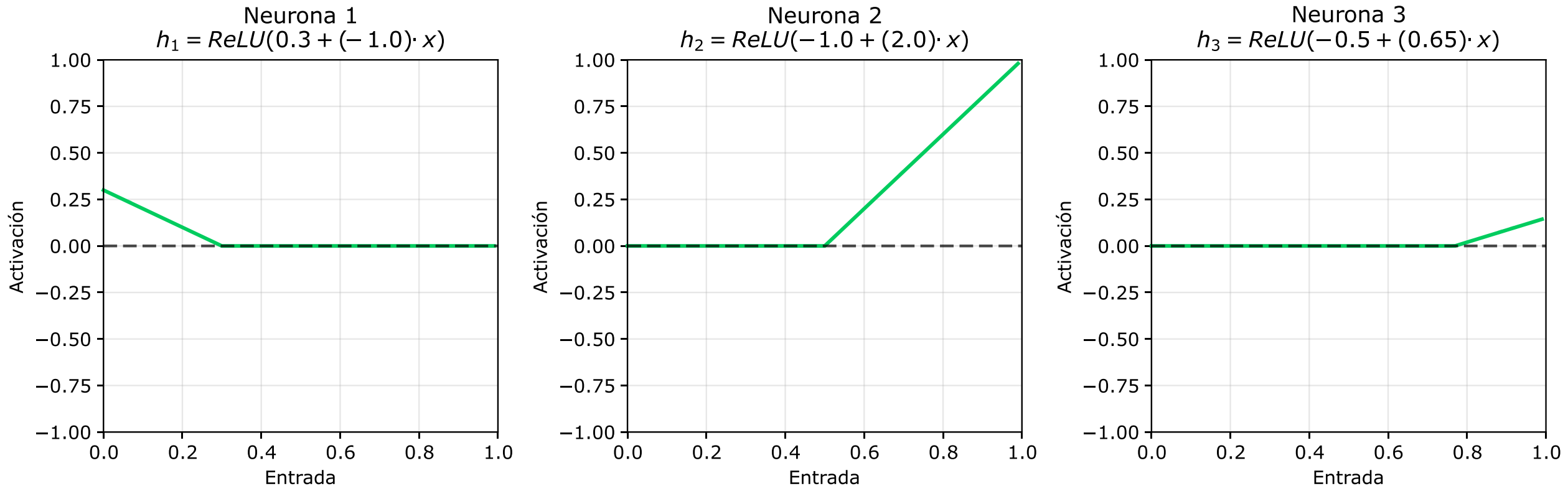
```
# Función de activación
def ReLU(x):
    return np.clip(x, 0, None)

# Activaciones
hs = [ReLU(z) for z in zs]

# Gráfica
fig = graficar_salidas(x, hs, 'Activaciones', 'Neurona')
```



Activaciones (h_i) de las neuronas ocultas



- La salida de la neurona aplica una función no lineal a la preactivación.
- La función ReLU es lineal para valores positivos y cero para negativos.



Graficación de activaciones ponderadas ($\phi_i h_i$)

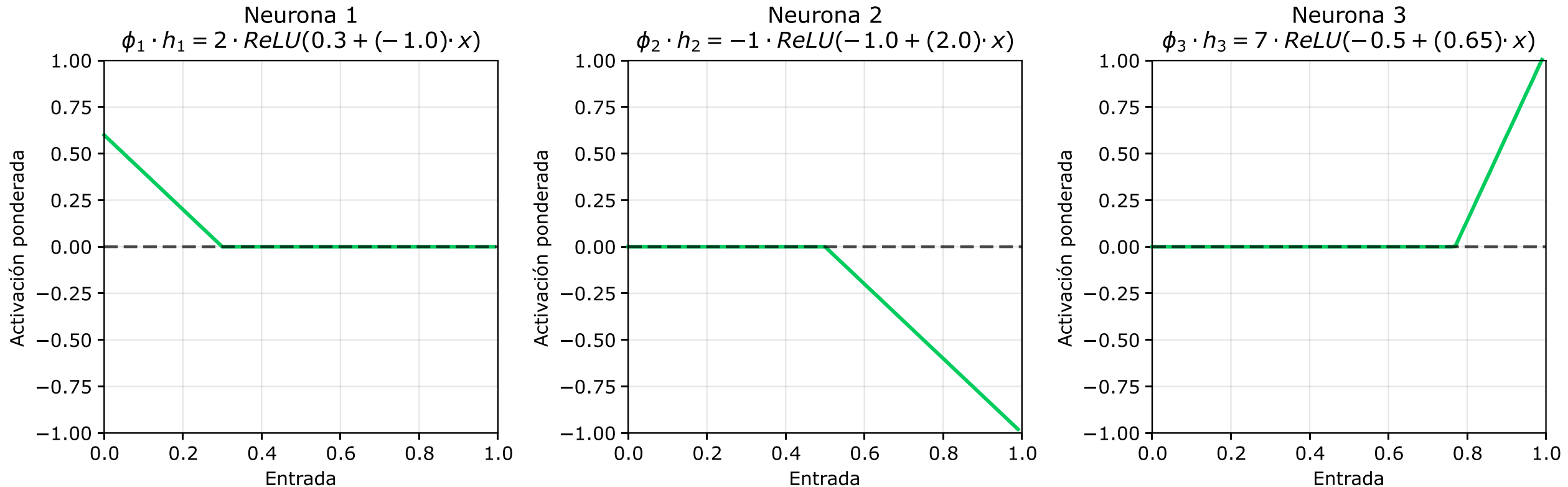
```
# Pesos phi
phis = [2, -1, 7]

# Activaciones ponderadas
whs = [w * h for w, h in zip(phis, hs)]

# Gráfica
fig = graficar_salidas(x, whs, 'Activaciones ponderada', 'Neurona')
```



Activaciones ponderadas ($\phi_i h_i$) de las neuronas ocultas



- Es necesario ponderar las salidas provenientes de las neuronas ocultas.
- Esto permitirá calcular la activación de la neurona de salida.



Graficación de la salida (y_1)

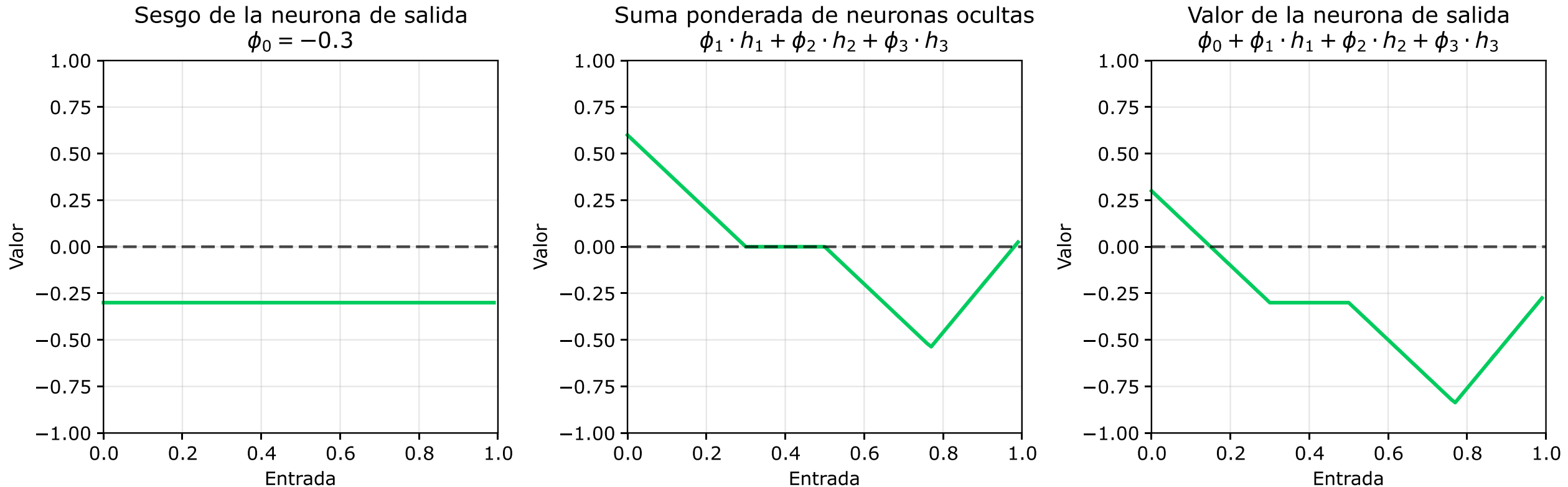
```
# Función de activación
def ReLU(x):
    return np.clip(x, 0, None)

# Activaciones
hs = [ReLU(z) for z in zs]

# Gráfica
fig = graficar_salidas(x, hs, 'Activaciones', 'Neurona')
```



Activación de la neurona de salida (y_1)



- La activación de la neurona de salida es una combinación lineal.
- Incluye el sesgo (ϕ_0) y los pesos de las salidas de otras neuronas (ϕ_i).



Ajuste de una red neuronal

- Ajustar los parámetros para que la predicción $\hat{y}$ se aproxime al valor real y .
- Es necesario minimizar la función de pérdida (MSE).
- Existe la alternativa clásica: backpropagation + descenso de gradiente.
 1. **Foward pass**: Obtener las predicciones y calcular el error.
 2. **Backward pass**: Derivar cómo cambia la pérdida al variar parámetros.
 3. **Actualización de parámetros**: Moverlos en dirección que reduce L .
 4. Repetir con todos los ejemplos y varias épocas hasta converger.
- En la actualidad existen alternativas más robustas.



Evaluación del desempeño de una red neuronal

- La pérdida es un número que mide qué tan mal está prediciendo el modelo.
 - Si la pérdida es alta, las predicciones están lejos de los valores reales.
 - Si la pérdida es baja, el modelo se acerca a la realidad.
- Las curvas de pérdida son una representación gráfica del aprendizaje.
- Muestran el cambio de la pérdida a lo largo del entrenamiento (épocas).
 - Pérdida de entrenamiento: error en los datos con los que se aprende.
 - Pérdida de validación: error en datos que el modelo no ha visto.
- Permiten ver si el modelo aprende, si se estanca o si hay sobreajuste.



Caso de estudio: prediciendo un polinomio

- Se genera un conjunto sintético usando una función de grado 3 con ruido.
 - **Variable independiente:** x con valores uniformes en el rango $[-2, 2]$
 - **Variable dependiente:** y definida por $0.5 + 1.2 \cdot x - 2.5 \cdot x^2 + 1.8 \cdot x^3 + \varepsilon$.
 - Donde $\varepsilon \sim N(0, 0.3)$.
- Pasos a realizar:
 1. Cargar el conjunto y seleccionar las variables de entrada y salida.
 2. Dividir y estandarizar los datos.
 3. Construir y entrenar dos redes neuronales.
 4. Ajustar una regresión lineal simple como modelo de referencia.
 5. Evaluar el desempeño de los tres modelos.



Generación y preprocesamiento de datos

```
# Generación de datos
np.random.seed(42)
n = 500
# Variable independiente
x = np.random.uniform(-3, 3, n)
# Variable objetivo
y = 0.5 + 1.2*x - 2.5*x**2 + 1.8*x**3 + np.random.normal(0, 1, n)

# Dividir en entrenamiento y prueba
x_e, x_p, y_e, y_p = train_test_split(x, y, test_size=0.2,
                                       random_state=42)

# Estandarizar el atributo
escalador = StandardScaler()
x_ee = escalador.fit_transform(x_e.reshape(-1, 1))
x_pe = escalador.transform(x_p.reshape(-1, 1))
```



Visualización de los datos preprocesados

```
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Gráfica 1: Datos originales
axes[0].scatter(x_e, y_e, alpha=0.6, color='#005BAA', label='Train')
axes[0].scatter(x_p, y_p, alpha=0.6, color='#00CC5E', label='Test')
axes[0].set_title('Antes de estandarización', fontsize=14)

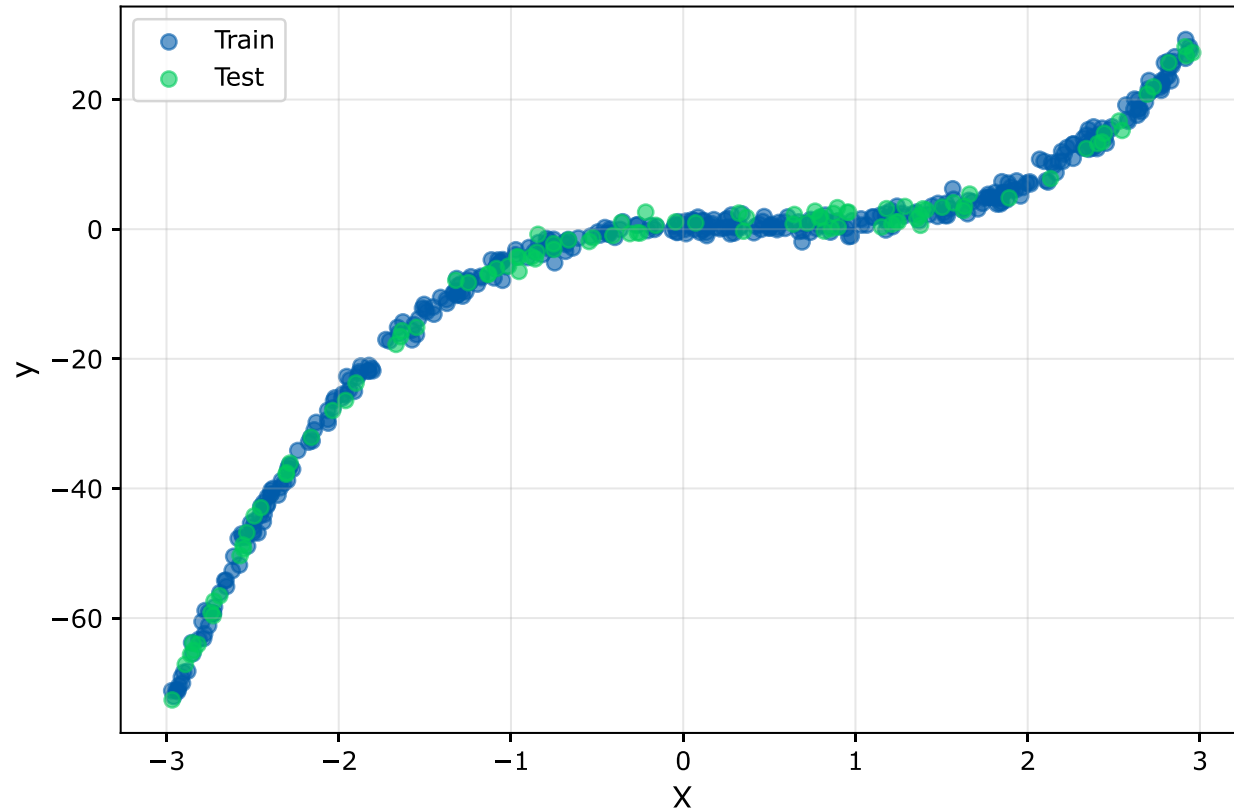
# Gráfica 2: Datos después de la estandarización
axes[1].scatter(x_ee, y_e, alpha=0.6, color='#005BAA', label='Train')
axes[1].scatter(x_pe, y_p, alpha=0.6, color='#00CC5E', label='Test')
axes[1].set_title('Después de estandarización', fontsize=14)

for ax in axes:
    ax.set_xlabel('X', fontsize=12)
    ax.set_ylabel('y', fontsize=12)
    ax.legend()
    ax.grid(alpha=0.3)
```

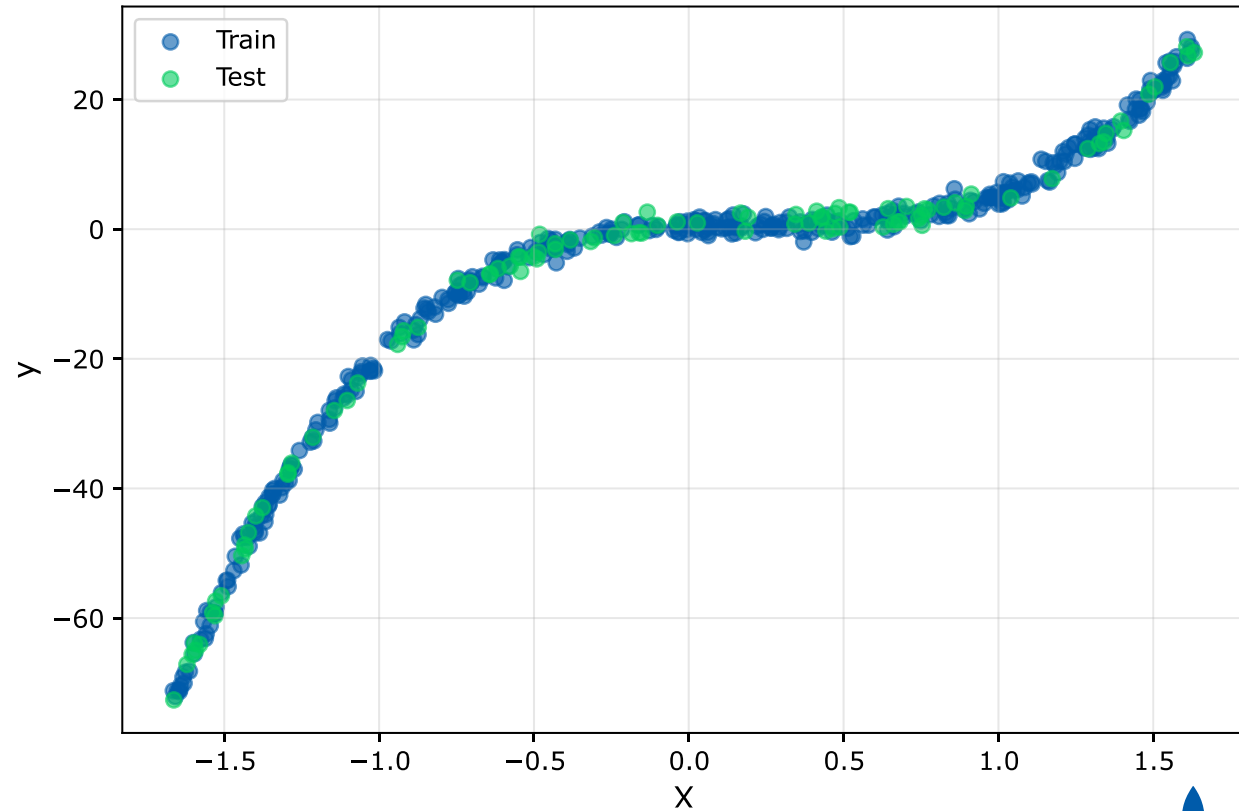


Visualización de la estandarización de datos

Antes de estandarización



Después de estandarización



Función para generar redes neuronales

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Input
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

def construir_snn(neuronas_ocultas):
    modelo = Sequential([
        Input(shape=(1,)),
        Dense(neuronas_ocultas, activation='relu'),
        Dense(1, activation='linear')
    ])
    modelo.compile(optimizer=Adam(learning_rate=0.01), loss='mse')
    return modelo
```



Creación y ajuste de modelos

```
snn5 = construir_snn(5)
hist5 = snn5.fit(x_ee, y_e, epochs=100,
                validation_split=0.2, verbose=0)

snn25 = construir_snn(25)
hist25 = snn25.fit(x_ee, y_e, epochs=100,
                  validation_split=0.2, verbose=0)

reg_lineal = LinearRegression()
reg_lineal.fit(X_ee, y_e)
```



Predicción y evaluación de los modelos

```
y_pred_snn5 = snn5.predict(x_pe, verbose=0).flatten()
y_pred_snn25 = snn25.predict(x_pe, verbose=0).flatten()
y_pred_lineal = reg_lineal.predict(x_pe)

mse_snn5 = mean_squared_error(y_p, y_pred_snn5)
mse_snn25 = mean_squared_error(y_p, y_pred_snn25)
mse_lineal = mean_squared_error(y_p, y_pred_lineal)

print(f"MSE de la SNN 5: {mse_snn5:.4f}")      # 126.4757
print(f"MSE de la SNN 25: {mse_snn25:.4f}")    # 1.1531
print(f"MSE de la RL: {mse_lineal:.4f}")       # 122.9231
```

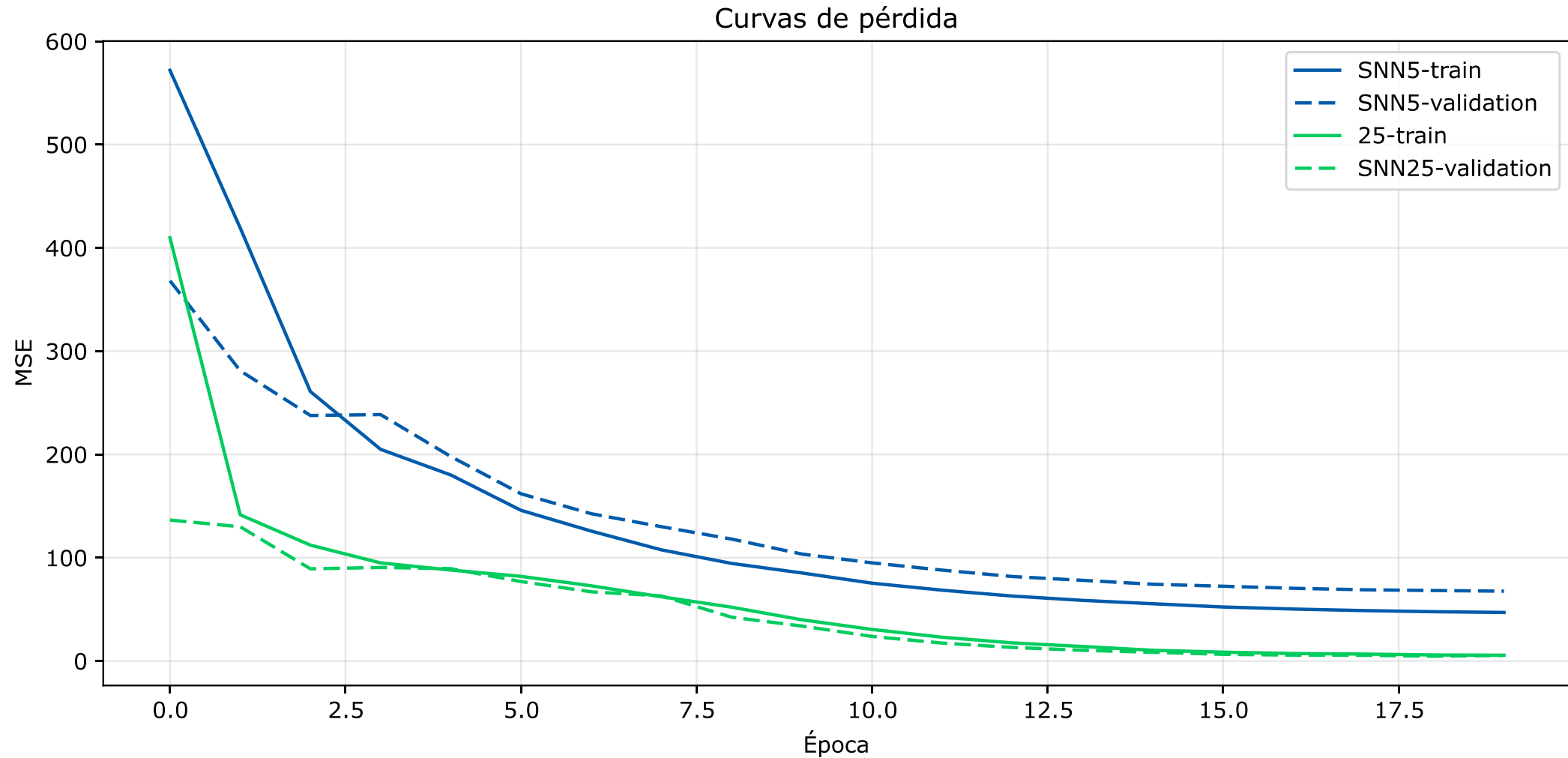


Visualización de las curvas de pérdida

```
plt.figure(figsize=(10, 5))
plt.plot(hist5.history['loss'], color="#005BAA", label='SNN5-train')
plt.plot(hist5.history['val_loss'], color="#005BAA",
         ls='--', label='SNN5-validation')
plt.plot(hist25.history['loss'], color='#00CC5E', label='25-train')
plt.plot(hist25.history['val_loss'], color='#00CC5E',
         ls='--', label='SNN25-validation')
plt.xlabel('Época')
plt.ylabel('MSE')
plt.title('Curvas de pérdida')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
```



Curvas de pérdida



Visualización del ajuste

```
x_pred = np.linspace(-3, 3, 100).reshape(-1, 1)
x_pred_esc = escalador.transform(x_pred)

y_pred_lineal = reg_lineal.predict(x_pred_esc)
y_pred_snn5 = snn5.predict(x_pred_esc, verbose=0).flatten()
y_pred_snn25 = snn25.predict(x_pred_esc, verbose=0).flatten()
```

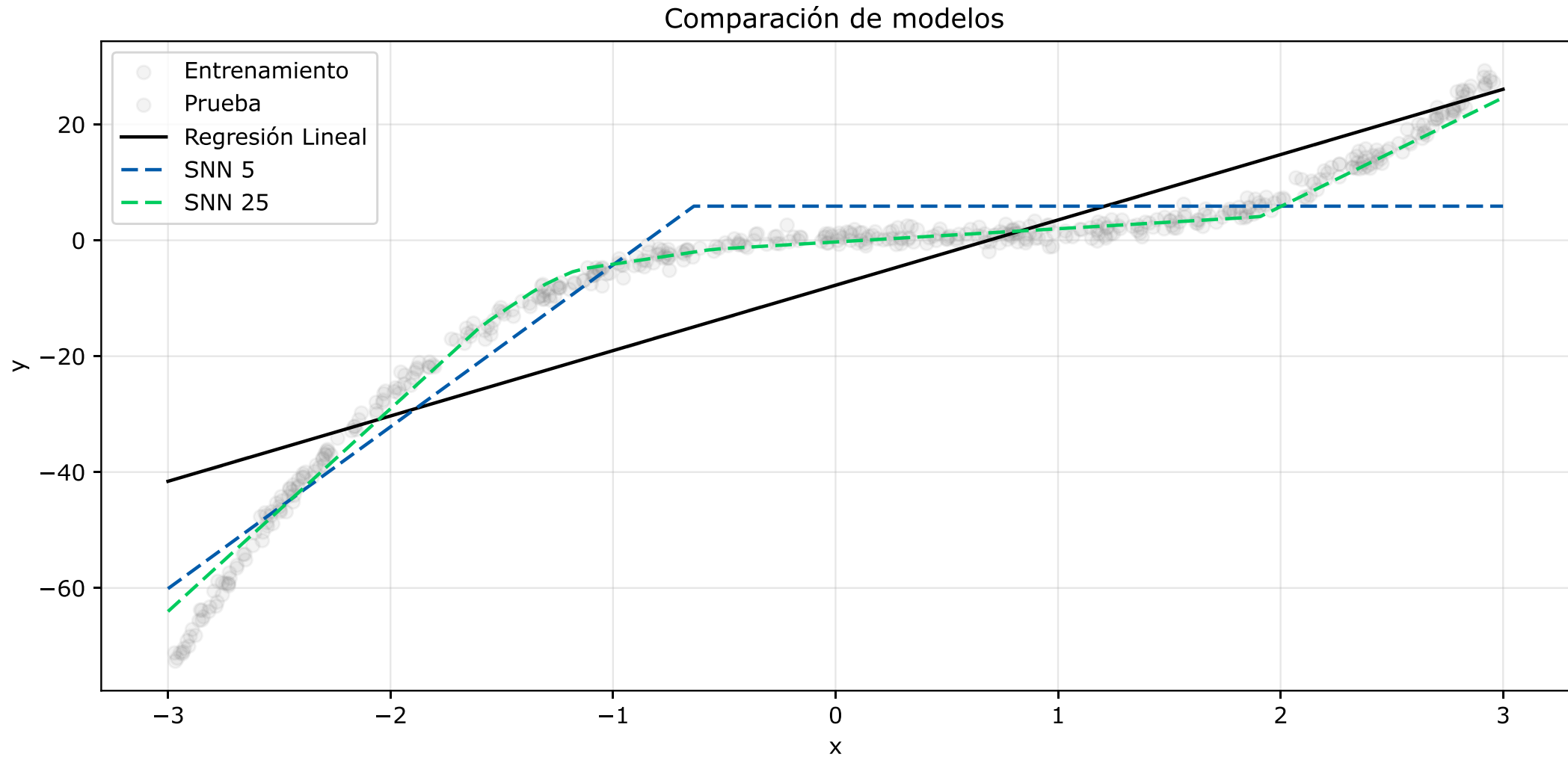


Visualización del ajuste II

```
plt.figure(figsize=(10, 5))
plt.scatter(x_e, y_e, alpha=0.1, color='gray', label='Entrenamiento')
plt.scatter(x_p, y_p, alpha=0.1, color='gray', label='Prueba')
plt.plot(x_pred, y_pred_lineal, color = "black",
         ls = '-', label='Regresión Lineal')
plt.plot(x_pred, y_pred_snn5, color = "#005BAA",
         ls = '--', label='SNN 5')
plt.plot(x_pred, y_pred_snn25, color = "#00CC5E",
         ls = '--', label='SNN 25')
plt.xlabel('x')
plt.ylabel('y')
plt.title('Comparación de modelos')
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

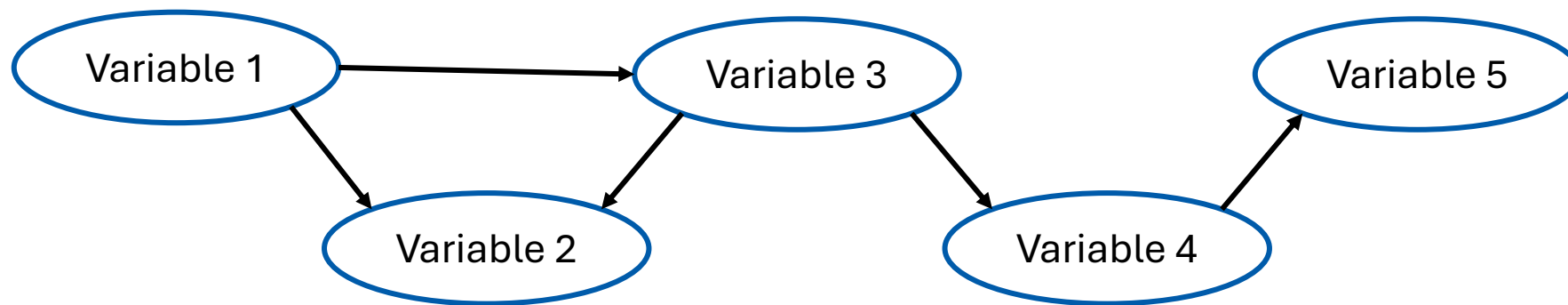


Ajuste de los datos



Fundamentos de redes bayesianas (RB)

- Una red bayesiana es un modelo gráfico de un dominio con incertidumbre.
- Los nodos de la red representan variables aleatorias del problema.
- Las conexiones dirigidas entre nodos indican influencias entre variables.
- La relación se expresa mediante distribuciones de probabilidad condicional.
- Las RB están restringidas a ser grafos acíclicos dirigidos (DAG).
- Esto significa que no se puede volver a un nodo siguiendo las flechas.
- Construir una RB requiere identificar variables y sus relaciones.



Ejemplo: elección de medios de transporte

- Se realiza una encuesta para investigar el uso de medios de **transporte**.
- Los factores demográficos y socioeconómicos pueden **influir** en la elección.
- La **edad** y el **sexo** de una persona influyen en el **nivel formativo** que alcanza.
- La **formación** influye en el tipo de **ocupación** de los encuestados.
- También la **formación** se relaciona con el tamaño de la **ciudad** donde vive.
- Tanto la **ocupación** como la **ciudad** influyen en el **transporte** preferido.
- El objetivo es entender la preferencia entre carro, tren u otros.
- La RB modela estas relaciones de influencia, permitiendo razonar.



Nodos y valores en una red bayesiana

- El primer paso al construir una RB es identificar las variables del dominio.
- Los nodos representan variables, y se debe definir qué valores pueden tomar.
- Por ahora, se consideran solo valores discretos, excluyentes y exhaustivos.
- Existen distintos tipos de nodos discretos, como:
 - Nodos booleanos, que toman valores verdaderos (T) o falsos (F).
 - Nodos con valores ordenados, como la contaminación: baja, media, alta.
 - Nodos con valores numéricos, como la edad, entre 1 y 120 años.
- A veces conviene agrupar valores (como edades) para simplificar el modelo.



Ejemplo de variables y valores

- Las RB se utilizan principalmente para la descripción de dominios.
- Estos modelos pueden representar variables continuas y discretas.
- En este curso se utilizarán para realizar clasificación con variables discretas.

Variable	Descripción	Valores posibles
Edad	Edad del individuo	joven (<30), adulto (30-60), mayor (>60)
Sexo	Sexo biológico	masculino, femenino
Formación	Nivel educativo	básico-medio, superior
Ocupación	Tipo de ocupación	empleado, propio
Ciudad	Tamaño de la ciudad	pequeña, grande
Transporte	Medio de transporte preferido	carro, tren, otro



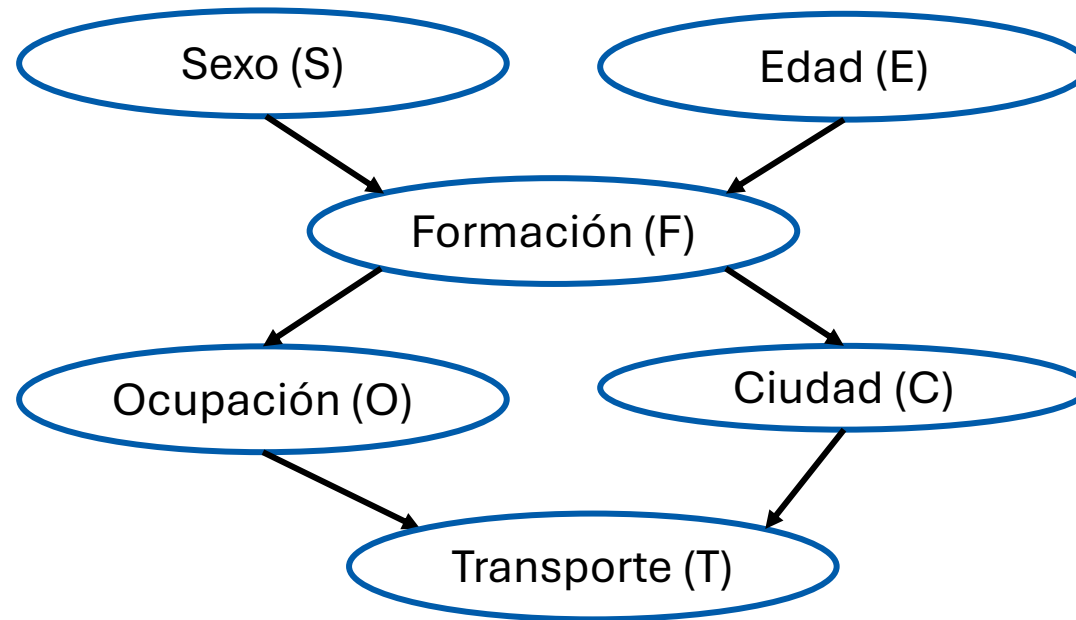
Estructura de una red bayesiana

- La estructura de una RB representa relaciones cualitativas entre variables.
- Dos nodos deben conectarse directamente si uno influye o afecta al otro.
- Para describir una estructura, se usa una metáfora familiar:
 - Un nodo es padre si tiene una flecha dirigida hacia otro nodo.
 - Un nodo es hijo si tiene una flecha dirigida hacia él.
 - Un ancestro aparece antes en una cadena de nodos.
 - Un descendiente aparece después en una cadena de nodos.



Ejemplo de estructura

- Edad y Sexo no dependen de otras variables, pero influyen en Formación.
- Formación está condicionada en Ocupación y Ciudad.
- Ocupación y Ciudad influyen en Transporte.



Tablas de probabilidad condicional (CPT)

Conditional Probability Table – CPT

- Para cada nodo, se define una tabla de probabilidad condicional (CPT).
- Los parámetros de la tabla se ajustan con base en un conjunto de datos.
- Si un nodo no tiene padres, se define su probabilidad marginal.
- La CPT indica la probabilidad condicional del nodo dados sus padres.
- El tamaño de la CPT crece exponencialmente con el número de padres.

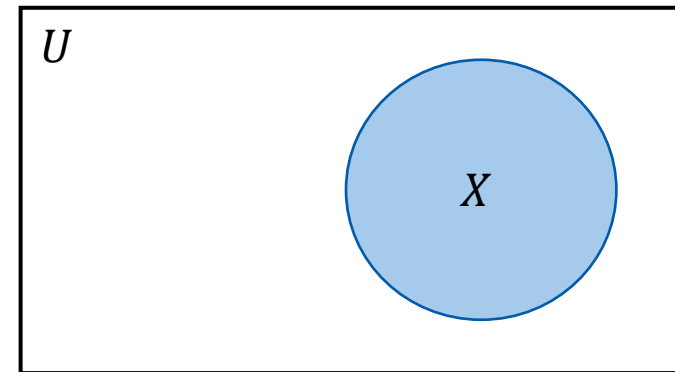


Repaso de probabilidad marginal

- La probabilidad mide la incertidumbre sobre la ocurrencia de un evento.
- Su valor está entre 0 (evento imposible) y 1 (evento seguro).
- La probabilidad marginal la probabilidad de un solo evento.

$$P(\text{evento}) = \frac{\text{ocurrencias del evento}}{\text{total de ocurrencias}}$$

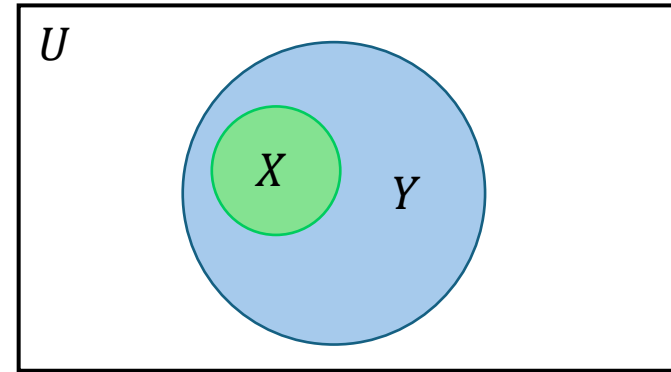
$$P(E = \text{joven}) = \frac{3 \text{ jóvenes}}{10 \text{ personas}} = 0.30$$



Repaso de probabilidad condicional

- Mide la probabilidad de que ocurra un evento dado que otro ha ocurrido.

$$P(X | Y) = \frac{P(X \cap Y)}{P(Y)}$$



- Si $P(Y) = 0$, la probabilidad condicional no está definida.

$$P(S=\text{masculino} | F=\text{superior}) = \frac{3 \text{ hombres con formación superior}}{5 \text{ personas con formación superior}} = 0.60$$



Ejemplo de un conjunto de datos

#	Edad	Sexo	Formación	Ciudad	Ocupación	Transporte
1	mayor	masculino	básico-medio	grande	Empleado	carro
2	adulto	masculino	básico-medio	grande	Propio	tren
3	mayor	masculino	superior	Pequeña	Empleado	otro
4	adulto	femenino	básico-medio	Grande	empleado	tren
5	joven	femenino	superior	Pequeña	empleado	otro
6	mayor	femenino	básico-medio	Grande	empleado	carro
7	adulto	femenino	básico-medio	Grande	empleado	carro
8	adulto	femenino	básico-medio	Pequeña	empleado	carro
9	mayor	masculino	básico-medio	Pequeña	empleado	tren
10	adulto	masculino	básico-medio	Pequeña	empleado	tren
11	joven	masculino	básico-medio	Grande	propio	carro
12	adulto	masculino	básico-medio	Grande	empleado	otro



Cálculo de la probabilidad marginal

Sexo (S)

femenino	masculino
$5/12 \approx 0.4167$	$7/12 \approx 0.5833$

Edad (E)

joven	adulto	mayor
$2/12 \approx 0.1667$	$6/12 = 0.5000$	$4/12 \approx 0.3333$



Cálculo de probabilidad condicionada en otra variable

Ocupación (O)

Formación	empleado	propio
básico-medio	$8/10 = 0.8000$	$2/10 = 0.2000$
superior	$2/2 = 1.0000$	$0/2 = 0.0000$

Ciudad (C)

Formación	pequeña	grande
básico-medio	$3/10 = 0.3000$	$7/10 = 0.7000$
superior	$2/2 = 1.0000$	$0/2 = 0.0000$



Cálculo de probabilidad condicionada en dos variables

Transporte (T)

Ciudad	Ocupación	carro	tren	otro
pequeña	empleado	$1/5 = 0.2000$	$2/5 = 0.4000$	$2/5 = 0.4000$
pequeña	propio	sin datos	sin datos	sin datos
grande	empleado	$3/5 = 0.6000$	$1/5 = 0.2000$	$1/5 = 0.2000$
grande	propio	$1/2 = 0.5000$	$1/2 = 0.5000$	$0/2 = 0.0000$



Cálculo de probabilidad condicionada en dos variables

Formación (F)

Edad	Sexo	básico-medio	superior
joven	femenino	$0/1 = 0.0000$	$1/1 = 1.0000$
joven	masculino	$1/1 = 1.0000$	$0/1 = 0.0000$
adulto	femenino	$3/3 = 1.0000$	$0/3 = 0.0000$
adulto	masculino	$3/3 = 1.0000$	$0/3 = 0.0000$
mayor	femenino	$1/1 = 1.0000$	$0/1 = 0.0000$
mayor	masculino	$2/3 \approx 0.6667$	$1/3 \approx 0.3333$



Clasificación con redes bayesianas

- Sabiendo que un ejemplo del conjunto tiene x como vector de atributos.
- ¿Cuál es la probabilidad de clasificarla en la clase y ?

$$P(y|x) = \frac{P(y \cap x)}{P(x)}$$

- Es difícil utilizar frecuencias para resolver esta probabilidad, muchos datos.
- Por ello se recurre al teorema de Bayes sobre probabilidad condicionada:

$$P(y \cap x) = P(x|y)P(y)$$



Teorema de Bayes

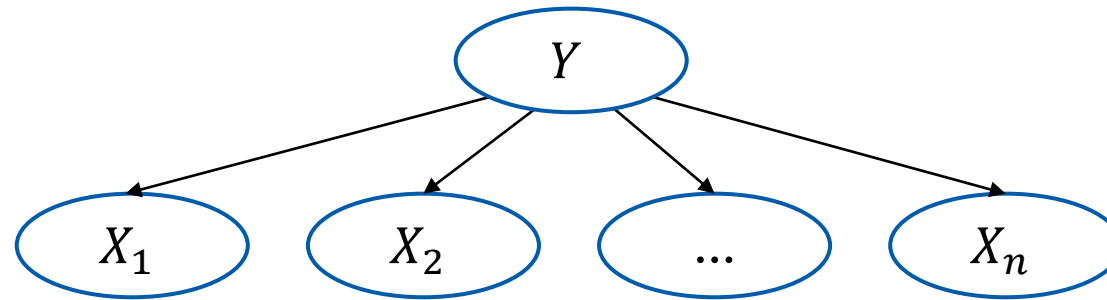
- Calcula la probabilidad de una hipótesis después de ver nueva evidencia.

$$P(y|x) = \frac{P(x|y)P(y)}{P(x)}$$

- $P(x|y)$ se denomina **verosimilitud**.
- $P(y|x)$ se llama **probabilidad a posteriori** de la clase.
- $P(y)$ se llama **probabilidad a priori** de la clase.
- $P(x)$ se llama **evidencia**.



Ingenuo bayesiano (*Naive Bayes*)



- *Naive Bayes* es un modelo de clasificación probabilístico y simple.
- Utiliza la probabilidad para predecir la clase a la que pertenece un ejemplo.
- Se llama ingenuo porque asume que un atributo no afecta a los demás.
- La parte bayesiana del nombre se refiere a su base en el teorema de Bayes.
- Si un ejemplo $\mathbf{x}$ está formado por n atributos, la verosimilitud es:

$$P(\mathbf{x}|\mathbf{y}) = \prod_{i=1}^n P(x_i|\mathbf{y}) = P(x_1|\mathbf{y}) \cdot P(x_2|\mathbf{y}) \cdots P(x_n|\mathbf{y})$$

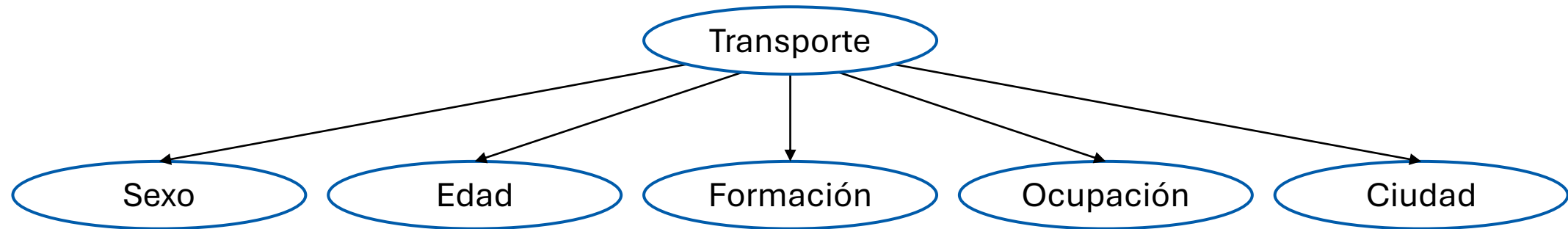


Ingenio bayesiano para la elección de transporte

Transporte	femenino	masculino
carro	$3/5 = 0.600$	$2/5 = 0.400$
tren	$1/4 = 0.250$	$3/4 = 0.750$
otro	$1/3 \approx 0.333$	$2/3 \approx 0.667$

Transporte	bás-med	superior
carro	$5/5 = 1.000$	$0/5 = 0.000$
tren	$4/4 = 1.000$	$0/4 = 0.000$
otro	$1/3 \approx 0.333$	$2/3 \approx 0.667$

Transporte	empleado	propio
carro	$4/5 = 0.800$	$1/5 = 0.200$
tren	$3/4 = 0.750$	$1/4 = 0.250$
otro	$3/3 = 1.000$	$0/3 = 0.000$



Transporte	joven	adulto	mayor
carro	$1/5 = 0.200$	$2/5 = 0.400$	$2/5 = 0.400$
tren	$0/4 = 0.000$	$3/4 = 0.750$	$1/4 = 0.250$
otro	$1/3 \approx 0.333$	$1/3 \approx 0.333$	$1/3 \approx 0.333$

Transporte	P(x)
carro	$5/12 \approx 0.42$
tren	$4/12 \approx 0.33$
otro	$3/12 \approx 0.25$

Transporte	pequeña	grande
carro	$1/5 = 0.200$	$4/5 = 0.800$
tren	$2/4 = 0.500$	$2/4 = 0.500$
otro	$2/3 \approx 0.667$	$1/3 \approx 0.333$



Características del Naive Bayes

Ventajas:

- Simple, rápido y efectivo.
- Da buenos resultados con datos con ruido.
- Requiere relativamente pocos ejemplos para entrenar.
- También funciona bien con grandes conjuntos de datos.
- Proporciona la probabilidad estimada para cada predicción.

Desventajas:

- Supone independencia entre atributos, lo cual normalmente es falso.



Clasificación usando un *Naive Bayes*

- Se toma aquella clase cuya probabilidad a posteriori es mayor.
- Es decir, aquella clase cuya $P(y|\mathbf{x})$ sea máxima:

$$clase(\mathbf{x}) = \arg \max_{y \in Y} P(y|\mathbf{x}) = \arg \max_{y \in Y} \frac{P(\mathbf{x}|y)P(y)}{P(\mathbf{x})}$$

- Como $P(\mathbf{x})$ no depende de y bastará con tomar el valor máximo de:

$$clase(\mathbf{x}) = \arg \max_{y \in Y} P(\mathbf{x}|y)P(y)$$



Detección de correo no deseado

- Se desea clasificar correos electrónicos como legítimos o no deseados.
- Se utilizan cuatro atributos discretos basados en el contenido del correo:
 1. Palabra "oferta": si aparece la palabra en el mensaje (sí / no).
 2. Palabra "ganaste": si aparece la palabra en el mensaje (sí / no).
 3. Longitud: corta (< 50 caracteres), media ($50 - 200$) o larga (> 200).
 4. Adjunto: si el correo contiene un archivo adjunto (sí / no).
- *Naive Bayes* asume que estos atributos son independientes dada la clase.

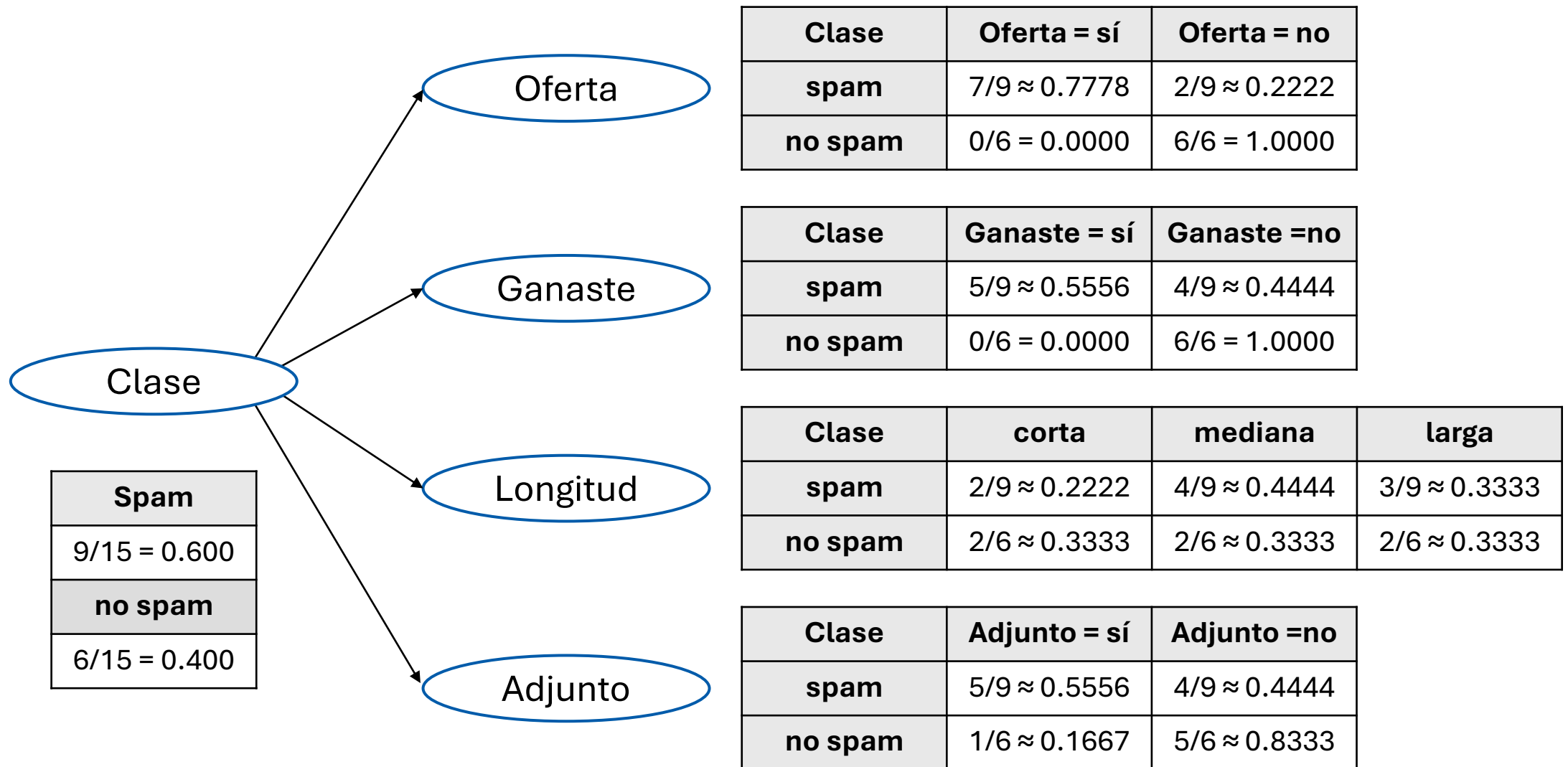


Datos para la detección de correo no deseado

#	Oferta	Ganaste	Longitud	Adjunto	Clase
1	sí	no	corta	no	spam
2	sí	sí	media	sí	spam
3	no	sí	larga	no	spam
4	sí	no	media	sí	spam
5	no	no	corta	no	no spam
6	no	no	media	no	no spam
7	sí	no	larga	sí	spam
8	no	no	larga	no	no spam
9	sí	sí	corta	sí	spam
10	no	sí	media	no	spam
11	no	no	corta	no	no spam
12	sí	no	media	no	spam
13	no	no	larga	sí	no spam
14	sí	sí	larga	sí	spam
15	no	no	media	no	no spam



Ingenuo Bayesiano para la detección de spam



Fórmula para calcular probabilidades *a posteriori*

- Se parte del teorema de Bayes:

$$P(\text{clase} \mid \text{evidencia}) = \frac{P(\text{evidencia} \mid \text{clase}) \cdot P(\text{clase})}{P(\text{evidencia})}$$

- La probabilidad de la evidencia se obtiene sumando sobre todas las clases:

$$P(\text{evidencia}) = \sum_{c \in \text{Clase}} P(\text{evidencia} \mid c) \cdot P(c)$$

- Sustituyendo, se obtiene:

$$P(\text{clase} \mid \text{evidencia}) = \frac{P(\text{evidencia} \mid \text{clase}) \cdot P(\text{clase})}{\sum_c P(\text{evidencia} \mid c) \cdot P(c)}$$



Clasificación de un ejemplo de prueba

Oferta	Ganaste	Longitud	Adjunto	Clase
no	no	corta	no	no spam

$P(\text{Clase} = \text{spam} \mid \text{evidencia})$		$P(\text{Clase} = \text{no spam} \mid \text{evidencia})$	
$P(\text{Oferta} = \text{no} \mid \text{spam})$	$2/9 \approx 0.22$	$P(\text{Oferta} = \text{no} \mid \text{no spam})$	$6/6 = 1.00$
$P(\text{Ganaste} = \text{no} \mid \text{spam})$	$4/9 \approx 0.44$	$P(\text{Ganaste} = \text{no} \mid \text{no spam})$	$6/6 = 1.00$
$P(\text{Longitud} = \text{corta} \mid \text{spam})$	$2/9 \approx 0.22$	$P(\text{Longitud} = \text{corta} \mid \text{no spam})$	$2/6 \approx 0.33$
$P(\text{Adjunto} = \text{no} \mid \text{spam})$	$4/9 \approx 0.44$	$P(\text{Adjunto} = \text{no} \mid \text{no spam})$	$5/6 \approx 0.83$
$P(\text{Clase} = \text{spam})$	$9/15 = 0.6$	$P(\text{Clase} = \text{no spam})$	$6/15 = 0.4$



Clasificación de un ejemplo de prueba II

- Se calcula el producto de la **verosimilitud** y la **priori** para cada clase.

$$P(\text{evidencia} \mid \text{spam}) \cdot P(\text{spam}) = \left(\frac{2}{9} \cdot \frac{4}{9} \cdot \frac{2}{9} \cdot \frac{4}{9} \right) \cdot \frac{9}{15} \approx 0.005853$$

$$P(\text{evidencia} \mid \text{no spam}) \cdot P(\text{no spam}) = \left(\frac{6}{6} \cdot \frac{6}{6} \cdot \frac{2}{6} \cdot \frac{5}{6} \right) \cdot \frac{6}{15} \approx 0.111111$$

- El producto más grande indica la clase más probable (aún sin normalizar).
- En este caso, **no spam** tiene mayor valor.



Clasificación de un ejemplo de prueba III

- Se divide cada producto entre la suma de ambos para obtener probabilidad.

$$P(\text{Clase} = \text{spam} \mid \text{evidencia}) = \frac{0.005853}{0.005853 + 0.111111} = \frac{0.005853}{0.116964} = 0.0500$$

$$P(\text{Clase} = \text{no spam} \mid \text{evidencia}) = \frac{0.111111}{0.005853 + 0.111111} = \frac{0.111111}{0.116964} = 0.9500$$

- La clase con mayor probabilidad a posteriori es **no spam**, un acierto.



Caso de estudio: predicciones de spam

- La evaluación de un modelo se realiza en un conjunto de prueba.
- La probabilidad de ser **spam** es la salida del clasificador.
- Fijando un umbral, generalmente **0.5**, se asigna la clase.
- El cálculo de las métricas de evaluación se realiza mediante la comparación.

Ejemplo	Probabilidad de spam	Clase predicha (umbral 0.5)	Clase verdadera
1	0.92	Spam	Spam
2	0.78	Spam	Spam
3	0.61	Spam	No spam
4	0.55	Spam	No spam
5	0.42	No spam	Spam
6	0.38	No spam	No spam
7	0.21	No spam	No spam
8	0.15	No spam	No spam



Matriz de confusión

- La matriz de confusión resume los aciertos y errores de un clasificador.
- Compara las clases reales con las predichas.

		Clase predicha	
		<i>positivo</i>	<i>negativo</i>
Clase real	<i>positivo</i>	VP Verdaderos Positivos	FN Falsos Negativos
	<i>negativo</i>	FP Falsos Positivos	VN Verdaderos Negativos

		Clase predicha	
		<i>spam</i>	<i>no spam</i>
Clase real	<i>spam</i>	VP = 2	FN = 1
	<i>no spam</i>	FP = 2	VN = 3

- A partir de la matriz de confusión se calculan varias métricas de evaluación.
- Una métrica cuantifica el rendimiento del clasificador.



Exactitud (*accuracy*)

- Proporción de aciertos totales entre todas las predicciones.

$$\text{Exactitud} = \frac{VP + VN}{VP + FN + FP + VN}$$

$$\frac{2 + 3}{2 + 1 + 2 + 3} = \frac{5}{8} = 0.625 \text{ (62.5\%)}$$

- Interpretación en spam:
 - De cada 100 correos, el clasificador acierta aproximadamente 63.



Precisión (*precision*)

- Porcentaje de predicciones correctas de todas las predicciones positivas.

$$\text{Precisión} = \frac{VP}{VP + FP}$$

$$\frac{2}{2 + 2} = 0.5 \text{ (50\%)}$$

- Interpretación en spam:
 - Cuando se etiqueta un correo como spam, la mitad de las veces acierta.
 - La otra mitad son mensajes legítimos marcados por error (FP).



Sensibilidad (*recall*)

- **Tasa de Verdaderos Positivos**
- Porcentaje de positivos correctamente detectados.

$$\text{Sensibilidad} = \frac{VP}{VP + FN}$$

$$\frac{2}{2 + 1} \approx 0.6667 \text{ (66.67\%)}$$

- Interpretación en spam:
 - El modelo detecta 2 de cada 3 correos spam reales.
 - Un 33% de los spam se cuelan en la bandeja de entrada (FN).



Especificidad (specificity)

- **Tasa de Verdaderos Negativos**
- Porcentaje de ejemplos negativos detectados correctamente.

$$\text{Especificidad} = \frac{VN}{VN + FP}$$

$$\frac{3}{3 + 2} = 0.6 \text{ (60\%)}$$

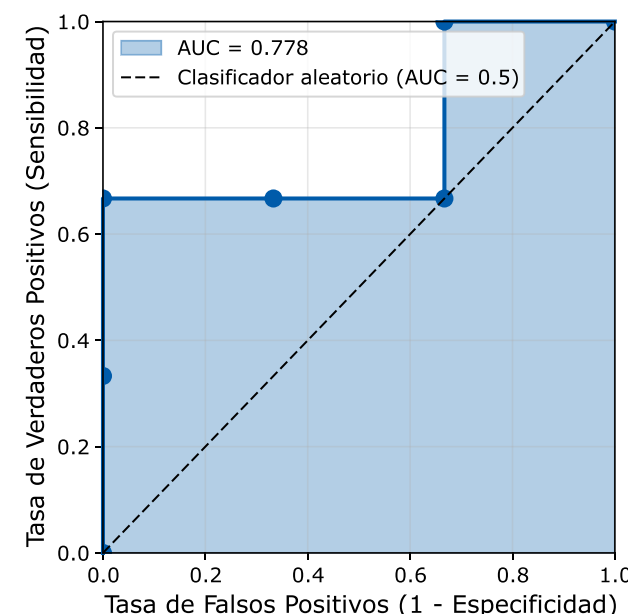
- Interpretación en spam:
 - El modelo identifica correctamente 3 de cada 5 correos legítimos.
 - El 40% restante (FP) son etiquetados como spam.



Área bajo la curva ROC (AUC-ROC)

- La curva ROC muestra la capacidad del modelo para distinguir entre clases.
- El AUC (*Area Under the Curve*) resume esta calidad en un solo número.
- La curva ROC y el AUC evalúan el rendimiento sin un umbral fijo.
- Un clasificador perfecto tiene $AUC = 1$; uno aleatorio tiene $AUC = 0.5$.

Umbral	Sensibilidad	1 – Especificidad
1.00	0.000	0.000
0.95	0.333	0.000
0.82	0.667	0.000
0.67	0.667	0.333
0.45	0.667	0.667
0.31	1.000	0.667
0.00	1.000	1.000



Clasificación binaria y multiclase

- **Clasificación binaria:** Solo dos clases.
 - Las métricas vistas se definen directamente.
- **Clasificación multiclase:** Más de dos clases.
 - Las métricas se extienden usando enfoques como uno contra todos.
 - La curva ROC se puede trazar para cada clase.
 - La matriz de confusión es de tamaño $c \times c$.
- Las métricas cambian en su interpretación y cálculo.



Ejercicio en clase

- Determina la clase predicha usando un umbral de 0.5.
- Construye la matriz de confusión y calcula las métricas de desempeño.
- Interpreta cada métrica en el contexto de detección de spam.

Ejemplo	Probabilidad de spam	Clase verdadera
1	0.91	Spam
2	0.76	Spam
3	0.62	No spam
4	0.58	Spam
5	0.44	No spam
6	0.39	Spam
7	0.57	No spam
8	0.23	No spam



Definición del modelo

- `sklearn.naive_bayes.CategoricalNB` es la versión para datos categóricos.
- Requiere los atributos codificados como valores enteros no negativos.
- El parámetro `alpha` evita probabilidades cero. Valor por defecto: 1.0.

```
from sklearn.naive_bayes import CategoricalNB  
  
# Crear modelo con suavizado Laplace  
modelo_nb = CategoricalNB(alpha=1.0)
```



Conjunto de datos categóricos

```
data = {
    'Oferta': ['si', 'si', 'no', 'si', 'no', 'no', 'si', 'no', 'si', 'no',
               'no', 'si', 'no', 'si', 'no'],
    'Ganaste': ['no', 'si', 'si', 'no', 'no', 'no', 'si', 'no', 'si', 'si',
                'no', 'no', 'no', 'si', 'no'],
    'Longitud': ['corta', 'media', 'larga', 'media', 'corta', 'media',
                 'larga', 'larga', 'corta', 'media', 'corta', 'media',
                 'larga', 'larga', 'media'],
    'Adjunto': ['no', 'si', 'no', 'si', 'no', 'no', 'si', 'no', 'si', 'no',
                'no', 'no', 'no', 'si', 'no'],
    'Clase': ['spam', 'spam', 'spam', 'spam', 'no_spam', 'no_spam', 'spam',
              'no_spam', 'spam', 'spam', 'no_spam', 'spam', 'no_spam',
              'spam', 'no_spam']
}

df = pd.DataFrame(data)
```



Codificación de variables categóricas

- `LabelEncoder` transforma categorías en números naturales.
- La codificación es arbitraria pero consistente; no implica orden real.

```
# Codificar cada columna categórica a enteros
encoders = {}
for col in df.columns:
    le = LabelEncoder()
    df[col+'_num'] = le.fit_transform(df[col])
    encoders[col] = le

# Variables predictoras y objetivo
X = df[['Oferta_num', 'Ganaste_num', 'Longitud_num',
        'Adjunto_num']].values
y = df['Clase_num'].values
```



Ajuste del modelo con partición *holdout*

- `train_test_split()` separa datos de forma aleatoria pero reproducible.
- `stratify=y` mantiene la misma proporción de clases en ambos conjuntos.
- `fit()` calcula las probabilidades a partir de los datos.

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import CategoricalNB

# 80% entrenamiento, 20% prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Crear y entrenar el modelo con suavizado Laplace
nb = CategoricalNB(alpha=1.0)
nb.fit(X_train, y_train)
```



Evaluación con conjunto de prueba

- `predict()` aplica el modelo a los correos que no usó durante el aprendizaje.
- Esto simula el comportamiento del clasificador con mensajes nuevos.
- `accuracy_score()` compara las predicciones con las etiquetas reales.

```
from sklearn.metrics import accuracy_score

# Predecir sobre el conjunto de prueba
y_pred = modelo_nb.predict(X_test)

# Exactitud
acc = accuracy_score(y_test, y_pred)
print(f"Exactitud: {acc:.2f}")
```



Evaluación con conjunto de prueba

- `confusion_matrix()` genera la matriz de confusión.
- Las filas representan las clases reales y las columnas, las clases predichas.
- `classification_report()` calcula varias métricas en una sola llamada.

```
from sklearn.metrics import confusion_matrix, classification_report

# Matriz de confusión
cm = confusion_matrix(y_test, y_pred)
print(cm)

# Reporte con precisión, recall, etc
print(classification_report(y_test, y_pred,
                           target_names=['no_spam', 'spam']))
```



Evaluación con validación cruzada

- Con pocos datos, una única partición puede dar resultados muy variables.
- La validación cruzada divide los datos en n grupos (*folds*).
- El promedio de los resultados es mucho más estable y realista.

```
from sklearn.model_selection import cross_val_score

# Exactitud en 5 folds
scores_acc = cross_val_score(nb, X, y, cv=5, scoring='accuracy')
print("Exactitud por fold:", scores_acc)
print(f"Exactitud promedio: {scores_acc.mean():.4f}")

# Otras métricas
scores_acc = cross_val_score(nb, X, y, cv=5, scoring='precision')
scores_rec = cross_val_score(nb, X, y, cv=5, scoring='recall')
```



Matriz de confusión global con CV

- `cross_val_predict()` devuelve una predicción para cada ejemplo.
- Cada predicción se genera con un modelo entrenado sin usar ese ejemplo.
- Se obtiene una única matriz de confusión global que resume el rendimiento.

```
from sklearn.model_selection import cross_val_predict

# Predicciones de todos los ejemplos usando validación cruzada
y_pred = cross_val_predict(modelo_nb, X, y, cv=5)

print("Matriz de confusión (validación cruzada):")
print(confusion_matrix(y, y_pred))

print("Reporte de clasificación (CV):")
print(classification_report(y, y_pred,
                             target_names=['no_spam', 'spam']))
```



Preparación de un nuevo ejemplo

- Los nuevos datos deben transformarse con los mismos `LabelEncoder`.
- Se usa el método `transform` para aplicar la misma codificación.

```
nuevo_correo = np.array([[
    encoders['Oferta'].transform(['no'])[0],
    encoders['Ganaste'].transform(['no'])[0],
    encoders['Longitud'].transform(['corta'])[0],
    encoders['Adjunto'].transform(['no'])[0]
]])
```



Predicción de un nuevo ejemplo

- `predict_proba()` devuelve la probabilidad de pertenecer a cada clase.
- La clase con mayor probabilidad es la que asigna `predict()`.

```
# Predecir clase
clase_predicha = nb.predict(nuevo_correo)
clase_nombre = encoders['Clase'].inverse_transform(clase_predicha)[0]

# Obtener probabilidades
probabilidades = nb.predict_proba(nuevo_correo)[0]
prob_no_spam = probabilidades[0]
prob_spam = probabilidades[1]

print(f"Clase predicha: {clase_nombre}")
print(f"Probabilidad de no_spam: {prob_no_spam:.4f}")
print(f"Probabilidad de spam: {prob_spam:.4f}")
```



Tarea 6: Ajuste de un ingenuo bayesiano

- Selecciona un conjunto de datos para clasificación de un repositorio.
- Realiza un análisis exploratorio:
 - Primeras filas, dimensiones, tipos de datos y distribución de la clase.
- Selecciona y codifica al menos cinco atributos categóricos y una clase.
- Divide los datos en entrenamiento (70%) y prueba (30%).
- Ajusta un ingenuo bayesiano con el entrenamiento.
- Evalúa el modelo con:
 - matriz de confusión, exactitud, precisión y sensibilidad.
- Aplica validación cruzada de 5 *folds* para obtener las mismas métricas.
- Compara los resultados de la partición *holdout* y la validación cruzada.



Introducción a los árboles de decisión

- Un árbol de decisión es un modelo de aprendizaje supervisado.
- Toma una observación (ejemplo) descrita a través de atributos.
- Los atributos de entrada pueden ser discretos o continuos.
- Devuelve una decisión (valor previsto de la salida) dada la entrada.
- El valor de la salida puede ser a su vez discreto o continuo.
 - Aprender una función de valores discretos se denomina **clasificación**.
 - Aprender una función continua se denomina **regresión**.



Estructura de un árbol de decisión

- Un árbol describe una **secuencia de condiciones** para llegar a una decisión.
- Cada **nodo interno** es una condición sobre el valor de uno de los atributos.
- Las **ramas** están etiquetadas con los posibles valores de dicho atributo.
- Cada **nodo hoja** representa el valor que ha de ser devuelto si es alcanzado.
- La representación es muy natural para los humanos.



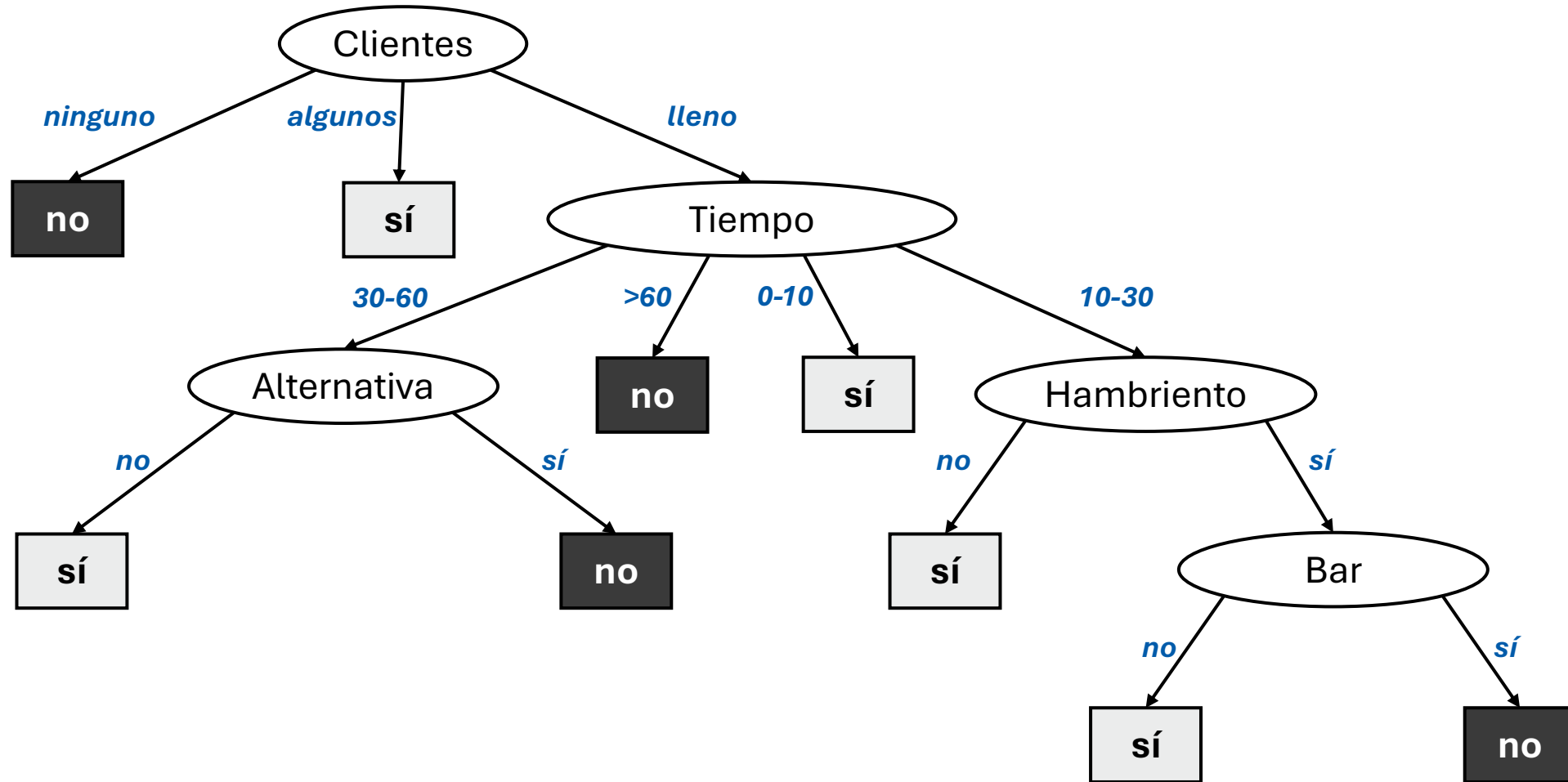
Tarea de clasificación: esperar en restaurante

- El problema es decidir si esperar por una mesa en un restaurante.
- Existen cinco atributos para tomar esta decisión.

#	Atributo	Definición	Valores
1	Alternativa	Hay otro restaurante adecuado cerca.	sí, no
2	Bar	Hay un área bar donde esperar.	sí, no
3	Hambriento	¿Estamos hambrientos?	sí, no
4	Clientes	Clientes en el restaurante.	ninguno, algunos, lleno
5	Tiempo	Minutos estimados de espera.	0-10, 10-30, 30-60, >60
	Esperar (clase)	Se espera por una mesa.	sí, no



Ejemplo de un árbol de decisión



Conjunto de datos: espera en restaurante

ID	Alternativa	Bar	Hambriento	Clientes	Tiempo	Esperar
X ₁	sí	no	sí	algunos	0-10	sí
X ₂	sí	no	sí	lleno	30-60	no
X ₃	no	sí	no	algunos	0-10	sí
X ₄	sí	no	no	lleno	10-30	sí
X ₅	sí	no	sí	lleno	>60	no
X ₆	no	sí	sí	algunos	0-10	sí
X ₇	no	sí	no	ninguno	0-10	no
X ₈	no	no	sí	algunos	0-10	sí
X ₉	no	sí	sí	lleno	>60	no
X ₁₀	sí	sí	sí	lleno	10-30	no
X ₁₁	no	no	no	ninguno	0-10	no
X ₁₂	sí	sí	no	lleno	30-60	sí



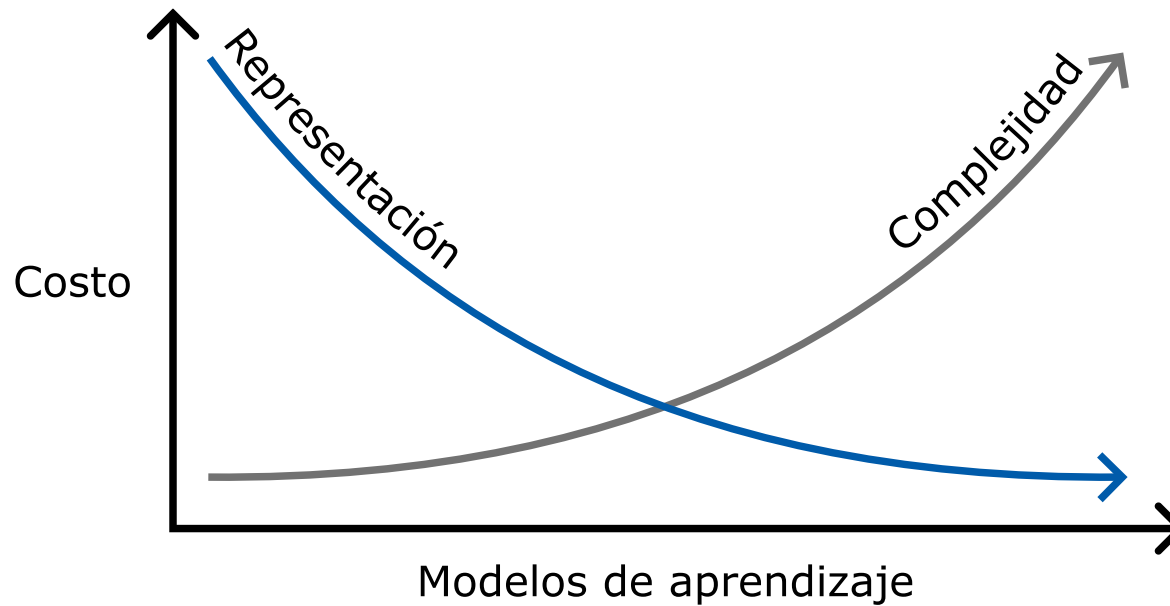
Inducción de un árbol

- Construir un árbol consistente con el entrenamiento es un problema.
- A partir de un conjunto de entrenamiento se pueden construir varios árboles.
- Encontrar el mejor árbol de decisión se vuelve un problema de búsqueda.
- La solución fácil es crear un árbol que tenga una rama para cada ejemplo.
- El problema con este árbol es que memoriza exactamente los ejemplos.
- La navaja de Ockham implica encontrar el árbol de decisión más pequeño que sea consistente con los ejemplos.



Navaja de Ockham

- En igualdad de condiciones, la solución más sencilla suele ser correcta.
- Se traduce en la búsqueda de modelos parsimoniosos, aquellos que:
 - Capturan de forma eficaz las relaciones en los datos.
 - Evitan la complejidad innecesaria.



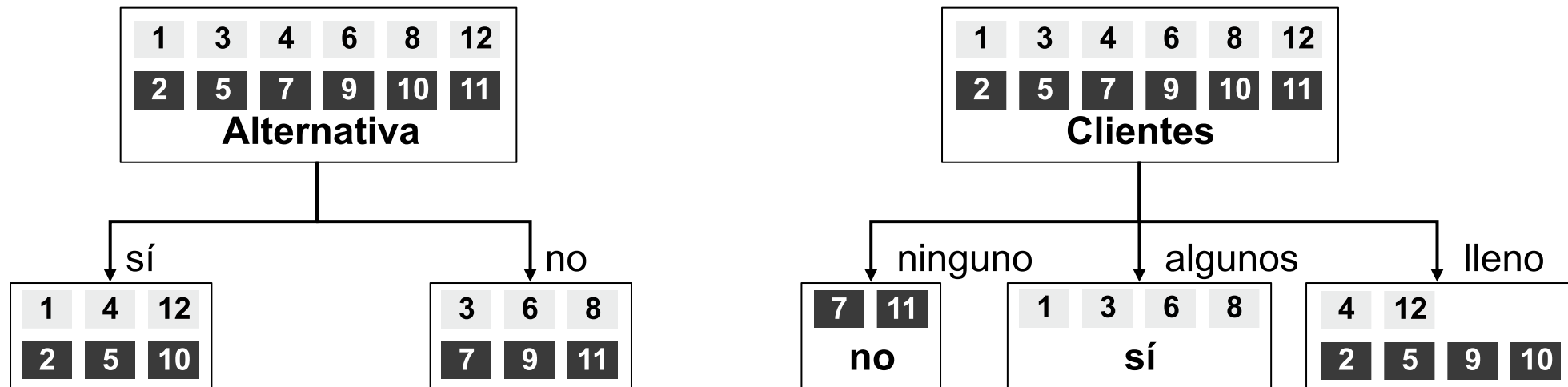
Algoritmo de inducción de árboles de decisión

- Encontrar el árbol más pequeño sigue siendo un problema.
- Sin embargo, con **heurísticas simples**, se puede hacer un buen trabajo.
- La idea es **realizar primero la condición sobre el atributo más importante**.
- El atributo más importante es **aquel que discrimina mejor los ejemplos**.
- Se espera la **clasificación correcta con el menor número de condiciones**.
- ¿Cómo seleccionar los mejores atributos?



Selección de atributos

- La selección de atributos optimiza la separación de los ejemplos por clase.
- Por ejemplo:
 - **Alternativa** genera dos subconjuntos de ejemplos con clases mixtas.
 - **Cientes** es un atributo importante, porque si el valor es **ninguno** o **algunos** se obtienen subconjuntos de ejemplos con clases separadas.



Separación de ejemplos

- Después de seleccionar un atributo, pueden existir cuatro casos:
 1. Si quedan ejemplos positivos y negativos, hay que elegir el siguiente mejor atributo para separarlos. Llamada **recursiva**.
 2. Si todos los ejemplos restantes tienen la misma clase, se ha **finalizado**, se puede definir una clase: sí o no.
 3. Si no quedan ejemplos, significa que ningún ejemplo ha sido observado y se devuelve un valor **por defecto**.
 4. Si no quedan atributos, pero sí ejemplos positivos y negativos existe un problema de **ruído**. Los ejemplos que quedan tienen exactamente la misma descripción, pero clases diferentes.



Algoritmo ID3

Entradas: **ejemplos** – conjunto de ejemplos, **atributos** – conjunto de atributos

Salidas: **árbol** – modelo final

```
01 Si ejemplos está vacío devolver hoja vacía.  
02 Si ejemplos tienen la misma clase devolver una hoja con la clase.  
03 Si atributos está vacío devolver una hoja con la clase mayoritaria.  
04 Si no:  
05     Elegir el mejor atributo del conjunto de atributos.  
06     Crear un árbol con el mejor atributo como raíz.  
07     Para cada valor del mejor atributo:  
08         Dividir el conjunto de ejemplos.  
09         Crear un subárbol con la llamada recursiva sin el mejor atributo.  
10         Añadir una rama con el valor del atributo y el subárbol.  
11 Devolver árbol.
```



Cantidad de información

- Se necesita una medida que indique si un atributo es **adecuado**.
- Debe ser máxima cuando el atributo separa perfectamente las clases.
- Debe ser mínima cuando el atributo no aporta información útil.
- La **teoría de la información** de Shannon (1949) proporciona esa medida.
- La información de una respuesta depende de cuánto se sabía antes.
 - Si ya se sabía la respuesta, la información es cero.
 - Si la respuesta es una sorpresa, contiene mucha información.



Fórmula de la cantidad de información

- Un bit es la información necesaria para responder una pregunta de sí/no.
- Si un evento tiene probabilidad q , su información es $-\log_2(q)$ bits.
- Para un atributo con varios valores, la información es la suma ponderada:

$$I(p(x_i = v_1), \dots, p(x_i = v_n)) = \sum_{j=1}^n -p(x_i = v_j) \cdot \log_2[p(x_i = v_j)]$$

- Esta es la entropía de Shannon y mide el grado de desorden o incertidumbre.



Información para una variable binaria

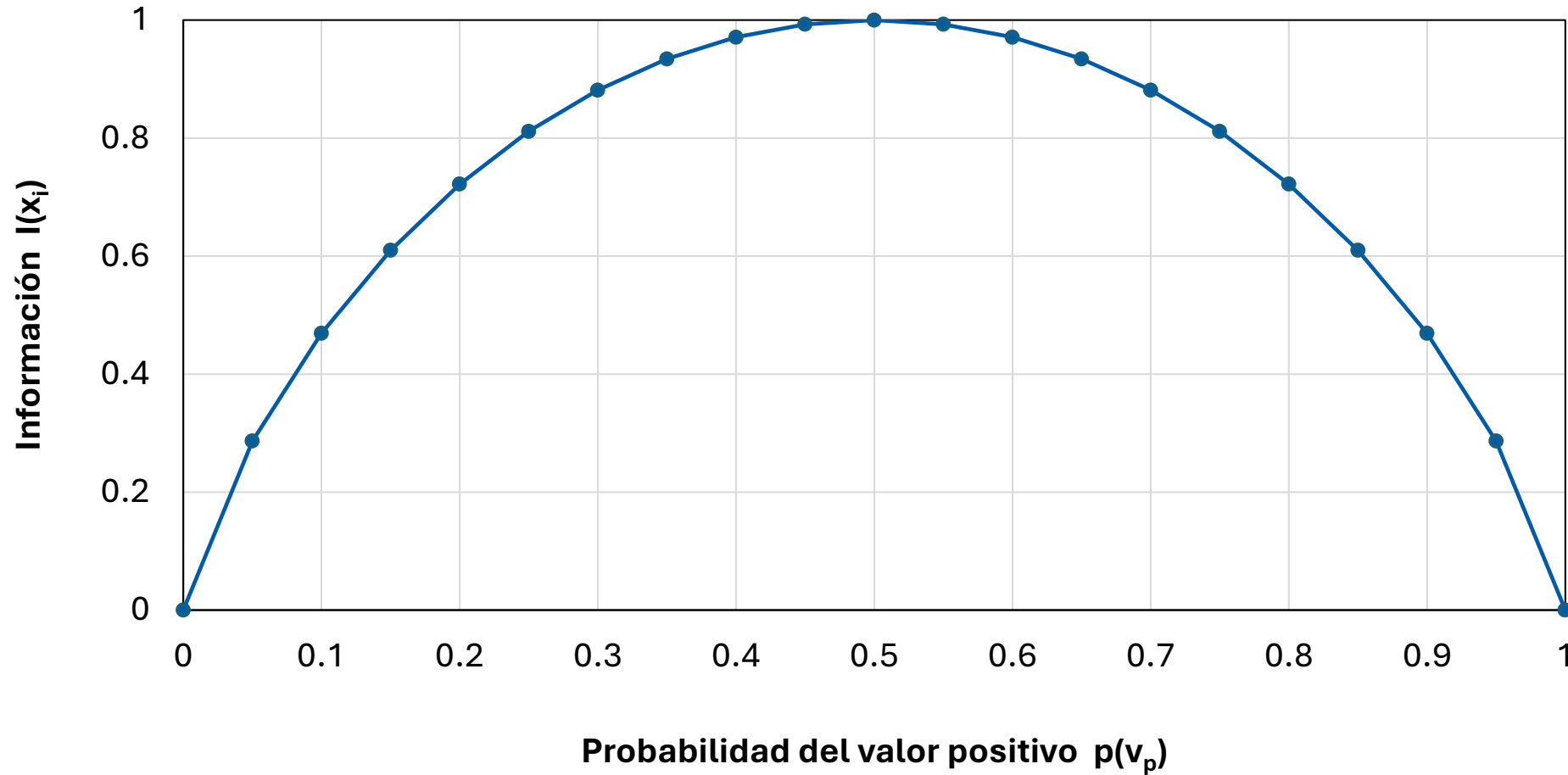
- Si una variable dentro de un conjunto tiene p ejemplos con etiqueta positiva y n ejemplos con etiqueta negativa, su información (entropía) es:

$$I(\text{Variable}) = -\frac{p}{p+n} \cdot \log_2 \left(\frac{p}{p+n} \right) - \frac{n}{p+n} \cdot \log_2 \left(\frac{n}{p+n} \right)$$

- Si todos los ejemplos son de una etiqueta, no hay incertidumbre ($I_{x_i} = 0$).
- Con un número igual de cada etiqueta, hay máxima incertidumbre ($I_{x_i} = 1$).



Comportamiento de la información



Ejemplo: Información de la clase

ID	X_1	X_2	X_3	X_4	X_5	X_6	X_7	X_8	X_9	X_{10}	X_{11}	X_{12}
Esperar	sí	no	sí	sí	no	sí	no	sí	no	no	no	sí

$$I(Clase) = -\frac{p}{p+n} \cdot \log_2 \left(\frac{p}{p+n} \right) - \frac{n}{p+n} \cdot \log_2 \left(\frac{n}{p+n} \right)$$

$$I(Esperar) = -\frac{6}{6+6} \cdot \log_2 \left(\frac{6}{6+6} \right) - \frac{6}{6+6} \cdot \log_2 \left(\frac{6}{6+6} \right)$$

$$I(Esperar) = -0.5 \cdot \log_2(0.5) - 0.5 \cdot \log_2(0.5) = 1$$



Información después de una partición

- Cualquier atributo x_i puede dividir el conjunto de entrenamiento E .
- Al dividir E se crean subconjuntos $E_1, \dots, E_j$ de acuerdo con j valores de x_i .
- Cada subconjunto E_j tiene p_j ejemplos positivos y n_j ejemplos negativos.
- La información necesaria de los ejemplos en una partición E_j es:

$$I(E_j) = I\left(\frac{p_j}{p_j + n_j}, \frac{n_j}{p_j + n_j}\right)$$

- La información promedio (**entropía condicional**) después de la partición es:

$$I(Clase|Atributo) = \sum_j \frac{p_j + n_j}{p + n} \cdot I\left(\frac{p_j}{p_j + n_j}, \frac{n_j}{p_j + n_j}\right)$$



Ejemplo: Información de la partición de clientes

ID	X ₇	X ₁₁	X ₁	X ₃	X ₆	X ₈	X ₂	X ₄	X ₅	X ₉	X ₁₀	X ₁₂
Clientes	ninguno	ninguno	algunos	algunos	algunos	algunos	lleno	lleno	lleno	lleno	lleno	lleno
Esperar	no	no	sí	sí	sí	sí	no	sí	no	no	no	sí

ninguno

algunos

lleno

$$I(Esperar|Clientes) = \frac{0+2}{6+6} \cdot I\left(\frac{0}{0+2}, \frac{2}{0+2}\right) + \frac{0+4}{6+6} \cdot I\left(\frac{0}{0+4}, \frac{4}{0+4}\right) + \frac{4+2}{6+6} \cdot I\left(\frac{4}{4+2}, \frac{2}{4+2}\right)$$

$$I(Esperar|Clientes) = \frac{2}{12} \cdot 0 + \frac{4}{12} \cdot 0 + \frac{6}{12} \cdot 0.9182$$

$$I(Esperar|Clientes) = 0.5 \cdot 0.9182 = 0.4591$$



Cálculo de la información de las particiones

$$I(\text{Esperar}|\text{Alternativa}) = \frac{6}{12} \cdot I\left(\frac{3}{6}, \frac{3}{6}\right) + \frac{6}{12} \cdot I\left(\frac{3}{6}, \frac{3}{6}\right) = 1.0$$

$$I(\text{Esperar} | \text{Bar}) = \frac{6}{12} \cdot I\left(\frac{3}{6}, \frac{3}{6}\right) + \frac{6}{12} \cdot I\left(\frac{3}{6}, \frac{3}{6}\right) = 1.0$$

$$I(\text{Esperar} | \text{Ambriento}) = \frac{7}{12} \cdot I\left(\frac{3}{7}, \frac{4}{7}\right) + \frac{5}{12} \cdot I\left(\frac{3}{5}, \frac{2}{5}\right) = 0.9791$$

$$I(\text{Esperar} | \text{Tiempo}) = \frac{6}{12} \cdot I\left(\frac{4}{6}, \frac{2}{6}\right) + \frac{2}{12} \cdot I\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{2}{12} \cdot I\left(\frac{1}{2}, \frac{1}{2}\right) + \frac{2}{12} \cdot I\left(\frac{0}{2}, \frac{2}{2}\right) = 0.7924$$



Ganancia de información

- Mide la reducción de la incertidumbre al usar un atributo.
- Puede ser usada como heurística para elegir el atributo más importante.
- Se calcula como:

$$Ganancia(Clase|Atributo) = I(Clase) - I(Clase|Atributo)$$

- Si el atributo separa perfectamente las clases:
 - la información de la partición es cero y la ganancia es la máxima posible.
- Si el atributo no aporta nada, es decir, realiza una partición aleatoria:
 - ambos términos son iguales y la ganancia tiende a cero.



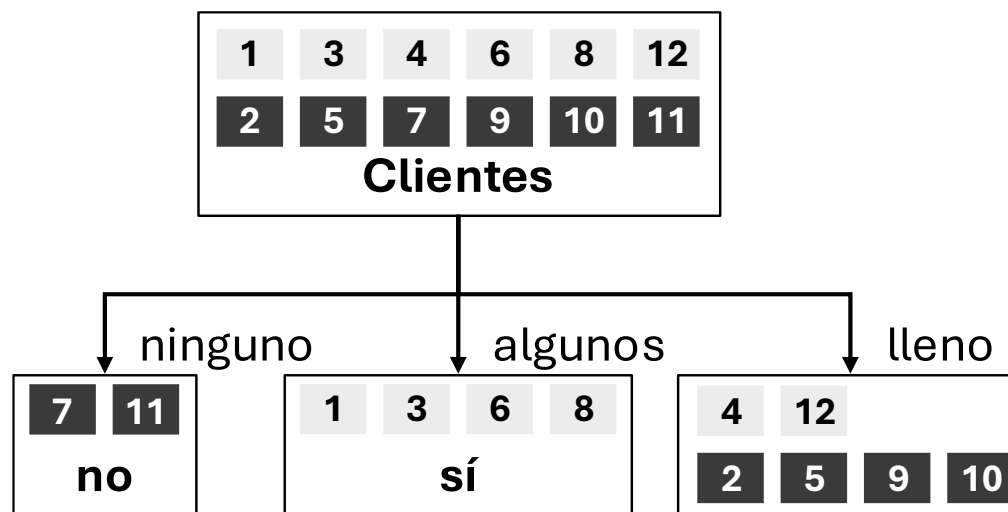
Ejercicio: Cálculo de la ganancia de la información

Atributo	Información de la clase $I(\text{Esperar})$	Información de la partición $I(\text{Esperar} \mid \text{Atributo})$	Ganancia de información $G(\text{Esperar} \mid \text{Atributo})$
Alternativa	1	1	0.0
Bar	1	1	0.0
Hambriento	1	0.9791	0.0209
Clientes	1	0.4591	0.5409
Tiempo-Estimado	1	0.7924	0.2076



Selección de atributos

- Se selecciona el atributo con mayor ganancia de información.
- Se divide el conjunto según los valores del atributo.
- Para cada subconjunto (rama), se repite el proceso recursivamente.
- Un atributo no puede volver a usarse en la misma rama.
- El proceso termina hasta que se separen los ejemplos o no haya atributos.



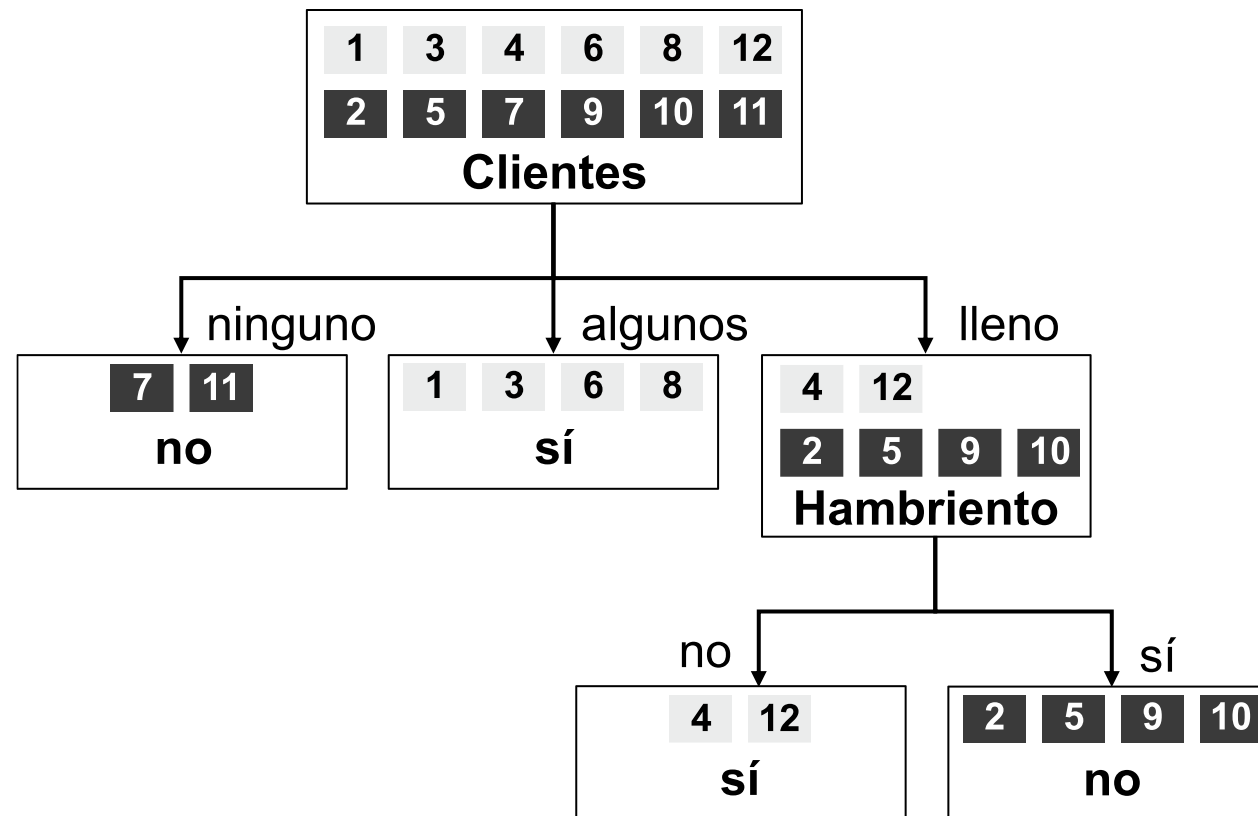
Particiones recursivas de nodos

- Se necesita un atributo que separe los ejemplos de la rama **Cliente=lleno**.
- En la llamada recursiva se omite el atributo **Cliente** porque ya fue usado.

Atributo	Información de la clase $I(\text{Esperar})$	Información de la partición $I(\text{Esperar} \mid \text{Atributo})$	Ganancia de información $G(\text{Esperar} \mid \text{Atributo})$
Alternativa	0.9182	0.8090	0.1092
Bar	0.9182	0.9182	0
Hambriento	0.9182	0	0.9182
Tiempo-Estimado	0.9182	0.6666	0.2516



Árbol de decisión para esperar en un restaurante



Ejercicio en clase: Inducción de un árbol

- Induce un árbol de decisión a partir del siguiente conjunto de entrenamiento.

ID	COLOR	ANILLOS	SATELITES	SUPERFICIE
Mercurio	gris	falso	verdadero	rocosa
Venus	café	falso	falso	rocosa
Tierra	azul	falso	verdadero	rocosa
Marte	rojo	falso	falso	rocosa
Júpiter	café	verdadero	verdadero	gaseosa
Saturno	café	verdadero	verdadero	gaseosa
Urano	azul	verdadero	verdadero	helada
Neptuno	azul	verdadero	verdadero	helada



Atributos continuos en árboles de decisión

- Los árboles de decisión pueden trabajar con atributos continuos.
- Para atributos continuos, se buscan puntos de corte que dividan los datos.
- Los puntos de corte son los puntos medios entre valores consecutivos.
- Cada punto medio genera una partición binaria ($\leq \text{valor}$ o $> \text{valor}$).
- Se evalúa la ganancia de información para cada posible corte.
- Se selecciona el corte que maximice la ganancia.



Versión continua de Tiempo

ID	Tiempo	Esperar
1	10	Sí
2	65	Sí
3	50	No
4	35	No
5	20	Sí

- Tiempo ordenado: 10, 20, 35, 50, 65
- Punto medio entre 10 y 20 = $(10+20)/2 = 15.0$
- Punto medio entre 20 y 35 = $(20+35)/2 = 27.5$
- Punto medio entre 35 y 50 = $(35+50)/2 = 42.5$
- Punto medio entre 50 y 65 = $(50+65)/2 = 57.5$



Cálculo de la información

$$I(Esperar) = I\left(\frac{3}{5}, \frac{2}{5}\right) = 0.9709$$

$$I(Esperar|Tiempo \leq 15) = \frac{1}{5} \cdot I\left(\frac{1}{1}, \frac{0}{1}\right) + \frac{4}{5} \cdot I\left(\frac{2}{4}, \frac{2}{4}\right) = 0.8$$

$$I(Esperar|Tiempo \leq 27.5) = \frac{2}{5} \cdot I\left(\frac{2}{2}, \frac{0}{2}\right) + \frac{3}{5} \cdot I\left(\frac{1}{3}, \frac{2}{3}\right) = 0.5509$$

$$I(Esperar|Tiempo \leq 42.5) = \frac{3}{5} \cdot I\left(\frac{2}{3}, \frac{1}{2}\right) + \frac{2}{5} \cdot I\left(\frac{1}{2}, \frac{1}{2}\right) = 0.9509$$

$$I(Esperar|Tiempo \leq 57.5) = \frac{4}{5} \cdot I\left(\frac{2}{4}, \frac{2}{4}\right) + \frac{1}{5} \cdot I\left(\frac{1}{1}, \frac{0}{1}\right) = 0.8$$



Cálculo de la ganancia de la información

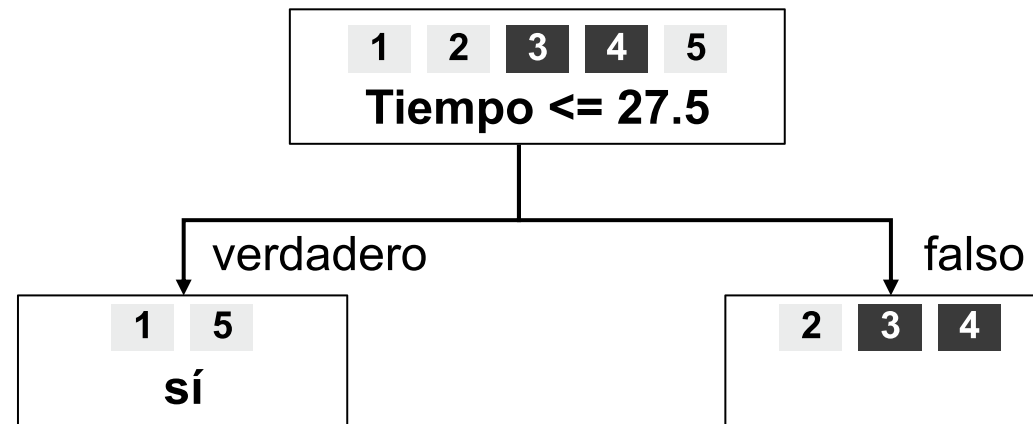
$$G(Esperar|Tiempo \leq 15) = 0.9709 - 0.8 = 0.1709$$

$$G(Esperar|Tiempo \leq 27.5) = 0.9709 - 0.5509 = 0.4200$$

$$G(Esperar|Tiempo \leq 42.5) = 0.9709 - 0.9509 = 0.0200$$

$$G(Esperar|Tiempo \leq 57.5) = 0.9709 - 0.8 = 0.1709$$

- La partición óptima es: $Tiempo \leq 27.5$ y $Tiempo > 27.5$.



Conjunto de datos categóricos

```
data = {
    'Alternativa': ['si', 'si', 'no', 'si', 'si', 'no', 'no', 'no', 'no',
                    'si', 'no', 'si'],
    'Bar':          ['no', 'no', 'si', 'no', 'no', 'si', 'si', 'no', 'si', 'si',
                    'no', 'si'],
    'Hambriento':  ['si', 'si', 'no', 'no', 'si', 'si', 'no', 'si', 'si', 'si',
                    'no', 'no'],
    'Clientes':     ['algunos', 'lleno', 'algunos', 'lleno', 'lleno',
                    'algunos', 'ninguno', 'algunos', 'lleno', 'lleno',
                    'ninguno', 'lleno'],
    'Tiempo':       ['0-10', '30-60', '0-10', '10-30', '>60', '0-10', '0-10',
                    '0-10', '>60', '10-30', '0-10', '30-60'],
    'Esperar':      ['si', 'no', 'si', 'si', 'no', 'si', 'no', 'si', 'no', 'no',
                    'no', 'si']
}
df = pd.DataFrame(data)
```



Codificación de variables categóricas

- Cada columna original genera una nueva columna numérica.
- Los valores se convierten, por ejemplo: no→0, sí→1.
- Se conservan los codificadores para transformar nuevos ejemplos.

```
# Codificar todas las columnas
encoders = {}
for col in df.columns:
    le = LabelEncoder()
    df[col+'_num'] = le.fit_transform(df[col])
    encoders[col] = le

X = df[['Alternativa_num', 'Bar_num', 'Hambriento_num', 'Clientes_num',
        'Tiempo_num']].values
y = df['Esperar_num'].values
```



Ajuste del modelo con partición *holdout*

- `train_test_split()` separa datos de forma aleatoria pero reproducible.
- El árbol se entrena con criterio `entropy` (similar a ganancia de información).
- Se limita la profundidad para evitar sobreajuste (`max_depth = 3`).

```
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier

# 80% entrenamiento, 20% prueba
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Crear y entrenar el modelo
modelo_dt = DecisionTreeClassifier(criterion='entropy', max_depth=3,
                                   random_state=42)
modelo_dt.fit(X_train, y_train)
```



Evaluación con conjunto de prueba

```
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.metrics import classification_report

# Predecir sobre el conjunto de prueba
y_pred = modelo_dt.predict(X_test)

acc = accuracy_score(y_test, y_pred)
print(f"Exactitud: {acc:.2f}")

print("Matriz de confusión:")
print(confusion_matrix(y_test, y_pred))

print("Reporte de clasificación:")
print(classification_report(y_test, y_pred,
                           target_names=['no_esperar', 'esperar']))
```



Evaluación con validación cruzada

```
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import cross_val_predict

# Exactitud, precisión y recall en 5 folds
acc = cross_val_score(modelo_dt, X, y, cv=5, scoring='accuracy')
prec = cross_val_score(modelo_dt, X, y, cv=5, scoring='precision')
rec = cross_val_score(modelo_dt, X, y, cv=5, scoring='recall')

print(f"Exactitud media: {acc.mean():.4f}")
print(f"Precisión media: {prec.mean():.4f}")
print(f"Sensibilidad media: {rec.mean():.4f}")

# Matriz de confusión global con CV
y_cv_pred = cross_val_predict(clf, X, y, cv=5)
print("Matriz de confusión (validación cruzada):")
print(confusion_matrix(y, y_cv_pred))
```



Predicción de un nuevo ejemplo

```
# Nuevo ejemplo: no, si, no, algunos, 0-10
nuevo = np.array([[
    encoders['Alternativa'].transform(['no'])[0],
    encoders['Bar'].transform(['si'])[0],
    encoders['Hambriento'].transform(['no'])[0],
    encoders['Clientes'].transform(['algunos'])[0],
    encoders['Tiempo'].transform(['0-10'])[0]
]])

clase_pred = modelo_dt.predict(nuevo)[0]
clase_nombre = encoders['Esperar'].inverse_transform([clase_pred])[0]
proba = modelo_dt.predict_proba(nuevo)[0]

print(f"Clase predicha: {clase_nombre}")
print(f"Probabilidad de no_esperar: {proba[0]:.4f}")
print(f"Probabilidad de esperar: {proba[1]:.4f}")
```



Tarea 7: Ajuste de un árbol de decisión

- Selecciona un conjunto de datos para clasificación de un repositorio.
- Realiza un análisis exploratorio:
 - Primeras filas, dimensiones, tipos de datos y distribución de la clase.
- Selecciona y codifica al menos cinco atributos categóricos y una clase.
- Divide los datos en entrenamiento (70%) y prueba (30%).
- Ajusta un árbol de decisión con el entrenamiento.
- Evalúa el modelo con:
 - matriz de confusión, exactitud, precisión y sensibilidad.
- Aplica validación cruzada de 5 *folds* para obtener las mismas métricas.
- Compara los resultados de la partición *holdout* y la validación cruzada.



Aprendizaje no supervisado

- A diferencia del supervisado, no se dispone de etiquetas conocidas.
- Los datos solo contienen variables de entrada (atributos).
- El objetivo es descubrir patrones, estructuras o relaciones ocultas.
- Se busca entender la organización natural de los datos.



Tipos de tareas no supervisadas

Tarea	Descripción	Ejemplo
Agrupación (<i>clustering</i>)	Dividir los datos en grupos (clústeres) según su similitud.	Segmentar clientes por comportamiento.
Reducción de dimensionalidad	Proyectar datos de alta dimensión a un espacio de menor dimensión.	Visualizar datos con PCA o t-SNE.
Estimación de densidad	Modelar la distribución de probabilidad de los datos.	Detección de anomalías.
Reglas de asociación	Encontrar relaciones entre atributos.	Análisis de compras.

- En este curso nos centraremos en **agrupación**.
- El algoritmo más representativo es ***k-means***.



Agrupación (*clustering*)

- Un clúster es un conjunto de ejemplos similares entre sí.
 - Los ejemplos de diferentes clústeres deben ser disímiles.
 - No hay etiquetas previas; el algoritmo debe descubrir los grupos.
 - La similitud se mide con distancias (euclidiana, Manhattan, etc.).
-
- **Aplicaciones en ciberseguridad:**
 - Detección de anomalías: puntos alejados de todos los clústeres.
 - Clasificación de malware: agrupar comportamientos similares.
 - Segmentación de tráfico de red: identificar patrones sospechosos.



Algoritmo *k*-means

- Objetivo: **particionar los datos en k clústeres (k es un parámetro fijo).**
- **Cada clúster se representa por su centroide (promedio de sus puntos).**
- **Asigna cada punto al centroide más cercano.**

Entradas: **ejemplos** – conjunto de ejemplos, **k** – número de grupos

Salidas: **grupos** – ejemplos agrupados, **centroides** – media de los datos

- 01 Inicializar k centroides aleatoriamente (o con algún método).
- 02 Repetir hasta convergencia:
 - 03 Asignar cada punto al centroide más cercano (distancia euclidiana).
 - 04 Recalcular cada centroide como la media de los puntos asignados a él.
- 05 Terminar cuando las asignaciones ya no cambian o los centroides se estabilizan.



Medidas de similitud entre puntos

Distancia euclidiana:

- Es la distancia en línea recta entre dos puntos (teorema de Pitágoras).

$$\text{euclidiana}(p, q) = \sqrt{\sum_d (p_d - q_d)^2}$$

Distancia Manhattan

- También llamada distancia L1 o distancia de ciudad bloque.

$$\text{manhattan}(p, q) = \sum_d |p_d - q_d|$$

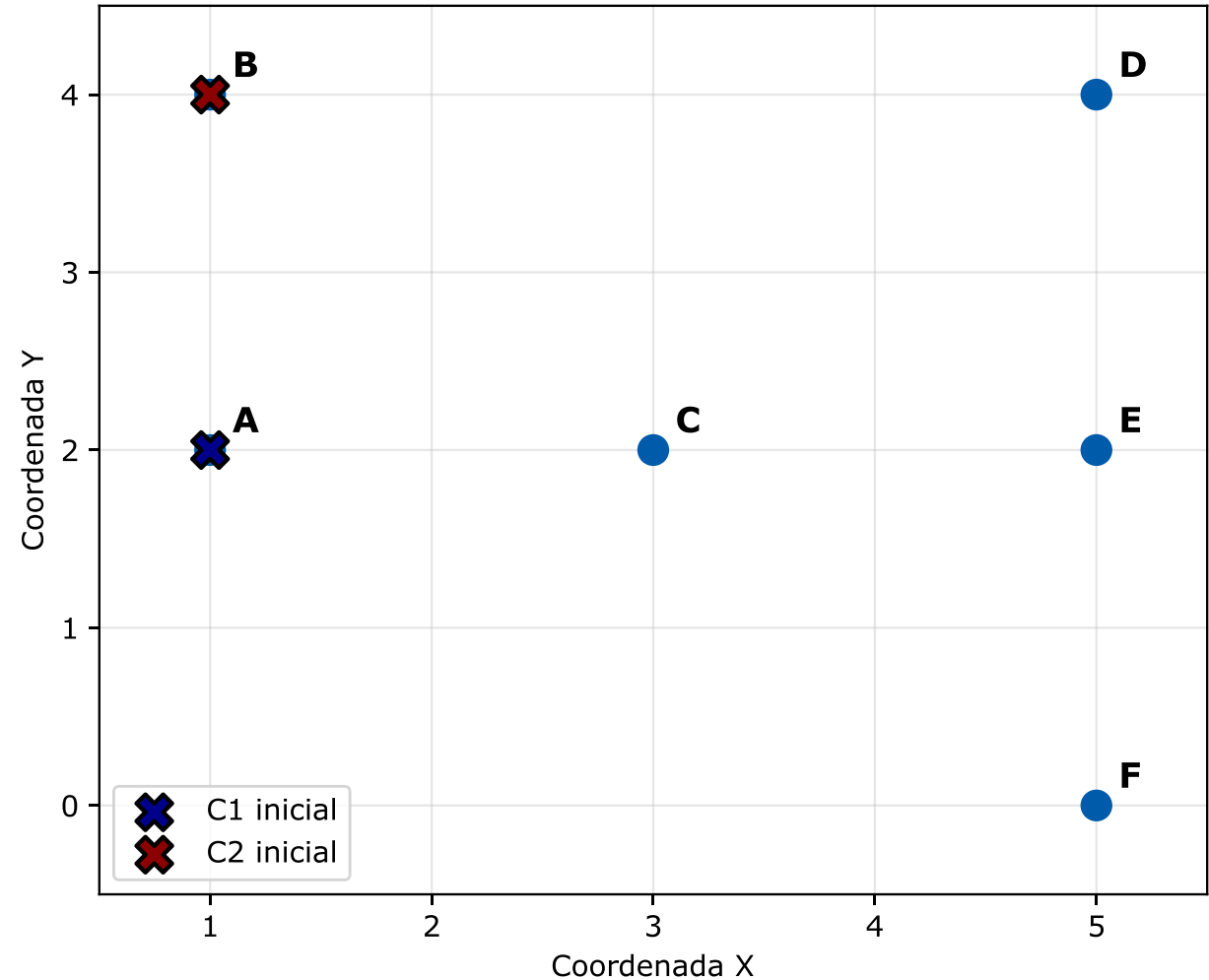


Ejemplo de ejecución de *K-means*

Datos para el ejercicio:

- $K = 2$
- $C1 = (1,2)$, $C2 = (1,4)$

ID	X	Y
A	1	2
B	1	4
C	3	2
D	5	4
E	5	2
F	5	0

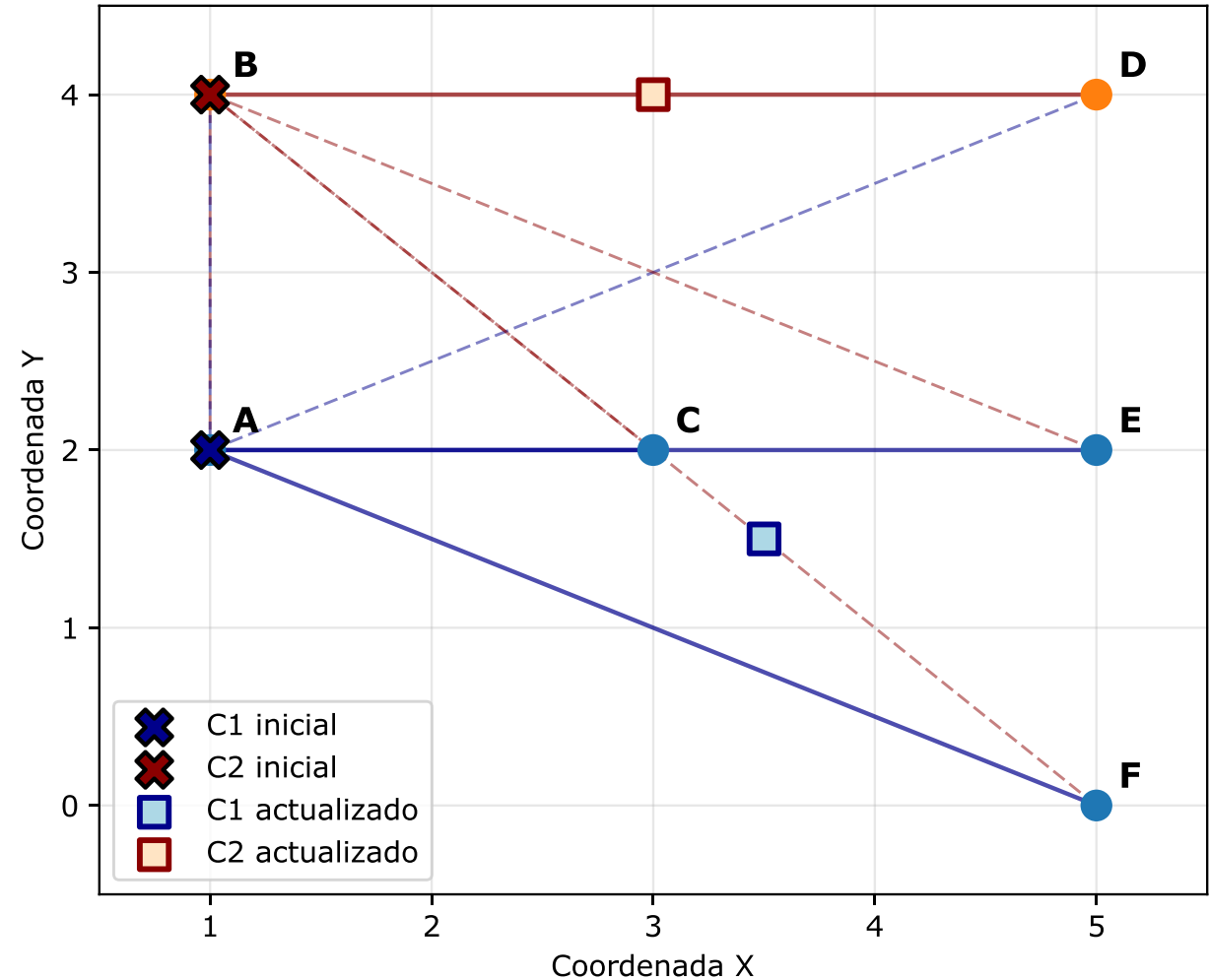


Iteración I

- $C1 = (1,2)$, $C2 = (1,4)$

Punto	$d(C1, p)$	$d(C2, p)$	Grupo
A(1,2)	0.00	2.00	C1
B(1,4)	2.00	0.00	C2
C(3,2)	2.00	2.83	C1
D(5,4)	4.47	4.00	C2
E(5,2)	4.00	4.47	C1
F(5,0)	4.47	5.66	C1

- $C1 = (3.5, 1.5)$, $C2 = (3, 4)$

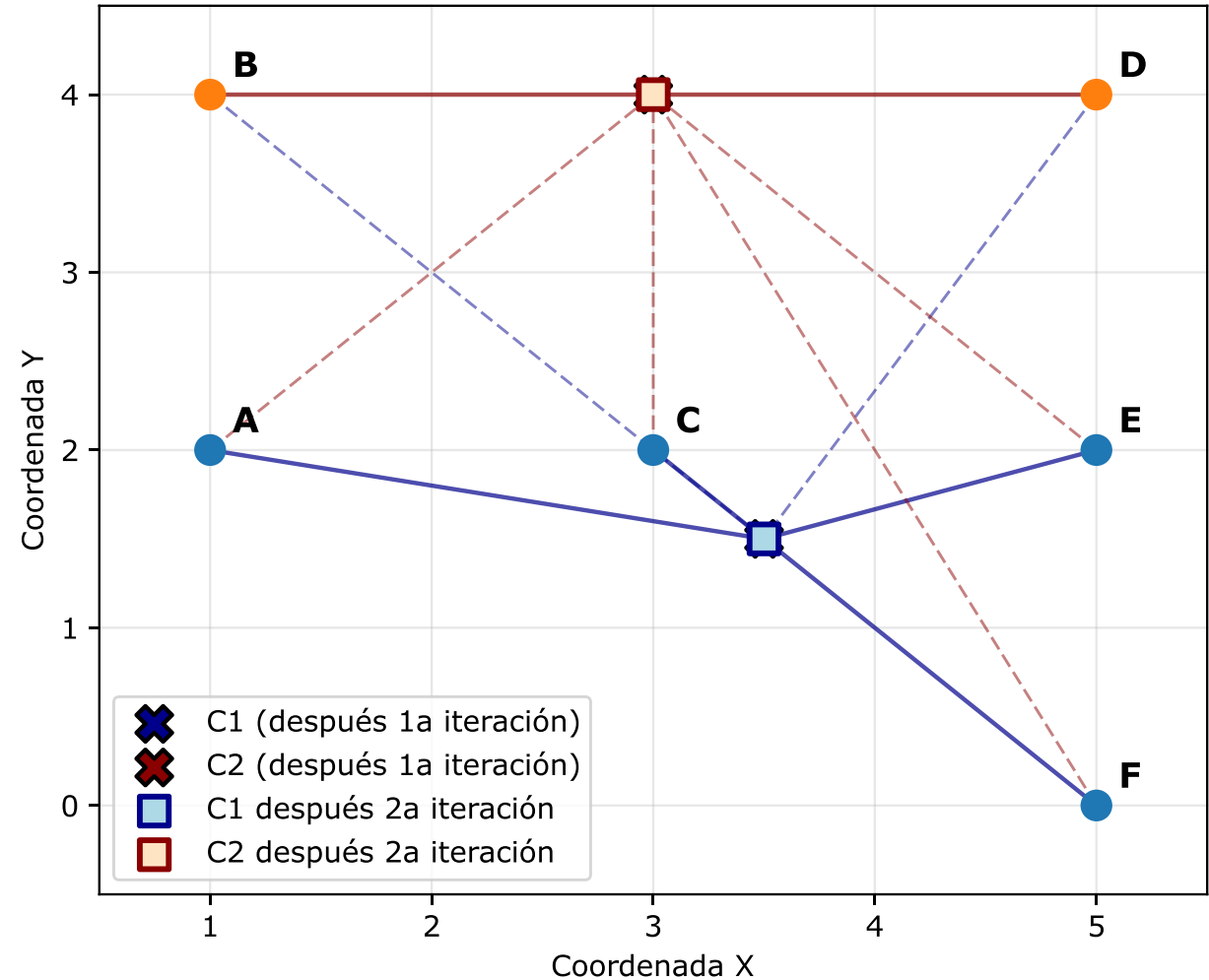


Iteración 2

- $C1 = (3.5, 1.5)$, $C2 = (3, 4)$

Punto	$d(C1, p)$	$d(C2, p)$	Grupo
A(1,2)	2.55	2.83	C1
B(1,4)	3.54	2.00	C2
C(3,2)	0.71	2.00	C1
D(5,4)	2.92	2.00	C2
E(5,2)	1.58	2.83	C1
F(5,0)	2.12	4.47	C1

- $C1 = (3.5, 1.5)$, $C2 = (3, 4)$



Métricas de evaluación – Inercia

Inercia

- Suma de la distancia euclidiana al cuadrado de cada punto a su centroide.
- Mide qué tan compactos son los clústeres.
- Menor inercia indica clústeres más apretados.
- *k-means* minimiza la inercia.

$$\text{Inercia} = \sum_{i=1}^k \sum_{p \in C_i} \|p - c_i\|^2$$



Ejemplo del cálculo de la inercia

Distancia entre el centroide (3.5, 1.5) y los puntos del clúster 1	
A (1, 2)	6.5
C (3, 2)	0.5
E (5, 2)	2.5
F (5, 0)	4.5
SUMA	14.0

Distancia entre el centroide (3, 4) y los puntos del clúster 2	
B (1, 4)	4.0
D (5, 4)	5.0
SUMA	8.0

INERCIA	22.0
----------------	-------------



Método del codo

- *k-means* minimiza la inercia, la suma de distancias al cuadrado.
- Al aumentar k , la inercia siempre disminuye.
- Entre más clústeres, los puntos tienden a estar más cerca de su centroide.
- Pero a partir de cierto k , la mejora se vuelve muy pequeña.
- El "codo" es el punto donde la ganancia en inercia se estabiliza.
- Ese k suele ser una buena elección.

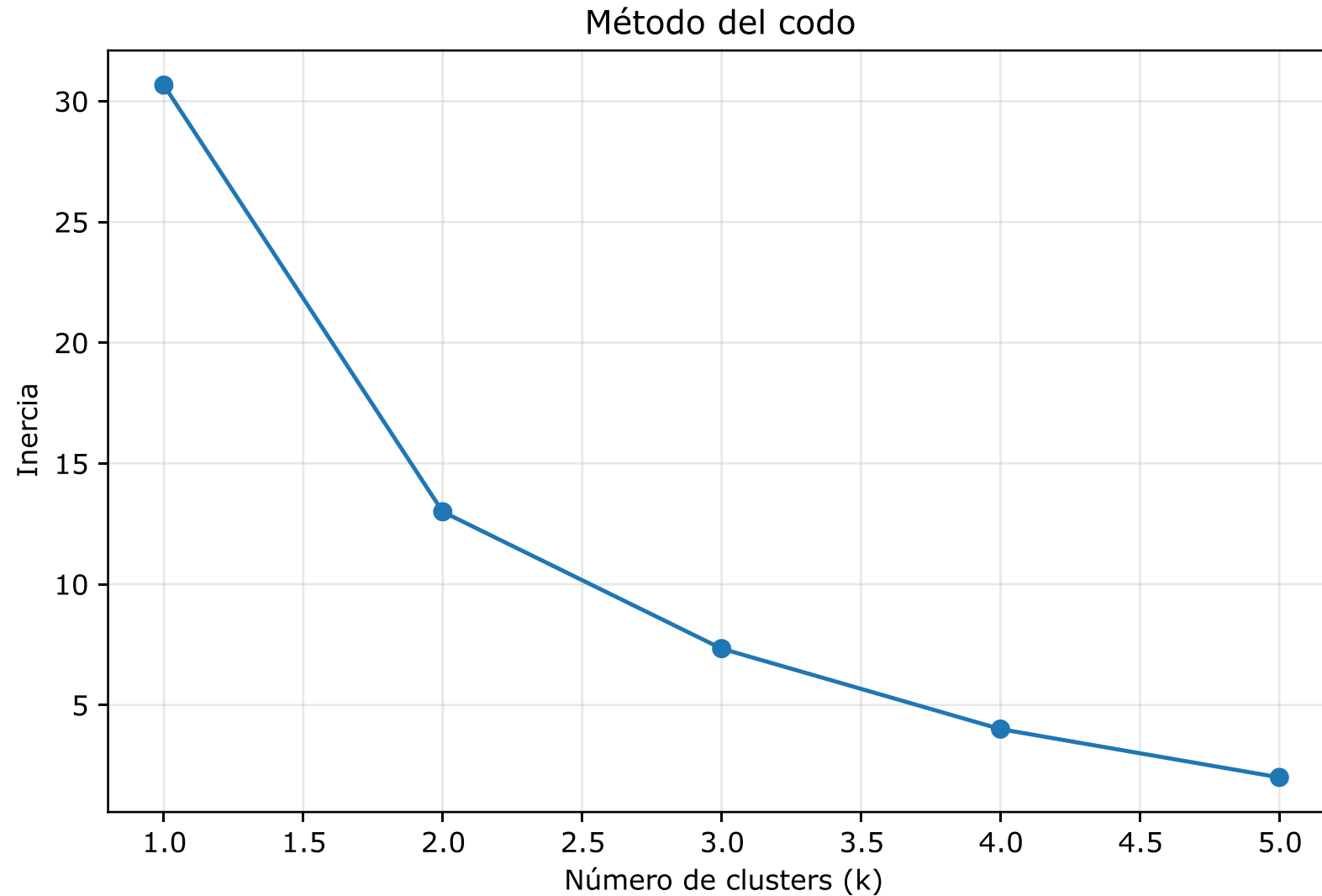


Procedimiento del método del codo

1. Ejecutar *k-means* para k , desde 1 hasta $K_{\max}$.
 - $K_{\max}$ es un valor límite práctico, como la raíz del número de puntos.
 - No es necesario llegar al número total de puntos.
2. Para cada valor de k , registrar la inercia final del modelo.
3. Graficar k (eje X) frente a la inercia (eje Y).
4. Localizar el punto donde la curva cambia bruscamente de pendiente.
 - Ese punto es el "codo" y sugiere el número adecuado de clústeres.



Ejemplo del método del codo



Agrupación de dígitos manuscritos con k-means

- El conjunto contiene 1797 imágenes de dígitos escritos a mano.
- Cada imagen se representa con 64 valores de gris (0 a 16).

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

digits = load_digits()
X = digits.data          # 1797 x 64
y = digits.target        # Etiquetas reales

print("Forma de X:", X.shape)
print("Número de clusters reales:", len(np.unique(y)))
```



Preprocesamiento y ajuste

- *k-means* es sensible a la escala de las variables.
- Estandarizar evita que ciertas variables dominen.
- Se elige $k=10$ porque se sabe que hay 10 dígitos.
- En un caso real sin etiquetas, se usaría el método del codo.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=10, random_state=42, n_init=10)
kmeans.fit(X_scaled)

print("Inercia final:", kmeans.inertia_)
print("Centroides forma:", kmeans.cluster_centers_.shape)
```



Visualización de los centroides como imágenes I

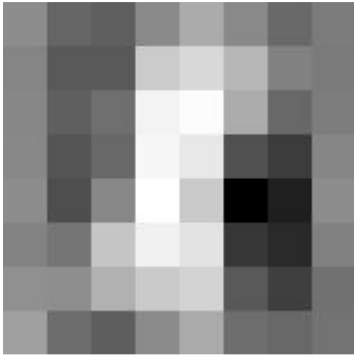
- Cada centroide es el "dígito promedio" de su clúster.
- Al convertirlo en imagen, se puede identificar el dígito que representa.
- Es probable que algunos centroides se parezcan a dígitos reales.

```
fig, axes = plt.subplots(2, 5, figsize=(10, 4))
for i, ax in enumerate(axes.flat):
    # Reconstruir imagen 8x8 desde el centroide
    ax.imshow(kmeans.cluster_centers_[i].reshape(8, 8), cmap='gray')
    ax.set_title(f'Centroide {i}')
    ax.axis('off')
plt.tight_layout()
plt.show()
```

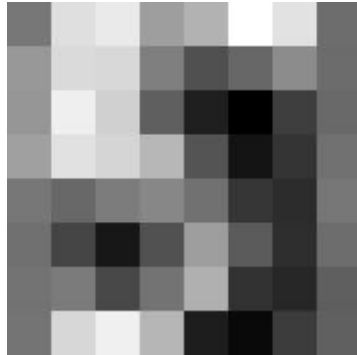


Visualización de los centroides como imágenes II

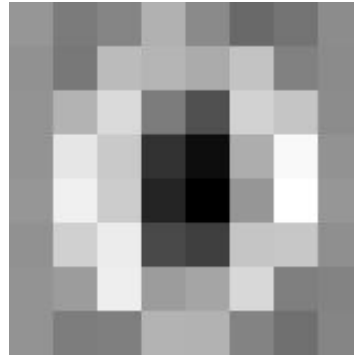
Centroide 0



Centroide 1



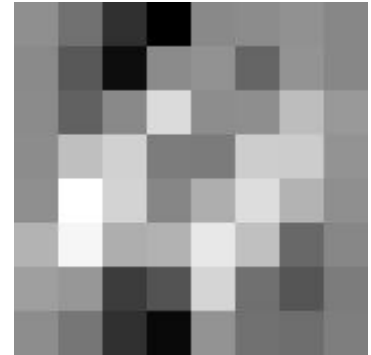
Centroide 2



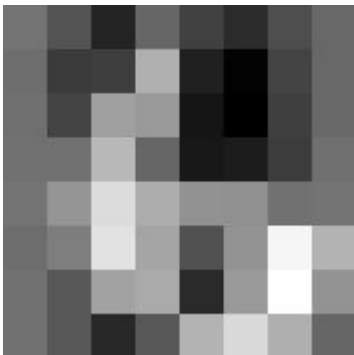
Centroide 3



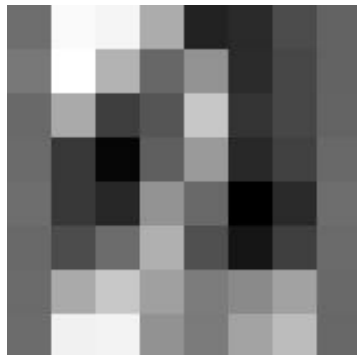
Centroide 4



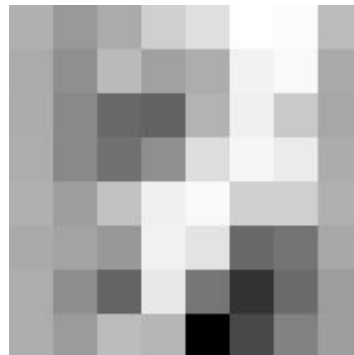
Centroide 5



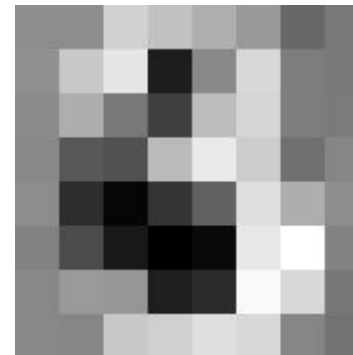
Centroide 6



Centroide 7



Centroide 8



Centroide 9



Método del codo para seleccionar k

- Se observa un codo alrededor de $k=10$.
- Coincide con el número real de dígitos.
- A partir de $k=10$, la ganancia en inercia se vuelve pequeña.

```
inertias = []
for k in range(1, 50):
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)

plt.plot(range(1, 50), inertias, marker='o')
plt.xlabel('k (número de clústeres)')
plt.ylabel('Inercia')
plt.grid(True, alpha=0.3)
plt.show()
```



Método del codo para seleccionar k II

